

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



Phạm Quang Minh

**GIẢI PHÁP GIAO TIẾP TRỰC TUYẾN CHO NGƯỜI
KHIÊM THÍNH**

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Công nghệ thông tin

HÀ NỘI - 2026

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



PHẠM QUANG MINH

**GIẢI PHÁP GIAO TIẾP TRỰC TUYẾN CHO NGƯỜI
KHIÊM THÍNH**

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Công nghệ thông tin

Cán bộ hướng dẫn: ThS. Trần Mạnh Cường

Cán bộ đồng hướng dẫn: TS. Nguyễn Thị Thủy

HÀ NỘI - 2026

**VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**



Phạm Quang Minh

**ONLINE COMMUNICATION SOLUTION FOR THE DEAF AND
HARD-OF-HEARING COMMUNITY**

BACHELOR'S THESIS

Major: Information Technology

Supervisor: M.Sc. Trần Mạnh Cường

Co-Supervisor: Dr. Nguyễn Thị Thủy

HANOI - 2026

Lời cảm ơn

Lời đầu tiên, tôi xin được bày tỏ lòng biết ơn sâu sắc nhất đến ThS. Trần Mạnh Cường và TS. Nguyễn Thị Thủy, những người đã trực tiếp hướng dẫn, tận tình chỉ bảo và định hướng cho tôi trong suốt quá trình triển khai và hoàn thành đề tài khóa luận này. Những lời khuyên chuyên môn quý báu và sự tận tâm của thầy cô giúp tôi vượt qua rất nhiều những khó khăn trong thời gian qua.

Tôi cũng xin trân trọng gửi lời cảm ơn tới các thầy cô giáo tại Trường Đại học Công nghệ – Đại học Quốc gia Hà Nội. Trong suốt những năm tháng học tập dưới mái trường UET, tri thức và sự dìu dắt của các thầy cô đã cho tôi nền tảng chuyên môn vững chắc và những tư duy quý giá để bước vào đời.

Cuối cùng, con xin dành lời tri ân sâu sắc nhất đến Mẹ. Mẹ là điểm tựa tinh thần vững chắc, luôn hy sinh, thấu hiểu và tiếp thêm động lực cho con trên mọi chặng đường học tập và cuộc sống.

Dù đã dành nhiều tâm huyết, song khóa luận khó tránh khỏi những thiếu sót. Tôi rất mong nhận được những ý kiến đóng góp quý báu của các thầy cô để đề tài được hoàn thiện hơn.

Tôi xin chân thành cảm ơn!

Lời cam đoan

Tôi là Phạm Quang Minh, sinh viên lớp QH-2021-I/CQ-I-IT15. Tôi xin cam đoan khóa luận tốt nghiệp với đề tài “Giải pháp giao tiếp trực tuyến cho người khiếm thính” được thực hiện dựa trên sự nỗ lực nghiên cứu và thực hiện của cá nhân tôi dưới sự chỉ dẫn từ ThS. Trần Mạnh Cường và sự giúp đỡ của TS. Nguyễn Thị Thủy. Toàn bộ số liệu và kết quả trình bày trong khóa luận hoàn toàn trung thực. Trong quá trình thực hiện, tôi có sử dụng AI để hỗ trợ trong quá trình nghiên cứu về ngôn ngữ ký hiệu, tìm hiểu về các hướng tối ưu, các công nghệ liên quan, cũng như kiểm tra chính tả và độ chặt chẽ của câu từ. Tuy nhiên, toàn bộ quá trình phân tích, thiết kế, phát triển, thử nghiệm và đánh giá đều do tôi thực hiện. Các tài liệu tham khảo được sử dụng trong khóa luận đều đã được trích dẫn và ghi nguồn rõ ràng theo đúng quy định. Tôi xin hoàn toàn chịu trách nhiệm về lời cam đoan này.

Hà Nội, ngày 20 tháng 6 năm 2026

Sinh viên

Minh

Phạm Quang Minh

Tóm tắt

Ngôn ngữ ký hiệu là phương tiện giao tiếp quan trọng của cộng đồng người điếc và khiếm thính, nhưng vẫn chưa được sử dụng rộng rãi bởi phần lớn người nghe. Điều này tạo ra nhu cầu phát triển các hệ thống hỗ trợ nhận dạng và dịch ngôn ngữ ký hiệu trong các tình huống giao tiếp trực tiếp. Khóa luận này xây dựng một nguyên mẫu hệ thống nhận dạng ngôn ngữ ký hiệu liên tục theo hướng trực tuyến trong môi trường gọi video. Hệ thống kế thừa phương pháp của Zuo và cộng sự, trong đó mô hình nhận dạng ký hiệu đơn lẻ được áp dụng trên các cửa sổ thời gian của luồng video đầu vào, sau đó kết quả được hậu xử lý để tạo chuỗi gloss đầu ra. Trên cơ sở đó, khóa luận tổ chức lại pipeline để xử lý trực tiếp luồng video, sau đó tích hợp pipeline vào một hệ thống bao gồm ứng dụng gọi video sử dụng LiveKit, Bot server và Server suy luận triển khai bằng Triton Inference Server. Các thực nghiệm được thực hiện nhằm đánh giá ảnh hưởng của việc thay thế mô hình phát hiện đối tượng và ước lượng tư thế đến độ chính xác, đồng thời đánh giá độ trễ của pipeline trong điều kiện giảm thiểu chi phí giao tiếp client-server. Kết quả cho thấy cấu hình sử dụng mô hình phát hiện đối tượng và ước lượng tư thế nhanh hơn làm tăng WER ở mức vừa phải so với cấu hình gốc, nhưng vẫn duy trì chất lượng nhận dạng tương đối tốt so với các mốc tham khảo. Khi triển khai client thử nghiệm cùng máy với Triton trên RTX 4090, pipeline đạt tốc độ xử lý xấp xỉ thời gian thực với RTF xấp xỉ 1 và độ trễ trung bình khoảng 1,36 giây. Kết quả này cho thấy tính khả thi ban đầu của việc tích hợp pipeline CSLR trực tuyến vào môi trường gọi video, đồng thời chỉ ra các giới hạn hiện tại về độ trễ giao tiếp, khả năng tổng quát hóa và phạm vi đánh giá nhiều luồng đồng thời.

Từ khóa: Nhận dạng ngôn ngữ ký hiệu liên tục, CSLR trực tuyến, nhận dạng gần thời gian thực, cửa sổ trượt, server suy luận, gọi video, độ trễ.

Abstract

Sign language is an important means of communication for Deaf and hard-of-hearing communities, yet it is not widely understood by most hearing people. This creates a need for systems that can support sign language recognition and translation in interactive communication scenarios. This thesis presents a prototype system for online continuous sign language recognition in a video calling environment. The system builds upon the method proposed by Zuo et al., where an isolated sign language recognition model is applied to short sliding windows of the input video stream, followed by post-processing to produce an output gloss sequence. Based on this approach, the thesis reorganizes the recognition pipeline to process video streams directly and integrates it with a video calling application, a Bot server, and an inference server deployed with NVIDIA Triton Inference Server. The experiments evaluate the impact of replacing the original object detection and pose estimation models with faster alternatives, as well as the latency of the pipeline when client-server communication overhead is minimized. The results show that replacing the object detection and pose estimation models moderately increases WER compared with the original configuration, while still maintaining relatively good recognition quality compared with reference baselines. When the test client and Triton are deployed on the same RTX 4090 machine, the pipeline achieves near-real-time processing with an RTF close to 1 and an average latency of approximately 1.36 seconds. These results demonstrate the initial feasibility of integrating an online CSLR pipeline into a video calling environment, while also highlighting current limitations related to communication latency, generalization, and evaluation under multiple concurrent streams.

Keywords: Continuous Sign Language Recognition, online CSLR, near-real-time recognition, sliding window, inference server, video calling, latency.

Mục lục

Lời cảm ơn

Lời cam đoan i

Tóm tắt ii

Abstract iii

Mục lục iv

Danh sách hình vẽ viii

Danh sách bảng ix

Danh mục các từ viết tắt x

Chương 1 Đặt vấn đề 1

1.1 Bối cảnh và động lực 1

1.2 Mục tiêu và phạm vi 3

1.3 Đóng góp của khóa luận 4

1.4 Cấu trúc khóa luận 5

Chương 2 Nền tảng, nghiên cứu và công nghệ liên quan 6

2.1 Cơ sở lý thuyết và nghiên cứu liên quan 6

2.1.1 Ngôn ngữ ký hiệu 6

2.1.2 So sánh CSLR và ISLR 6

2.1.3 Gloss 7

2.1.4 Các tập dữ liệu ngôn ngữ ký hiệu 7

2.1.5 Nhận dạng và dịch ngôn ngữ ký hiệu liên tục trực tuyến 9

2.1.6 Các giải pháp hỗ trợ giao tiếp hiện có 10

2.1.7 Ước lượng tư thế và phát hiện đối tượng 12

2.1.8	Phương pháp CSLR trực tuyến của Zuo và cộng sự	12
2.1.9	Mô hình producer-consumer	16
2.1.10	Server suy luận và triển khai mô hình học sâu	17
2.1.11	Suy luận theo batch	18
2.1.12	Cơ chế thực thi suy luận trên GPU sử dụng CUDA	18
2.2	Các công cụ và công nghệ được sử dụng	20
2.2.1	Triton Inference Server và các công cụ hỗ trợ triển khai suy luận	20
2.2.2	LiveKit và các công cụ hỗ trợ truyền thông thời gian thực . . .	21
2.2.3	Các thư viện OpenMMLab và các mô hình Det/Pose	22
2.2.4	Flutter	22
2.2.5	WebSocket	23
2.2.6	FastAPI	23
2.2.7	PyTorch và TensorRT	24
2.2.8	RTX 3090 và RTX 4090	24
Chương 3	Phân tích và thiết kế hệ thống	25
3.1	Phân tích yêu cầu hệ thống	25
3.1.1	Yêu cầu chức năng	25
3.1.2	Yêu cầu phi chức năng	26
3.1.3	Ràng buộc và phạm vi hệ thống	28
3.2	Thiết kế kiến trúc tổng thể	29
3.2.1	Các thành phần của hệ thống	30
3.2.2	Luồng xử lý chính của hệ thống	32
3.3	Thiết kế các luồng xử lý của hệ thống	32
3.3.1	Các luồng tham gia và rời phòng gọi	32
3.3.2	Các luồng điều khiển phiên nhận dạng	33
3.3.3	Luồng suy luận từ Bot server đến Server suy luận	36
3.3.4	Luồng trả kết quả gloss về frontend	39
3.4	Thiết kế tổ chức pipeline nhận dạng	40
3.4.1	Tổ chức pipeline theo mô hình producer-consumer	40
3.4.2	Vị trí các bước xử lý	41
3.4.3	Thiết kế buffer trung gian và xử lý theo batch	41
3.5	Thiết kế lập lịch và quản lý thực thi trong Server suy luận	43
3.5.1	Thiết kế của phiên bản Server suy luận tự xây dựng	43

3.5.2	Hạn chế của thiết kế ban đầu	45
3.5.3	Cơ chế lập lịch và quản lý của Triton Inference Server	48
3.5.4	Lý do chuyển sang sử dụng Triton	50
3.6	Thiết kế tổ chức suy luận trên Triton	50
3.6.1	Phương án phục vụ từng mô hình riêng lẻ	50
3.6.2	Phương án phục vụ toàn bộ pipeline	52
3.6.3	So sánh hai phương án thiết kế	53
3.7	Thiết kế điều khiển phiên nhận dạng	54
3.8	Thiết kế điều phối suy luận của Bot server	55
3.9	Thiết kế kiến trúc phân lớp của ứng dụng frontend	55
3.10	Thiết kế kiểm soát độ trễ	56
3.10.1	Nguồn phát sinh độ trễ trong hệ thống	56
3.10.2	Độ trễ tích lũy trong pipeline nhận dạng	57
3.10.3	Các quyết định thiết kế nhằm giảm độ trễ	57
3.10.4	Cân bằng giữa độ trễ, độ ổn định và độ chính xác	58
Chương 4	Cài đặt, thực nghiệm và đánh giá hệ thống	60
4.1	Phiên bản hiện thực ban đầu của pipeline nhận dạng	60
4.2	Thử nghiệm ban đầu	61
4.2.1	Phương pháp đo độ trễ	62
4.2.2	Kết quả thử nghiệm	62
4.3	Phân tích và tối ưu độ trễ	63
4.3.1	Thiết lập baseline mới với YOLOv3 và ResNet50	63
4.3.2	Xử lý hiện tượng độ trễ tích lũy	65
4.3.3	Tăng throughput bằng kích cỡ batch và CUDA stream	66
4.3.4	Tăng năng lực xử lý bằng GPU mạnh hơn	66
4.3.5	Giảm số cửa sổ cần xử lý bằng cách tăng stride	66
4.3.6	Hai cấu hình tối ưu đạt độ trễ ổn định	67
4.4	Cài đặt phiên bản Server suy luận tự xây dựng	68
4.5	Triển khai Triton theo phương án phục vụ từng mô hình riêng lẻ	68
4.5.1	Đóng gói các mô hình	68
4.5.2	Vấn đề độ trễ giao tiếp khi triển khai từ xa	69
4.5.3	Đánh giá độ trễ pipeline khi giảm thiểu chi phí client-server	72
4.6	Cài đặt ứng dụng frontend và Bot server	74

Kết luận	76
Tài liệu tham khảo	78

Danh sách hình vẽ

2.1	Pipeline CSLR trực tuyến	14
2.2	Mô hình producer-consumer	16
2.3	So sánh giữa khi hiện tượng chồng lấn không xảy ra và khi nó có xảy ra	19
3.1	Biểu đồ ca sử dụng của hệ thống	27
3.2	Kiến trúc tổng thể của hệ thống	30
3.3	Biểu đồ tuần tự của ca sử dụng <i>Tham gia phòng gọi</i>	33
3.4	Biểu đồ tuần tự của ca sử dụng <i>Rời phòng gọi</i>	34
3.5	Biểu đồ tuần tự của ca sử dụng <i>Bắt đầu phiên nhận dạng</i>	35
3.6	Biểu đồ tuần tự của các ca sử dụng điều khiển phiên nhận dạng . . .	36
3.7	Biểu đồ trạng thái của một phiên nhận dạng	36
3.8	Biểu đồ tuần tự luồng suy luận theo phương án Server suy luận phục vụ từng mô hình riêng lẻ	37
3.9	Biểu đồ tuần tự luồng suy luận theo phương án Server suy luận chứa toàn bộ pipeline	38
3.10	Biểu đồ tuần tự luồng trả kết quả gloss về frontend	39
3.11	Thiết kế của Server suy luận tự xây dựng	44
3.12	So sánh thiết kế hướng client và thiết kế hướng tài nguyên	46
3.13	Phương án Triton chỉ phục vụ các mô hình học sâu	51
3.14	Phương án Triton phục vụ toàn bộ pipeline	52
4.1	Độ trễ theo thời gian của cấu hình thử nghiệm đầu	62
4.2	So sánh độ trễ theo thời gian giữa cấu hình thay Det/Pose và hai cấu hình đặt độ trễ ổn định	63
4.3	Độ trễ theo thời gian khi triển khai Bot server và Triton xa nhau . .	69
4.4	Màn hình phòng gọi video trong một phiên nhận dạng ở nền tảng Web và Android	75

Danh sách bảng

3.1	Các yêu cầu phi chức năng của hệ thống	28
3.2	Vị trí các bước xử lý trong hai phương án tổ chức pipeline	41
3.3	So sánh hai phương án tổ chức suy luận trên Triton	54
4.1	So sánh WER giữa cấu hình gốc, cấu hình thay mô hình Det/Pose và một số phương pháp offline được áp dụng cơ chế cửa sổ trượt . . .	64
4.2	Hai cấu hình có độ trễ ổn định	67
4.3	Kết quả đo thời gian xử lý server và độ trễ giao tiếp ước lượng khi gọi Triton từ xa	71
4.4	Kết quả đo tốc độ xử lý và độ trễ của pipeline	73

Danh mục các từ viết tắt

Từ viết tắt	Cụm từ đầy đủ	Ý nghĩa / Mô tả
NNKH	Ngôn ngữ ký hiệu	Ngôn ngữ cử chỉ – thị giác của người khiếm thính
SLR	Sign Language Recognition	Nhận dạng ngôn ngữ ký hiệu
SLT	Sign Language Translation	Dịch ngôn ngữ ký hiệu
CSLR	Continuous Sign Language Recognition	Nhận dạng ngôn ngữ ký hiệu liên tục
ISLR	Isolated Sign Language Recognition	Nhận dạng ngôn ngữ ký hiệu đơn lẻ
Det	Object Detection	Phát hiện đối tượng
Pose	Pose Estimation	Ước lượng tư thế
ME	Movement Epenthesis	Cử động thêm hay cử động chuyển tiếp

Chương 1

Đặt vấn đề

1.1 Bối cảnh và động lực

Ngôn ngữ ký hiệu (NNKH) là ngôn ngữ tự nhiên được truyền tải qua cử động của tay, nét mặt, đầu và thân. Khác với cách hiểu lầm thường gặp, mỗi ngôn ngữ ký hiệu không phải là việc biểu diễn từng từ của một ngôn ngữ nói bằng ký hiệu, mà là ngôn ngữ độc lập với cấu trúc, ngữ pháp và quy ước riêng [1]. Giống như các ngôn ngữ tự nhiên khác, ngôn ngữ ký hiệu không phải là một ngôn ngữ chung toàn cầu. Mặc dù được sử dụng làm phương tiện giao tiếp chính của người khiếm thính, ngôn ngữ ký hiệu vẫn chưa được biết đến và sử dụng bởi hầu hết người nghe bình thường [2, 3]. Nhiều nỗ lực nghiên cứu đã được thực hiện với mong muốn làm giảm rào cản ngôn ngữ này, trong đó có các nghiên cứu về nhận dạng và dịch ngôn ngữ ký hiệu.

Nhận dạng ngôn ngữ ký hiệu (*Sign Language Recognition* – SLR) là bài toán xác định các ký hiệu có trong các cử chỉ đầu vào. Dịch ngôn ngữ ký hiệu (*Sign Language Translation* – SLT) hướng đến việc chuyển chuỗi ký hiệu đầu vào sang một câu văn hoàn chỉnh, đúng ngữ pháp và tương đương về mặt ý nghĩa trong một ngôn ngữ nói. Khóa luận này tập trung vào hướng tiếp cận dựa trên dữ liệu thị giác, trong đó đầu vào là luồng video của người ký hiệu. Đầu ra của SLR thường là gloss, một dạng biểu diễn dưới dạng chữ viết của ký hiệu.

Nhận dạng ngôn ngữ ký hiệu thường được chia thành hai loại chính là nhận dạng ngôn ngữ ký hiệu *đơn lẻ* (*Isolated Sign Language Recognition* – ISLR) và nhận dạng ngôn ngữ ký hiệu *liên tục* (*Continuous Sign Language Recognition* – CSLR). Trong ISLR, đầu vào thường chỉ gồm một ký hiệu, tương ứng với việc người ký hiệu thực hiện một ký hiệu riêng biệt rồi dừng lại. Ngược lại, trong CSLR, đầu vào là một chuỗi ký hiệu được thực hiện liên tiếp, gần với cách ngôn ngữ ký hiệu được sử dụng trong giao tiếp tự nhiên. Có thể hình dung gần đúng rằng một ký hiệu đơn lẻ tương tự như một từ trong ngôn ngữ nói, còn một chuỗi hoặc một câu ký hiệu liên tục tương ứng với một câu nói trong hội thoại.

CSLR khó hơn ISLR vì hệ thống không chỉ cần nhận dạng từng ký hiệu riêng lẻ, mà còn phải xử lý các cử động chuyển tiếp giữa các ký hiệu, sự ảnh hưởng lẫn nhau giữa các ký hiệu liên kề và ngữ cảnh được tạo ra khi nhiều ký hiệu kết hợp thành một câu hoàn chỉnh. Ngoài ra, ISLR thường được xem là bài toán phân loại có giám sát đầy đủ, trong đó mỗi mẫu video được gán trực tiếp với một nhãn ký hiệu. Ngược lại, các bộ dữ liệu CSLR hiện tại chủ yếu chỉ cung cấp chuỗi nhãn ký hiệu tương ứng với toàn bộ câu ký hiệu mà không phân định rõ thời điểm bắt đầu và kết thúc của từng ký hiệu. Vì vậy, CSLR thường được xem là một bài toán học có giám sát yếu [4, 5].

Hiện nay, nghiên cứu về SLR và SLT đã đạt được nhiều tiến triển đáng kể. Tuy nhiên, lĩnh vực này vẫn đang đối mặt với nhiều thách thức, bao gồm quy mô và mức độ đa dạng còn hạn chế của các bộ dữ liệu [6–8], khó khăn trong việc mô hình hóa các đặc trưng ngôn ngữ phức tạp của ngôn ngữ ký hiệu [9–12], cũng như các yêu cầu khắt khe về độ chính xác, độ trễ, tính ổn định và khả năng hoạt động trong môi trường thực tế [8, 11, 13]. Trong bối cảnh đó, việc phát triển các hệ thống nhận dạng và dịch ngôn ngữ ký hiệu liên tục theo hướng trực tuyến, thời gian thực hoặc gần thời gian thực nhận được nhiều sự quan tâm nhờ tiềm năng ứng dụng trực tiếp trong giao tiếp giữa người khiếm thính và người nghe [13]. Tuy nhiên, phần lớn các nghiên cứu SLR/SLT, đặc biệt là nhận dạng và dịch ngôn ngữ ký hiệu liên tục, đến nay vẫn chủ yếu được tiếp cận theo hướng ngoại tuyến [6, 8]. Trong các phương pháp này, mô hình thường cần tiếp nhận toàn bộ video đầu vào trước khi đưa ra kết quả nhận dạng hoặc dịch, dẫn đến độ trễ cao và làm hạn chế khả năng ứng dụng trong các tình huống giao tiếp trực tiếp [14]. Mặc dù một số nghiên cứu gần đây theo hướng xử lý trực tuyến đã cho thấy nhiều triển vọng, các kết quả hiện tại vẫn chưa đáp ứng đầy đủ các yêu cầu của môi trường thực tế [8, 15]. Về mặt triển khai, mặc dù đã xuất hiện một số sản phẩm thương mại, nguyên mẫu và hệ thống thử nghiệm liên quan đến nhận dạng hoặc dịch ngôn ngữ ký hiệu, phần lớn các hệ thống này thường bị giới hạn ở nhận dạng theo hướng ngoại tuyến, chỉ xử lý ký hiệu đơn lẻ, tập từ vựng hoặc miền ứng dụng hẹp, hoặc không công bố đủ thông tin kỹ thuật để xác minh khả năng xử lý ngôn ngữ ký hiệu liên tục theo thời gian thực/gần thời gian thực trong điều kiện thực tế.

Xuất phát từ thực trạng đó, khóa luận này xây dựng thử nghiệm một hệ thống CSLR trực tuyến gần thời gian thực, qua đó tìm hiểu các vấn đề kỹ thuật và giới

hạn thực tiễn khi phát triển một hệ thống nhận dạng ngôn ngữ ký hiệu liên tục trong môi trường gần với sử dụng thực tế. Trong phạm vi khóa luận, “gần thời gian thực” được hiểu là khả năng xử lý luồng video liên tục và trả kết quả nhận dạng với độ trễ dưới 5 giây trong các điều kiện thử nghiệm được mô tả ở các chương sau.

1.2 Mục tiêu và phạm vi

Phương pháp được sử dụng trong khóa luận này được kế thừa từ công trình của Zuo và cộng sự [14]. Thay vì đi theo hướng CSLR ngoại tuyến thường dựa trên CTC và xử lý toàn bộ video đầu vào trong quá trình suy luận, nhóm tác giả tiếp cận CSLR trực tuyến thông qua việc áp dụng mô hình ISLR trên các đoạn ngắn của video đầu vào theo cơ chế cửa sổ trượt. Cụ thể, phương pháp của họ xây dựng một từ điển ký hiệu từ dữ liệu CSLR, huấn luyện một mô hình ISLR trên từ điển đó, sau đó áp dụng mô hình ISLR lên luồng video liên tục để tạo ra dự đoán cho từng cửa sổ thời gian. Các kết quả dự đoán sau đó được hậu xử lý nhằm loại bỏ các dự đoán trùng lặp và các dự đoán thuộc lớp nền. Cách tiếp cận này phù hợp với mục tiêu của khóa luận vì nó cung cấp khả năng xử lý trực tuyến trên từng đoạn video ngắn thay vì phải chờ toàn bộ video, do đó tạo cơ sở để triển khai một hệ thống CSLR gần thời gian thực.

Trên cơ sở đó, khóa luận này kế thừa pipeline nhận dạng được đề xuất trong công trình trên làm thành phần nhận dạng cốt lõi cho backend của hệ thống. Khi triển khai backend dựa trên pipeline này, khóa luận không thực hiện lại giai đoạn xây dựng từ điển ký hiệu hoặc huấn luyện mô hình ISLR, mà sử dụng mô hình ISLR đã được nhóm tác giả huấn luyện sẵn trên tập dữ liệu RWTH-PHOENIX-Weather 2014T [16]. Trọng tâm của khóa luận nằm ở việc tích hợp mô hình vào pipeline suy luận trực tuyến, xử lý luồng video đầu vào, áp dụng cơ chế cửa sổ trượt, hậu xử lý kết quả dự đoán và trả kết quả nhận dạng cho frontend. Trong quá trình này, một trọng tâm kỹ thuật quan trọng là kiểm soát và tối ưu hóa độ trễ của toàn bộ pipeline, nhằm bảo đảm hệ thống có thể vận hành trong bối cảnh gần thời gian thực. Bên cạnh backend nhận dạng, khóa luận phát triển một ứng dụng frontend gọi video để thu nhận và truyền video, hiển thị kết quả nhận dạng, và phục vụ việc đánh giá khả năng vận hành của hệ thống trong bối cảnh gần thời gian thực.

Đối tượng sử dụng hướng đến của hệ thống là những người tham gia trong một cuộc gọi video có nhu cầu hỗ trợ giao tiếp bằng ngôn ngữ ký hiệu. Trong đó, một người tham gia có thể sử dụng tính năng nhận dạng bằng cách chia sẻ luồng video ký hiệu của mình; sau đó, hệ thống xử lý luồng video này và trả về kết quả nhận dạng dưới dạng gloss để những người tham gia khác trong phòng gọi có thể theo dõi.

Trong phạm vi khóa luận, hệ thống được xây dựng như một nguyên mẫu thử nghiệm nhằm đánh giá khả năng tích hợp pipeline nhận dạng ngôn ngữ ký hiệu liên tục vào môi trường gọi video, chưa hướng đến thay thế hoàn toàn người phiên dịch hoặc sử dụng trực tiếp trong các tình huống thực tế đòi hỏi độ chính xác cao. Hệ thống tập trung vào xử lý luồng video đầu vào theo hướng trực tuyến có độ trễ thấp và trả về kết quả nhận dạng dưới dạng gloss; chưa thực hiện dịch gloss sang câu ngôn ngữ tự nhiên. Khóa luận sử dụng mô hình nhận dạng đã huấn luyện sẵn, không đặt mục tiêu huấn luyện lại mô hình hoặc xây dựng một bộ dữ liệu ngôn ngữ ký hiệu mới. Khả năng nhận dạng của hệ thống cũng bị giới hạn bởi miền dữ liệu của tập dữ liệu được sử dụng trong huấn luyện mô hình nhận dạng.

1.3 Đóng góp của khóa luận

Việc đạt được những mục tiêu trên có ý nghĩa ở cả hai khía cạnh: ứng dụng và kỹ thuật. Về mặt ứng dụng, hệ thống nguyên mẫu cho thấy khả năng tích hợp pipeline nhận dạng ngôn ngữ ký hiệu liên tục vào một môi trường gọi video, qua đó tạo tiền đề cho các công cụ hỗ trợ giao tiếp giữa người sử dụng ngôn ngữ ký hiệu và người nghe trong các tình huống trực tuyến. Về mặt kỹ thuật, quá trình xây dựng hệ thống giúp làm rõ một số vấn đề phát sinh khi chuyển một phương pháp CSLR từ môi trường nghiên cứu sang một pipeline xử lý luồng gần thời gian thực, đặc biệt là các vấn đề về tổ chức pipeline, giao tiếp giữa các thành phần, triển khai mô hình suy luận và kiểm soát độ trễ. Do đó, kết quả của khóa luận có thể được xem là một bước thử nghiệm ban đầu, cung cấp thêm kinh nghiệm triển khai và cơ sở tham khảo cho các hướng phát triển CSLR/SLT trực tuyến trong tương lai.

Các đóng góp chính của khóa luận được tóm tắt như sau. Thứ nhất, khóa luận xây dựng một nguyên mẫu hệ thống hỗ trợ nhận dạng ngôn ngữ ký hiệu liên tục

trong môi trường gọi video, trong đó kết quả nhận dạng được trả về dưới dạng gloss và hiển thị cho những người tham gia phòng gọi. Thứ hai, khóa luận tổ chức lại pipeline nhận dạng được kế thừa từ công trình trước theo hướng xử lý trực tiếp luồng video đầu vào, bao gồm các bước phát hiện người ký hiệu, ước lượng tư thế, tạo cửa sổ thời gian, suy luận ISLR và hậu xử lý kết quả. Thứ ba, khóa luận thiết kế kiến trúc hệ thống gồm ứng dụng frontend, Server truyền thông thời gian thực, Bot Server và Server suy luận, qua đó tách biệt chức năng truyền video, điều phối xử lý và suy luận mô hình. Cuối cùng, khóa luận thực hiện một số thực nghiệm đánh giá ban đầu về độ chính xác và độ trễ, bao gồm đánh giá ảnh hưởng của việc thay thế mô hình phát hiện đối tượng và ước lượng tư thế đến WER và đánh giá độ trễ của pipeline khi giảm thiểu chi phí giao tiếp client-server.

1.4 Cấu trúc khóa luận

Phần còn lại của khóa luận được tổ chức như sau. Chương 2 trình bày các nền tảng lý thuyết, nghiên cứu liên quan và công nghệ được sử dụng trong khóa luận, bao gồm nhận dạng ngôn ngữ ký hiệu liên tục, pipeline nhận dạng được kế thừa, server suy luận và các công cụ triển khai hệ thống. Chương 3 phân tích yêu cầu và trình bày thiết kế hệ thống, bao gồm kiến trúc tổng thể, các luồng xử lý, cách tổ chức pipeline nhận dạng, thiết kế Server suy luận, cơ chế điều khiển phiên nhận dạng và kiến trúc ứng dụng frontend. Chương 4 trình bày quá trình cài đặt, thực nghiệm và đánh giá hệ thống, tập trung vào các thử nghiệm về độ chính xác, độ trễ và các quyết định tối ưu trong quá trình triển khai. Cuối cùng, chương Kết luận tổng kết các kết quả đạt được, nêu ra các giới hạn hiện tại và đề xuất một số hướng phát triển tiếp theo.

Chương 2

Nền tảng, nghiên cứu và công nghệ liên quan

2.1 Cơ sở lý thuyết và nghiên cứu liên quan

2.1.1 Ngôn ngữ ký hiệu

Ngôn ngữ ký hiệu biểu đạt thông tin thông qua hai nhóm đặc trưng cử chỉ – thị giác: đặc trưng thủ hiệu (*manual features*), gồm các đặc trưng liên quan đến bàn tay và chuyển động tay; và đặc trưng phi thủ hiệu (*non-manual features*), gồm các tín hiệu không được thể hiện bằng tay như nét mặt, ánh mắt, khẩu hình, chuyển động đầu và tư thế vai [17]. Khi nhận dạng hoặc dịch ký hiệu, tất cả các đặc trưng trên đều cần được xem xét để có thể nắm bắt toàn bộ ý nghĩa của các ký hiệu. Tuy nhiên, việc khai thác toàn bộ các đặc trưng này, đặc biệt là các đặc trưng phi thủ hiệu, vẫn đang là một thách thức lớn [4, 15, 18].

Các cộng đồng khiếm thính ở các địa phương khác nhau phát triển nhiều ngôn ngữ ký hiệu khác nhau. Có khoảng 150–300 ngôn ngữ ký hiệu trên thế giới [19, 20], trong đó ở Việt Nam có: NNKH Hà Nội, NNKH Hồ Chí Minh, và NNKH Hải Phòng. Ba ngôn ngữ này có mối liên hệ về mặt lịch sử nhưng vẫn là các ngôn ngữ vùng miền riêng biệt [21, 22].

2.1.2 So sánh CSLR và ISLR

Nhận dạng một câu ký hiệu liên tục đối mặt với nhiều thử thách hơn nhận dạng một ký hiệu đơn lẻ. Khi một người thực hiện hai ký hiệu liên tiếp, sự chuyển giao giữa hai ký hiệu đó gây ra hai hiện tượng: cử động thêm (*movement epenthesis* – ME) và hiệu ứng co-articulation. ME là các cử động xảy ra khi tay chuyển từ vị trí cuối của ký hiệu trước đến vị trí đầu của ký hiệu sau. ME không mang ý nghĩa nhưng lại có thể có thời lượng bằng cả một ký hiệu thật. Trong ngôn ngữ nói, hiệu ứng co-articulation xảy ra khi cách phát âm của một âm bị ảnh hưởng bởi các âm

xung quanh nó. Tương tự như vậy, cách thực hiện một ký hiệu cũng có thể bị tác động bởi ký hiệu liền trước và liền sau nó, khiến cho hình dạng của nó có thể bị xoay, kéo, giãn hoặc vặn xoắn. ME được coi là một vấn đề nền tảng và được đề cập đến bởi hầu hết các nghiên cứu về CSLR. Co-articulation ít xảy ra hơn ME nhưng có thể làm tăng khả năng bỏ qua các hình dạng tay hợp lệ, dẫn đến nhận dạng sai. Ngoài ra, các ký hiệu có thời lượng ngắn cũng thường dễ bị bỏ sót vì tương ứng với một lượng dữ liệu đầu vào ít hơn [4].

Dữ liệu ISLR có thể được tận dụng để hỗ trợ bài toán CSLR, đặc biệt trong bối cảnh dữ liệu video liên tục có chú thích gloss ở mức câu còn hạn chế. Một số hướng khai thác bao gồm sử dụng dữ liệu ISLR để tiền huấn luyện mô hình trước khi huấn luyện trên dữ liệu CSLR [23, 24], sử dụng các ký hiệu đơn lẻ như nguồn dữ liệu phụ trợ [5], hoặc dùng mô hình học từ ký hiệu đơn lẻ để hỗ trợ xác định ranh giới giữa các ký hiệu trong chuỗi ký hiệu liên tục [25]. Tuy nhiên, dữ liệu ISLR không thể thay thế hoàn toàn dữ liệu CSLR chuyên biệt, vì CSLR đòi hỏi mô hình phải xử lý chuỗi ký hiệu nối tiếp nhau, không có ranh giới rõ ràng, chịu ảnh hưởng của ME và co-articulation, ngữ cảnh câu và sự khác nhau giữa người ký hiệu [4, 26].

2.1.3 Gloss

Gloss là biểu diễn dưới dạng chữ viết của một ký hiệu và thể hiện ngữ nghĩa của ký hiệu đó. Gloss thường là các từ ở dạng cơ bản của ngôn ngữ nói liên quan, và thường được viết in hoa, ví dụ: MÈO, NĂM, BÀN. Gloss được sử dụng rộng rãi trong các tập dữ liệu SLR để làm nhãn cho các ký hiệu. Tuy nhiên, gloss không biểu diễn đầy đủ được các ký hiệu và có thể bỏ sót nhiều tín hiệu ngôn ngữ quan trọng. Việc tạo chú thích gloss cho một tập dữ liệu cũng đòi hỏi kiến thức chuyên môn của các nhà ngôn ngữ học và tốn rất nhiều thời gian. Ngoài ra, gloss là một công cụ ngôn ngữ được dùng trong nghiên cứu chứ không phải là hệ chữ viết tự nhiên được cộng đồng người khiếm thính sử dụng trong đời sống hằng ngày [27].

2.1.4 Các tập dữ liệu ngôn ngữ ký hiệu

Các tập dữ liệu SLR và SLT hiện nay đang hình thành một hệ sinh thái ngày càng phát triển, bao gồm các tập dữ liệu ký hiệu đơn lẻ như MS-ASL [28] (NNKH Mỹ), AUTSL [29] (NNKH Thổ Nhĩ Kỳ), và CNSL [30] (NNKH Trung Quốc), cùng

với các tập dữ liệu ký hiệu liên tục như RWTH-PHOENIX- Weather [16] (NNKH Đức), How2Sign [31] (NNKH Mỹ), và CSL-Daily [32] (NNKH Trung Quốc). Mặc dù đã có nhiều tiến bộ về quy mô và độ đa dạng, các bộ ngữ liệu ngôn ngữ ký hiệu vẫn còn nhỏ hơn nhiều lần và độ chuẩn hóa thấp hơn các bộ ngữ liệu ngôn ngữ nói tương ứng [33, 34]. Nhiều bộ dữ liệu chỉ bao gồm số lượng người ký hiệu hạn chế, miền nội dung hẹp, điều kiện ghi hình được kiểm soát và chưa đại diện đầy đủ cho các ngôn ngữ ký hiệu, phương ngữ, phong cách ký hiệu cũng như các đặc trưng phi thủ hiệu. Vì vậy, các mô hình SLR/SLT có thể đạt kết quả tốt trên benchmark nhưng vẫn khó tổng quát hóa ra môi trường thực tế [6, 11, 12, 15, 35].

Đối với NNKH Việt Nam, đến nay đã có các tập dữ liệu phục vụ bài toán ISLR như Multi-VSL [36], ViSL One-shot [37], và VSL-WRF-01/02 [38], với quy mô từ vài chục đến vài nghìn ký hiệu. Trong đó, đáng chú ý là bộ dữ liệu Multi-VSL. Theo công bố của nhóm tác giả, Multi-VSL là bộ dữ liệu RGB đa góc nhìn đầu tiên của NNKH Việt Nam và có quy mô lớn nhất trong số các bộ dữ liệu đa góc nhìn trên thế giới tại thời điểm được công bố [36]. Tuy nhiên, như đã đề cập ở Mục 2.1.2, dữ liệu ISLR chỉ chứa các mẫu ký hiệu rời rạc nên chưa đủ để huấn luyện trực tiếp mô hình CSLR, vốn yêu cầu video gồm chuỗi ký hiệu liên tục cùng nhãn tương ứng ở mức chuỗi gloss hoặc câu. Vì vậy, mặc dù các tập dữ liệu ISLR cho NNKH Việt Nam là nguồn dữ liệu có giá trị, chúng chưa đáp ứng đầy đủ yêu cầu dữ liệu của bài toán nhận dạng ngôn ngữ ký hiệu Việt Nam liên tục. Trong phạm vi khảo sát tài liệu và các nguồn dữ liệu công khai của khóa luận, tác giả chưa tìm thấy một tập dữ liệu CSLR công khai cho NNKH Việt Nam phù hợp để huấn luyện hoặc đánh giá trực tiếp phương pháp được sử dụng trong khóa luận.

RWTH-PHOENIX-Weather là một trong những bộ dữ liệu chuẩn có ảnh hưởng lớn trong nghiên cứu nhận dạng và dịch ngôn ngữ ký hiệu liên tục. Bộ dữ liệu này được xây dựng từ các bản tin dự báo thời tiết của đài truyền hình công cộng PHOENIX tại Đức, trong đó nội dung NNKH Đức được phiên dịch từ nội dung được nói trong bản tin. Trong lịch sử phát triển của các bộ dữ liệu CSLR, RWTH-PHOENIX-Weather giữ vai trò như một mốc chuyển tiếp quan trọng, góp phần đưa lĩnh vực này từ các bộ dữ liệu nhỏ hoặc thiên về ký hiệu đơn lẻ sang các benchmark công khai, liên tục và có khả năng so sánh giữa các phương pháp [39, 40].

Phiên bản RWTH-PHOENIX-Weather 2014 cung cấp dữ liệu video phục vụ bài toán CSLR, với các câu ký hiệu được gắn nhãn gloss [41]. Phiên bản mở rộng

RWTH-PHOENIX-Weather 2014T, thường được gọi là PHOENIX-2014T, bổ sung bản dịch sang tiếng Đức, qua đó cho phép đánh giá thêm bài toán dịch ngôn ngữ ký hiệu [16]. PHOENIX-2014T có tổng cộng khoảng 9,2 giờ video ký hiệu được thực hiện bởi 9 người ký hiệu và ghi hình ở tốc độ 25 FPS [16, 42].

Trong khóa luận này, PHOENIX-2014T được tiếp tục sử dụng theo thiết lập của công trình Zuo và cộng sự [14]. Việc lựa chọn này phù hợp với phạm vi khóa luận vì PHOENIX-2014T là một benchmark công khai cho bài toán ngôn ngữ ký hiệu liên tục, có quy mô tương đối lớn và đã được sử dụng rộng rãi trong các nghiên cứu CSLR/SLT. Bên cạnh đó, do vai trò lịch sử của RWTH-PHOENIX-Weather trong việc chuẩn hóa đánh giá CSLR và việc PHOENIX-2014T mở rộng benchmark này sang bài toán SLT, sử dụng PHOENIX-2014T giúp kết quả thử nghiệm của khóa luận có cơ sở so sánh với các công trình trước [16, 40].

Tuy nhiên, RWTH-PHOENIX-Weather vẫn có một số hạn chế. Miền nội dung của bộ dữ liệu chủ yếu giới hạn trong dự báo thời tiết; bối cảnh ghi hình tương đối cố định: người ký hiệu thường mặc trang phục tối màu trước nền xám; và số lượng người ký hiệu không lớn [16, 39, 40, 42]. Do đó, các mô hình huấn luyện hoặc đánh giá trên bộ dữ liệu này có thể gặp khó khăn khi áp dụng cho video ngoài miền dữ liệu, đặc biệt trong các điều kiện khác biệt về nền, ánh sáng, góc quay hoặc người ký hiệu.

2.1.5 Nhận dạng và dịch ngôn ngữ ký hiệu liên tục trực tuyến

Trong những năm gần đây, nghiên cứu về nhận dạng ngôn ngữ ký hiệu liên tục và dịch ngôn ngữ ký hiệu theo hướng trực tuyến, thời gian thực hoặc gần thời gian thực nhận được nhiều sự quan tâm hơn. Mục tiêu của hướng nghiên cứu này là cho phép hệ thống xử lý luồng ký hiệu đang diễn ra và tạo kết quả dần theo thời gian, thay vì chờ toàn bộ video kết thúc mới thực hiện nhận dạng hoặc dịch. Đây là yêu cầu quan trọng đối với các bối cảnh ứng dụng như hội thoại trực tiếp, gọi video, lớp học trực tuyến, dịch vụ khách hàng hoặc các nền tảng họp trực tuyến [13, 43, 44]. Về mặt phương pháp, lĩnh vực SLR/SLT nói chung đã chuyển dần từ các phương pháp dựa trên đặc trưng thủ hiệu và cảm biến sang các mô hình học sâu như CNN, RNN/LSTM, Transformer và các kiến trúc lai [6, 8, 15, 45, 46]. Các phương pháp dựa trên thị giác máy tính được sử dụng phổ biến vì không yêu cầu người ký hiệu đeo thiết bị đặc biệt, nhưng vẫn chịu ảnh hưởng bởi điều kiện ánh sáng, phong

nền, góc quay, chất lượng camera và sự khác biệt giữa các người ký hiệu [8, 11, 47]. Trong khi đó, các hệ thống dựa trên cảm biến có thể cung cấp tín hiệu ổn định hơn trong một số điều kiện, nhưng kém tự nhiên và khó được chấp nhận rộng rãi trong giao tiếp hàng ngày [10, 48].

Mặc dù nhiều hệ thống thời gian thực đã đạt kết quả cao trên bài toán nhận dạng ký hiệu đơn lẻ hoặc tập từ vựng nhỏ, nhận dạng và dịch ngôn ngữ ký hiệu liên tục vẫn là bài toán khó hơn đáng kể [45, 49, 50]. Trong CSLR, hệ thống phải xử lý chuỗi ký hiệu liên tục với ranh giới không rõ ràng giữa các ký hiệu, độ dài ký hiệu thay đổi, các ME và co-articulation. Vì vậy, hiệu năng thường giảm khi chuyển từ nhận dạng ký hiệu đơn lẻ sang nhận dạng câu ký hiệu liên tục hoặc miền từ vựng lớn [8, 15, 51]. Bên cạnh đó, các hệ thống CSLR/SLT trực tuyến còn phải đáp ứng yêu cầu về độ trễ thấp và ổn định, trong khi vẫn phải bảo đảm chất lượng nhận dạng trong các điều kiện thực tế. Những thách thức chính hiện nay bao gồm sự thiếu hụt dữ liệu lớn và đa dạng, khả năng tổng quát hóa kém giữa các người ký hiệu và môi trường khác nhau, khó khăn trong mô hình hóa đặc trưng phi thủ hiệu, giới hạn về tài nguyên tính toán và thiếu các benchmark chuẩn hóa cho thiết lập trực tuyến [8, 11, 15, 51]. Do đó, bên cạnh việc cải thiện mô hình nhận dạng hoặc dịch, các nghiên cứu theo hướng trực tuyến cũng cần quan tâm đến toàn bộ pipeline xử lý, bao gồm thu nhận video, tiền xử lý, suy luận mô hình, hậu xử lý, truyền dữ liệu và kiểm soát độ trễ đầu cuối.

2.1.6 Các giải pháp hỗ trợ giao tiếp hiện có

Hiện nay, các giải pháp hỗ trợ giao tiếp cho người điếc và khiếm thính có thể được chia thành một số nhóm chính. Nhóm thứ nhất là các ứng dụng và dịch vụ hỗ trợ giao tiếp trực tiếp, bao gồm dịch vụ phiên dịch qua video, chuyển đổi giọng nói thành văn bản, chuyển đổi văn bản thành giọng nói, hoặc chuyển đổi văn bản/giọng nói sang ngôn ngữ ký hiệu thông qua avatar hay hoạt hình phiên dịch.

Dịch vụ phiên dịch qua video (*Video Relay Service*) cho phép người khiếm thính giao tiếp với người nghe thông qua một phiên dịch viên ngôn ngữ ký hiệu tham gia cuộc gọi qua video. Phiên dịch viên có nhiệm vụ dịch nội dung ký hiệu thành lời nói và dịch lời nói của người nghe trở lại thành ngôn ngữ ký hiệu, nhờ đó hỗ trợ giao tiếp hai chiều theo thời gian thực [52]. Dịch vụ này giúp giao tiếp tự nhiên

hơn so với các hình thức chỉ dựa trên văn bản, nhưng vẫn phụ thuộc vào sự tham gia của phiên dịch viên con người.

Chuyển đổi văn bản hoặc giọng nói sang ngôn ngữ ký hiệu thông qua avatar phiên dịch là hướng tiếp cận trong đó nội dung đầu vào ở dạng văn bản hoặc lời nói trước hết được chuyển thành biểu diễn văn bản, sau đó được ánh xạ sang chuỗi ký hiệu tương ứng. Chuỗi ký hiệu này được hiển thị bằng một nhân vật ảo hoặc avatar 2D/3D thực hiện các động tác ký hiệu thay cho phiên dịch viên con người.

Theo một khảo sát năm 2023, nhiều ứng dụng Android và iOS phổ biến cho người điếc và khiếm thính hiện đã hỗ trợ các chức năng Video Relay Service, speech-to-text, text-to-speech và avatar phiên dịch. Tuy nhiên, trong số các ứng dụng này, chưa có ứng dụng nào được báo cáo là có khả năng tự động nhận diện ký hiệu của người dùng và chuyển đổi trực tiếp sang văn bản hoặc giọng nói [46].

Nhóm thứ hai là các mô hình nhận dạng ký hiệu đơn lẻ. Các nghiên cứu ISLR trong những năm gần đây đã đạt kết quả tương đối cao trên một số tập benchmark, chẳng hạn như TD-SLR, SignBERT, SAM-SLR hoặc MC3-18 [53]. Tuy nhiên, các mô hình ISLR chủ yếu nhận dạng từng ký hiệu riêng lẻ trong các đoạn video ngắn, do đó chưa trực tiếp giải quyết bài toán giao tiếp liên tục trong hội thoại thực tế.

Bên cạnh đó, một số hệ thống mới đang hướng tới dịch ngôn ngữ ký hiệu tự động ở mức thực tế hơn. Sign.mt là một dự án mã nguồn mở đang được phát triển để hướng tới dịch hai chiều giữa ngôn ngữ nói và ngôn ngữ ký hiệu. Hiện tại, sign.mt đã hỗ trợ dịch một số ngôn ngữ nói sang một số ngôn ngữ ký hiệu qua avatar phiên dịch [54]. SignCaption là một sản phẩm thương mại cho phép người dùng tải lên tệp video ngôn ngữ ký hiệu Mỹ, sau đó dịch vụ này dịch ký hiệu trong video sang tiếng Anh. Kết quả dịch được hiển thị dưới dạng phụ đề và người dùng có thể chỉnh sửa lại phụ đề nếu muốn. SignCaption cũng hỗ trợ dịch từ tiếng Anh sang hoạt hình avatar NNKH Mỹ. Ngoài ra, SignCaption còn đang phát triển một dịch vụ dịch tự động luồng video theo hướng trực tuyến để tạo phụ đề trong thời gian thực [55]. SignGemma được Google giới thiệu như một mô hình mở cho bài toán dịch ngôn ngữ ký hiệu sang văn bản ngôn ngữ nói. SignGemma ban đầu được giới thiệu là sẽ được công bố vào cuối năm 2025. Tuy nhiên, sản phẩm này đến nay vẫn đang trong giai đoạn nghiên cứu và thử nghiệm, và vẫn chưa đưa ra thời gian ra mắt được ấn định cụ thể [56]. Các hệ thống này cho thấy xu hướng chuyển từ các mô hình nhận dạng ký hiệu đơn lẻ sang các giải pháp hỗ trợ giao tiếp hoàn

chỉnh hơn. Tuy nhiên, nhiều giải pháp vẫn còn giới hạn về ngôn ngữ ký hiệu được hỗ trợ, trạng thái triển khai, khả năng hoạt động trực tuyến và mức độ sẵn sàng trong các bối cảnh gọi video thực tế.

2.1.7 Ước lượng tư thế và phát hiện đối tượng

Ước lượng tư thế (*Pose Estimation* – Pose) là bài toán xác định vị trí các điểm khớp hoặc điểm mốc quan trọng trên cơ thể người, chẳng hạn như vai, khuỷu tay, cổ tay, bàn tay hoặc khuôn mặt. Các điểm này được gọi là các keypoint. Trong ngữ cảnh nhận dạng ngôn ngữ ký hiệu, các keypoint thường được dùng để biểu diễn chuyển động của tay, thân trên và các thành phần phi thủ hiệu như nét mặt hoặc cử động miệng. Phát hiện đối tượng (*Object Detection* – Det) là bài toán xác định vị trí và phân loại các đối tượng trong ảnh hoặc video, thường thông qua các hộp giới hạn (bounding box). Đối với các mô hình ước lượng tư thế theo hướng từ trên xuống (top-down), phát hiện đối tượng thường được dùng như bước đầu tiên để xác định vùng chứa người trước khi ước lượng keypoint chi tiết.

2.1.8 Phương pháp CSLR trực tuyến của Zuo và cộng sự

Phần lớn các mô hình CSLR dựa trên CTC thường được thiết kế cho thiết lập ngoại tuyến, trong đó toàn bộ video cần được xử lý trước khi đưa ra dự đoán. Cách tiếp cận này giúp mô hình khai thác đầy đủ ngữ cảnh của cả câu ký hiệu, nhưng dẫn đến độ trễ cao và sử dụng nhiều bộ nhớ, do đó chưa phù hợp với các ứng dụng nhận dạng theo luồng. Trong bối cảnh đó, Zuo và cộng sự [14] đề xuất một trong những phương pháp đầu tiên chuyển bài toán CSLR từ thiết lập ngoại tuyến sang thiết lập trực tuyến. Tại thời điểm công bố, phương pháp này đạt kết quả state-of-the-art trên các benchmark được đánh giá, cho thấy hướng tiếp cận trực tuyến vẫn có thể duy trì hiệu quả nhận dạng cao [14].

Phương pháp của họ huấn luyện một mô hình ISLR trên từ điển ký hiệu được xây dựng từ dữ liệu CSLR, sau đó áp dụng mô hình này lên luồng video liên tục bằng cơ chế cửa sổ trượt. Cần lưu ý rằng việc sử dụng mô hình ISLR trong phương pháp này không có nghĩa là có thể thay thế hoàn toàn dữ liệu CSLR bằng dữ liệu ISLR thông thường. Nguyên nhân là từ điển ký hiệu dùng để huấn luyện ISLR vẫn được xây dựng từ các đoạn ký hiệu được tách ra từ dữ liệu CSLR, nhờ đó phản ánh tốt

hơn phân phối dữ liệu, ngữ cảnh và cách thực hiện ký hiệu trong chuỗi ký hiệu liên tục. Do các bộ dữ liệu CSLR thường không cung cấp nhãn biên thời gian cho từng ký hiệu, nhóm tác giả sử dụng một mô hình CSLR đã huấn luyện trước để phân tách video liên tục trong bộ dữ liệu CSLR thành các đoạn video chứa ký hiệu đơn lẻ, từ đó xây dựng từ điển ký hiệu phục vụ huấn luyện ISLR. Cửa sổ trượt có thể đi qua một ký hiệu nhiều lần, dẫn đến cùng một dự đoán bị lặp lại. Ngoài ra, các cửa sổ tương ứng với ME được mô hình gán nhãn nên vì các ME không mang ý nghĩa. Do đó, phương pháp còn cung cấp bước hậu xử lý để loại bỏ các dự đoán trùng lặp và các dự đoán thuộc lớp nền [14].

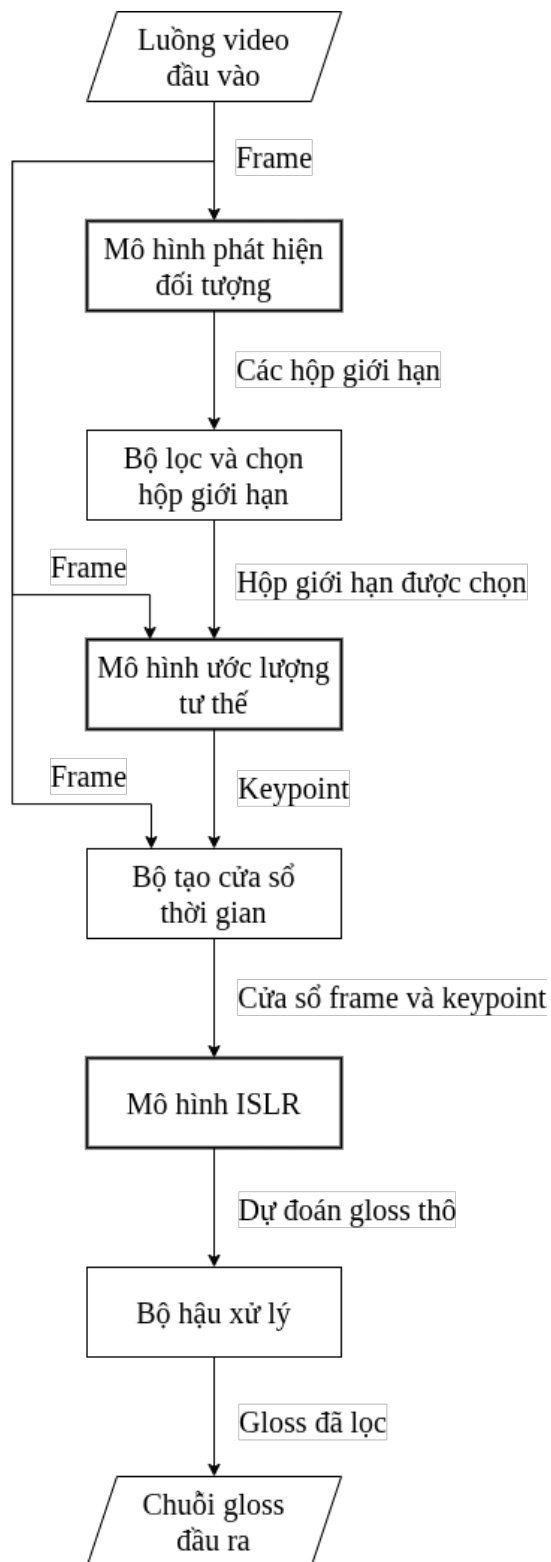
Kết quả thực nghiệm của nghiên cứu cho thấy hướng tiếp cận này có tiềm năng trong việc giảm độ trễ và hỗ trợ nhận dạng trực tuyến. Tuy nhiên, phương pháp vẫn phụ thuộc vào gloss, chất lượng của quá trình tạo từ điển ký hiệu và kiến trúc của mô hình ISLR [14].

Cấu trúc hai luồng

Một điểm đáng chú ý trong pipeline của Zuo và cộng sự là mô hình ISLR không chỉ sử dụng dữ liệu RGB, mà còn sử dụng thông tin keypoint của người ký hiệu thông qua kiến trúc hai luồng kế thừa từ TwoStream-SLR [57]. Trong đó, một luồng xử lý video RGB, còn luồng còn lại xử lý các heatmap được tạo từ keypoint của cơ thể, bàn tay và vùng miệng [14]. Do đó, trước khi đưa dữ liệu vào mô hình nhận dạng, hệ thống cần thực hiện bước ước lượng tư thế người để trích xuất các keypoint cần thiết. Theo mã nguồn của bài báo [58], mô hình ước lượng tư thế được sử dụng thuộc nhóm top-down và do đó cần thêm một mô hình phát hiện đối tượng để xác định vùng chứa người thực hiện ký hiệu trước khi ước lượng keypoint. Các bước tiền xử lý này không trực tiếp thuộc về mô hình nhận dạng, nhưng có ảnh hưởng đáng kể đến độ trễ, độ ổn định và chất lượng đầu vào của toàn bộ pipeline.

Pipeline CSLR trực tuyến

Hình 2.1 mô tả pipeline CSLR trực tuyến được trích xuất từ bài báo và mã nguồn của nhóm tác giả [14, 59]. Pipeline này được kế thừa làm phần cốt lõi của giải pháp. Mục tiêu của pipeline là chuyển đổi luồng video đầu vào thành chuỗi gloss đầu ra thông qua một chuỗi bước xử lý liên tiếp.



Hình 2.1: Pipeline CSLR trực tuyến

Bước đầu tiên là phát hiện đối tượng. Trong ngữ cảnh của hệ thống, đối tượng cần quan tâm là người ký hiệu. Mô hình phát hiện đối tượng nhận frame ảnh làm đầu vào và trả về các hộp giới hạn ứng viên thể hiện vị trí của người trong frame.

Sau đó, bộ lọc và chọn hộp giới hạn loại bỏ các hộp có độ tin cậy thấp hoặc diện tích quá nhỏ. Trong trường hợp một frame có nhiều hộp giới hạn hợp lệ, pipeline chỉ giữ lại hộp có độ tin cậy cao nhất để đại diện cho người ký hiệu cần nhận dạng. Cách xử lý này đơn giản hóa việc xử lý và phù hợp với phạm vi khóa luận, trong đó hệ thống giả định mỗi thời điểm chỉ có một người ký hiệu chính cần được nhận dạng.

Bước tiếp theo là ước lượng tư thế. Mô hình ước lượng tư thế sử dụng frame ảnh và hộp giới hạn của người ký hiệu để trích xuất các điểm keypoint. Trong bài toán nhận dạng ngôn ngữ ký hiệu, keypoint đóng vai trò quan trọng vì chúng biểu diễn thông tin về chuyển động của cơ thể, bàn tay và một số vùng liên quan đến đặc trưng phi thủ hiệu. Các keypoint này sau đó được sử dụng cùng với dữ liệu ảnh RGB để tạo đầu vào cho mô hình nhận dạng.

Trong trường hợp không có hộp giới hạn hợp lệ cho một frame, pipeline bỏ qua bước ước lượng tư thế cho frame đó và sử dụng một tensor keypoint rỗng, được biểu diễn bằng zero tensor có cùng shape với keypoint thông thường. Cách xử lý này giúp duy trì độ dài và cấu trúc dữ liệu theo thời gian, ngay cả khi một số frame không có kết quả phát hiện người hợp lệ.

Sau khi có chuỗi frame và keypoint tương ứng, pipeline tiến hành tạo cửa sổ thời gian. Do mô hình ISLR không nhận một frame đơn lẻ mà nhận một đoạn ngắn theo thời gian, các frame và keypoint liên tiếp được gom thành các cửa sổ. Nếu gọi kích thước cửa sổ là W và bước trượt là S , thì để tạo ra cửa sổ mới từ cửa sổ hiện tại, $(W - S)$ các cặp frame và keypoint ở cuối cửa sổ hiện tại sẽ được nối với S cặp tiếp theo. Cơ chế cửa sổ trượt cho phép hệ thống xử lý luồng video theo từng đoạn ngắn, từ đó tạo ra dự đoán theo thời gian thay vì chờ toàn bộ video đầu vào.

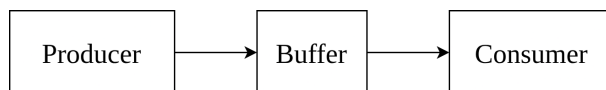
Mỗi cửa sổ thời gian sau đó được đưa vào mô hình ISLR để dự đoán gloss tương ứng. Mô hình ISLR trong hệ thống được xem là thành phần nhận dạng cốt lõi: nó nhận đầu vào là một đoạn video ngắn và các biểu diễn keypoint tương ứng, sau đó trả về dự đoán gloss. Tuy nhiên, do cơ chế cửa sổ trượt có thể quét cùng một ký hiệu nhiều lần, các kết quả dự đoán thô thường có thể chứa dự đoán trùng lặp.

Vì vậy, bước hậu xử lý là cần thiết để chuyển chuỗi dự đoán thô của mô hình ISLR thành chuỗi gloss ổn định hơn. Về nguyên tắc, bước này cũng sử dụng một cơ chế cửa sổ trượt trên chuỗi dự đoán thô, nhưng để tránh nhầm lẫn với cửa sổ thời gian đầu vào của mô hình ISLR, cửa sổ này được gọi là *voting bag*. Mỗi voting bag bao gồm một số dự đoán gloss thô liên tiếp, tương ứng với các cửa sổ video đầu vào đã được mô hình ISLR xử lý.

Đối với mỗi voting bag, pipeline thực hiện voting để chọn một kết quả đại diện. Trong thuật toán gốc, kết quả được chọn là lớp xuất hiện đủ thường xuyên trong voting bag; nếu không có lớp nào thỏa mãn điều kiện, voting bag được xem là thuộc lớp nền. Sau đó, pipeline loại bỏ các kết quả thuộc lớp nền và chỉ thêm một gloss mới vào chuỗi đầu ra nếu gloss đó khác với gloss không nền vừa được xuất ra trước đó. Cách xử lý này giúp giảm hai loại nhiễu thường gặp trong suy luận trực tuyến: các dự đoán nền tương ứng với đoạn chuyển tiếp hoặc đoạn không mang ý nghĩa ký hiệu rõ ràng, và các dự đoán lặp do nhiều cửa sổ video liên tiếp cùng quét qua một ký hiệu.

2.1.9 Mô hình producer-consumer

Trong các hệ thống xử lý dữ liệu liên tục, một cách tổ chức phổ biến là mô hình *producer-consumer*. Trong mô hình này, *producer* là thành phần sinh ra dữ liệu, còn *consumer* là thành phần lấy dữ liệu đó để xử lý. Như minh họa ở Hình 2.2, hai thành phần này thường không giao tiếp trực tiếp với nhau, mà trao đổi dữ liệu thông qua một buffer hoặc hàng đợi trung gian. Producer đưa dữ liệu vào buffer, trong khi consumer lấy dữ liệu từ buffer để xử lý vào thời điểm phù hợp.



Hình 2.2: Mô hình producer-consumer

Mô hình producer-consumer đặc biệt hữu ích khi tốc độ sinh dữ liệu và tốc độ xử lý dữ liệu không giống nhau. Nếu producer tạo dữ liệu nhanh hơn consumer xử lý, buffer sẽ tạm thời lưu các phần tử chưa được xử lý. Ngược lại, nếu consumer xử lý nhanh hơn tốc độ dữ liệu được tạo ra, consumer có thể phải chờ đến khi buffer có thêm dữ liệu mới. Nhờ cơ chế buffer trung gian, producer và consumer không

cần phải hoạt động hoàn toàn đồng bộ theo từng phần tử dữ liệu, từ đó giúp hệ thống linh hoạt hơn trong việc tổ chức các bước xử lý liên tiếp.

Đối với các luồng xử lý gồm nhiều bước, mô hình producer-consumer có thể được dùng để tổ chức các bước xử lý nối tiếp nhau. Mỗi bước có thể đóng vai trò là consumer đối với dữ liệu từ bước trước, đồng thời đóng vai trò là producer đối với bước sau. Ví dụ, trong pipeline nhận dạng được sử dụng, bước nhận frame tạo ra dữ liệu đầu vào cho bước phát hiện đối tượng; bước phát hiện đối tượng tạo ra hộp giới hạn cho bước ước lượng tư thế; bước ước lượng tư thế tạo ra keypoint cho bước nhận dạng tiếp theo. Các buffer trung gian giúp tách nhịp xử lý giữa các bước, vì mỗi bước có thể có thời gian xử lý khác nhau.

2.1.10 Server suy luận và triển khai mô hình học sâu

Sau khi một mô hình học sâu được huấn luyện, mô hình cần được triển khai để có thể tiếp nhận dữ liệu đầu vào mới và trả về kết quả dự đoán. Giai đoạn này được gọi là suy luận (*inference*). Khác với giai đoạn huấn luyện, suy luận thường được đặt trong bối cảnh hệ thống đang vận hành, nơi mô hình cần xử lý request từ các ứng dụng khác, đáp ứng yêu cầu về độ trễ, độ ổn định và khả năng sử dụng tài nguyên tính toán hiệu quả. Vì vậy, triển khai mô hình học sâu không chỉ liên quan đến bản thân mô hình, mà còn bao gồm cách đóng gói mô hình, quản lý đầu vào/đầu ra, lựa chọn phần cứng thực thi và tổ chức giao tiếp giữa mô hình với các thành phần khác trong hệ thống.

Server suy luận là thành phần chuyên phục vụ các mô hình học sâu trong quá trình suy luận. Thành phần này thường cung cấp một giao diện để client gửi request suy luận, thực thi mô hình trên CPU hoặc GPU, sau đó trả kết quả dự đoán. Việc sử dụng server suy luận giúp tách phần suy luận mô hình khỏi phần logic ứng dụng, từ đó thuận lợi hơn cho việc quản lý, thay thế, cập nhật và mở rộng mô hình. Trong các hệ thống có yêu cầu xử lý liên tục hoặc gần thời gian thực, server suy luận còn đóng vai trò quan trọng trong việc kiểm soát throughput, độ trễ và khả năng phục vụ nhiều request liên tiếp [60].

2.1.11 Suy luận theo batch

Batch trong học sâu là một nhóm nhiều mẫu dữ liệu được đưa vào mô hình và xử lý trong cùng một lần tính toán. Thay vì suy luận từng mẫu (đầu vào) riêng lẻ, việc gom nhiều mẫu thành một batch cho phép mô hình tận dụng tốt hơn khả năng tính toán song song của GPU, đặc biệt là trong các phép toán ma trận và tensor. Vì vậy, kích cỡ batch thường ảnh hưởng trực tiếp đến throughput, tức số lượng mẫu mà mô hình có thể xử lý trong một đơn vị thời gian. Kích cỡ batch lớn hơn có thể giúp tăng throughput và giảm chi phí tính toán cho mỗi mẫu, nhưng đồng thời cũng làm tăng nhu cầu bộ nhớ GPU và có thể làm tăng thời gian chờ để gom đủ dữ liệu vào một batch. Do đó, trong các hệ thống gần thời gian thực, kích thước batch cần được lựa chọn cẩn thận để cân bằng giữa khả năng tận dụng GPU, độ trễ của từng request và giới hạn bộ nhớ phần cứng.

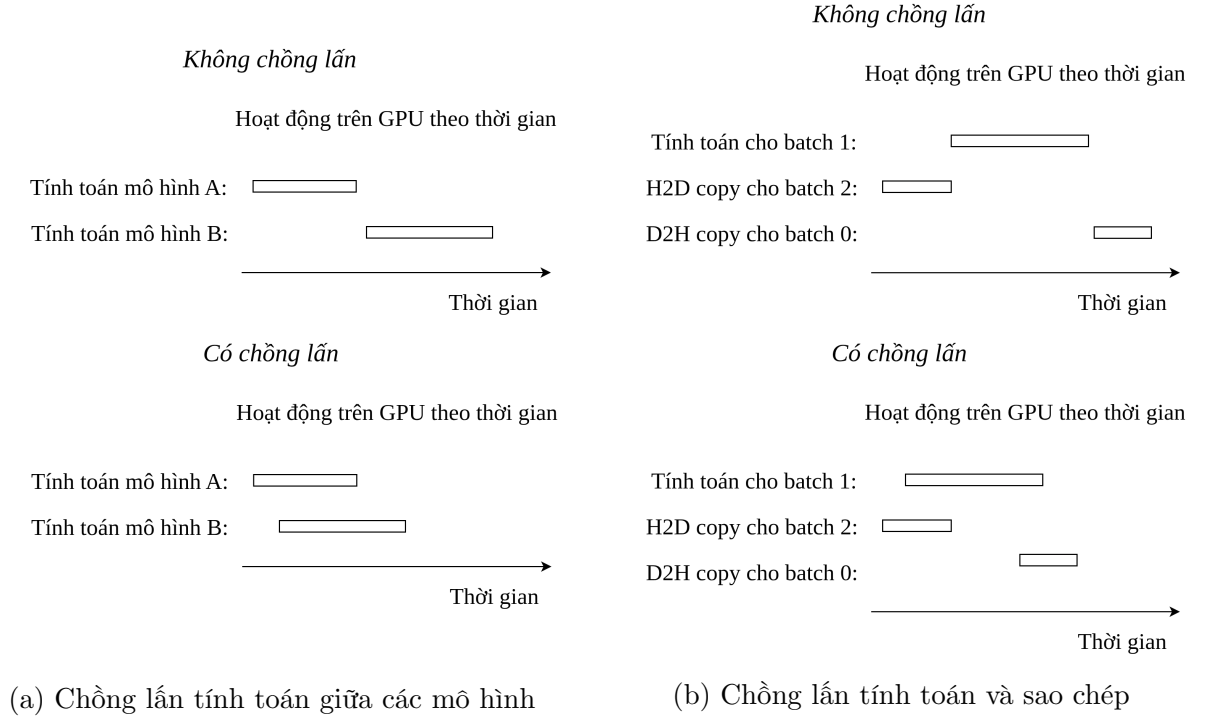
2.1.12 Cơ chế thực thi suy luận trên GPU sử dụng CUDA

Trong các hệ thống suy luận sử dụng GPU, mô hình học sâu thường được nạp vào bộ nhớ của GPU trước khi thực hiện suy luận. Khi đó, các tham số của mô hình, chẳng hạn như trọng số của các lớp mạng nơ-ron, được lưu trong bộ nhớ device, tức bộ nhớ trên GPU (VRAM). Dữ liệu đầu vào của hệ thống thường được tạo ra hoặc tiền xử lý ở phía CPU, nên ban đầu nằm trong bộ nhớ host, tức bộ nhớ chính do CPU quản lý (RAM) [61].

Vì GPU chỉ trực tiếp tính toán trên dữ liệu nằm trong bộ nhớ device, dữ liệu đầu vào cần được sao chép từ host sang device trước khi mô hình thực hiện suy luận. Thao tác này được gọi là sao chép host-to-device, viết tắt là H2D copy. Sau khi dữ liệu đã nằm trên GPU, mô hình thực hiện các phép tính suy luận trên dữ liệu đầu vào để tạo kết quả dự đoán. Nếu kết quả này cần được CPU sử dụng cho các bước như hậu xử lý, lưu trữ hoặc gửi trả về client, dữ liệu đầu ra cần được sao chép ngược từ device về host. Thao tác này được gọi là sao chép device-to-host, viết tắt là D2H copy. Như vậy, một lần suy luận trên GPU có thể được mô tả đơn giản qua ba nhóm thao tác chính: H2D copy, tính toán trên GPU và D2H copy [61].

Trong CUDA, các thao tác H2D copy, tính toán và D2H copy có thể được đưa vào CUDA stream để lập lịch thực thi. Một CUDA stream có thể được hiểu là một hàng đợi các tác vụ CUDA. Các tác vụ trong cùng một stream được thực hiện

theo thứ tự, trong khi các tác vụ thuộc các stream khác nhau có thể được thực thi chồng lấn nếu chúng độc lập và phần cứng còn đủ tài nguyên [61]. Trong ngữ cảnh này, chồng lấn có thể hiểu là việc hai hoặc nhiều thao tác có khoảng thời gian thực thi trùng nhau và được thực thi song song, thay vì phải chờ thao tác trước kết thúc hoàn toàn rồi mới bắt đầu thao tác sau. Hai dạng chồng lấn quan trọng trong suy luận theo luồng là chồng lấn giữa tính toán của các mô hình khác nhau và chồng lấn giữa truyền dữ liệu với tính toán được biểu diễn trong Hình 2.3.



Hình 2.3: So sánh giữa khi hiện tượng chồng lấn không xảy ra và khi nó có xảy ra

Trong trường hợp thứ nhất, các mô hình khác nhau có thể được đưa vào các CUDA stream khác nhau để tạo cơ hội cho GPU thực thi một phần công việc tính toán song song. Trong trường hợp thứ hai, hệ thống có thể sao chép dữ liệu đầu vào của batch tiếp theo lên GPU trong khi GPU đang tính toán batch hiện tại, hoặc sao chép kết quả của batch trước đó về CPU trong khi GPU tiếp tục xử lý batch sau. Cách tổ chức này giúp giảm thời gian chờ giữa các bước truyền dữ liệu và tính toán khi xử lý một luồng đầu vào liên tục [61].

Tuy nhiên, CUDA stream chỉ tạo điều kiện cho thực thi bất đồng bộ và chồng lấn tác vụ, chứ không bảo đảm rằng mọi thao tác sẽ luôn chạy song song. Việc chồng lấn có thực sự xảy ra hay không phụ thuộc vào nhiều yếu tố, bao gồm tài nguyên

còn lại của GPU, kích thước batch, chi phí tính toán của từng mô hình, cũng như các điểm đồng bộ trong chương trình [61].

2.2 Các công cụ và công nghệ được sử dụng

2.2.1 Triton Inference Server và các công cụ hỗ trợ triển khai suy luận

NVIDIA Triton Inference Server là một nền tảng phục vụ suy luận mô hình học sâu, cho phép triển khai mô hình từ nhiều framework khác nhau, trong đó có TensorRT, PyTorch, và Python. Triton tiếp nhận request suy luận thông qua HTTP/REST hoặc gRPC, sau đó định tuyến request đến bộ lập lịch và backend tương ứng của từng mô hình [60].

Backend thực thi chuyên dụng là thành phần chịu trách nhiệm thực thi suy luận cho một loại mô hình hoặc framework cụ thể. Ví dụ, Triton có thể sử dụng TensorRT backend cho các mô hình đã được tối ưu thành TensorRT engine, hoặc Python backend cho các mô hình có logic xử lý đặc thù bằng Python. Cơ chế backend giúp tách phần quản lý request của Triton khỏi phần thực thi mô hình cụ thể [62].

Streaming gRPC là cơ chế giao tiếp cho phép client và Triton gửi một chuỗi request/response trên cùng một gRPC stream. Cơ chế này phù hợp với các bài toán có đầu vào liên tục, vì client có thể duy trì một kết nối luồng để gửi nhiều request suy luận liên tiếp thay vì tạo các request độc lập hoàn toàn [63].

Ensemble model là cơ chế cho phép biểu diễn một pipeline gồm nhiều bước xử lý hoặc nhiều mô hình bên trong Triton. Khi sử dụng ensemble model, bộ lập lịch ensemble xác định luồng dữ liệu giữa các bước, ánh xạ đầu ra của bước trước thành đầu vào của bước sau, và nhìn từ phía client thì toàn bộ pipeline có thể được gọi như một mô hình duy nhất. Cách tổ chức này giúp giảm số lần client phải gửi request và hạn chế việc truyền tensor trung gian ra ngoài Triton [64].

Dynamic batching là cơ chế cho phép Triton tự động gom nhiều request suy luận riêng lẻ thành một batch trước khi đưa vào mô hình. Cơ chế này thường được dùng cho các mô hình không lưu trạng thái nhằm tăng throughput và tận dụng GPU tốt hơn, nhưng cần cân bằng với độ trễ do request có thể phải chờ để được gom batch [65].

Sequence batching là cơ chế gom batch dành cho các mô hình hoặc pipeline có trạng thái theo chuỗi request. Cơ chế này cho phép client gán một mã định danh sequence ID cho các request liên tiếp thuộc cùng một phiên xử lý, ví dụ các request được tạo ra từ cùng một luồng video nhận dạng. Các request có cùng sequence ID sẽ được Triton định tuyến đến cùng một thực thể xử lý, hay model instance, để đảm bảo trạng thái của chuỗi được duy trì nhất quán [66].

Triton Performance Analyzer là công cụ dùng để tạo tải suy luận đến mô hình đang được phục vụ bởi Triton và đo các chỉ số hiệu năng như throughput, độ trễ trung bình và thời gian xử lý phía server. Công cụ này hỗ trợ đánh giá hiệu năng qua HTTP hoặc gRPC và thường được dùng để phân tích đánh đổi giữa throughput, độ trễ và cấu hình triển khai mô hình [67].

2.2.2 LiveKit và các công cụ hỗ trợ truyền thông thời gian thực

LiveKit là một nền tảng mã nguồn mở hỗ trợ xây dựng các ứng dụng truyền thông thời gian thực dựa trên WebRTC, bao gồm truyền âm thanh, video và dữ liệu giữa nhiều người tham gia trong cùng một phòng. Trong khóa luận, LiveKit được sử dụng làm hạ tầng truyền thông thời gian thực cho hệ thống [68].

LiveKit Client SDK cho Flutter là bộ thư viện giúp ứng dụng Flutter kết nối đến LiveKit server, tham gia phòng, truyền nhận video và nhận các dữ liệu khác được gửi trong phòng [69]. Nhờ Client SDK, frontend có thể tập trung vào giao diện gọi video và hiển thị kết quả nhận dạng mà không cần tự hiện thực các chi tiết mức thấp của WebRTC.

LiveKit Server SDK cho Python là bộ thư viện hỗ trợ phía backend trong việc tương tác với LiveKit, chẳng hạn như quản lý phòng, quản lý thành viên hoặc xây dựng các thành phần backend tự động tham gia phòng [70]. Trong hệ thống của khóa luận, Bot server sử dụng SDK phía Python để cho bot tham gia phòng gọi, nhận luồng video và gửi dữ liệu kết quả về phòng.

Text stream là cơ chế truyền dữ liệu văn bản giữa các thành viên trong phòng LiveKit. Mỗi luồng văn bản có thể được gán với một topic, cho phép bên nhận đăng ký xử lý các luồng thuộc topic tương ứng [71]. Cơ chế này phù hợp với việc truyền các kết quả nhận dạng về frontend.

RPC (Remote Procedure Call) là cơ chế gọi phương thức từ xa giữa các thành viên trong cùng một phòng LiveKit. Một thành viên có thể đăng ký một phương thức RPC, sau đó thành viên khác trong cùng phòng có thể gọi phương thức này và chờ phản hồi [72]. Trong hệ thống của khóa luận, RPC phù hợp với các thao tác điều khiển phiên nhận dạng như bắt đầu, tạm dừng, tiếp tục hoặc kết thúc phiên nhận dạng.

2.2.3 Các thư viện OpenMMLab và các mô hình Det/Pose

OpenMMLab là một hệ sinh thái các thư viện mã nguồn mở phục vụ nghiên cứu và triển khai các bài toán thị giác máy tính. Trong khóa luận, ba thư viện chính được sử dụng là MMDetection, MMPose và MMDeploy. MMDetection là thư viện hỗ trợ các bài toán phát hiện đối tượng, cung cấp nhiều mô hình và cấu hình huấn luyện/suy luận khác nhau. MMPose là thư viện dành cho bài toán ước lượng tư thế, bao gồm các mô hình ước lượng keypoint người, tay, mặt và toàn thân. MMDeploy là bộ công cụ hỗ trợ chuyển đổi và triển khai các mô hình trong hệ sinh thái OpenMMLab sang các backend suy luận tối ưu hơn, chẳng hạn như TensorRT, nhằm phục vụ quá trình triển khai mô hình trong môi trường suy luận [73–75]. Các thư viện OpenMMLab được sử dụng trong khóa luận vì pipeline kế thừa từ nghiên cứu liên quan phụ thuộc vào các mô hình phát hiện người và ước lượng tư thế thuộc hệ sinh thái này [58].

Trong pipeline của khóa luận, mô hình phát hiện người và mô hình ước lượng tư thế ảnh hưởng trực tiếp đến cả độ trễ xử lý và chất lượng đầu vào của mô hình nhận dạng. Nhìn chung, Faster R-CNN [76] và HRNet [77] thường được ưu tiên về độ chính xác, nhưng có chi phí suy luận lớn hơn; trong khi đó, YOLOv3 [78] và ResNet50 [79] có tốc độ xử lý nhanh hơn, phù hợp hơn với yêu cầu gần thời gian thực, nhưng có thể làm giảm chất lượng hộp giới hạn và keypoint. Vì vậy, việc thay Faster R-CNN/HRNet bằng YOLOv3/ResNet50 ở Mục 4.3.1 được xem là một đánh đổi giữa tốc độ xử lý và độ chính xác nhận dạng của toàn bộ pipeline.

2.2.4 Flutter

Flutter [80] là framework giao diện đa nền tảng được sử dụng để phát triển ứng dụng frontend gọi video của hệ thống. Trong khóa luận này, Flutter được lựa chọn

vì cho phép phát triển giao diện người dùng cho nhiều nền tảng từ cùng một mã nguồn, phù hợp với mục tiêu hỗ trợ nhiều thiết bị của hệ thống. Bên cạnh đó, Flutter khuyến nghị tổ chức ứng dụng theo kiến trúc phân lớp và mô hình MVVM nhằm tách biệt rõ phần giao diện, logic hiển thị và xử lý dữ liệu. Cách tổ chức này giúp mã nguồn dễ bảo trì hơn, hạn chế việc trộn lẫn logic xử lý vào các thành phần giao diện, đồng thời tạo điều kiện thuận lợi cho việc kiểm thử, mở rộng và thay đổi từng thành phần của ứng dụng [81]. Phần mô tả chi tiết kiến trúc phân lớp và MVVM của ứng dụng sẽ được trình bày trong chương thiết kế hệ thống.

2.2.5 WebSocket

WebSocket [82] là một giao thức giao tiếp cho phép thiết lập kết nối hai chiều giữa client và server trên cùng một phiên kết nối. Khác với mô hình HTTP request-response thông thường, WebSocket cho phép hai phía chủ động gửi dữ liệu cho nhau sau khi kết nối đã được thiết lập. Vì vậy, WebSocket thường được sử dụng trong các ứng dụng cần trao đổi dữ liệu theo thời gian thực, chẳng hạn như trong việc gửi luồng video để xử lý và nhận về kết quả nhận dạng.

2.2.6 FastAPI

FastAPI là một web framework của Python dùng để xây dựng API [83] có hiệu năng cao, hỗ trợ mô hình lập trình bất đồng bộ và tích hợp tốt với hệ sinh thái Python. Nhờ đó, FastAPI phù hợp với các backend cần phát triển nhanh, có cấu trúc rõ ràng và dễ kiểm thử.

Trong khóa luận này, FastAPI được sử dụng để phát triển Bot server. FastAPI được lựa chọn vì backend cần được phát triển bằng Python để thuận tiện tích hợp với pipeline suy luận, các thư viện học sâu và các SDK liên quan. Bên cạnh các endpoint HTTP thông thường, FastAPI cũng hỗ trợ xây dựng endpoint WebSocket, phù hợp với các chức năng cần trao đổi dữ liệu hai chiều hoặc theo thời gian thực giữa client và server [83]. Ngoài ra, khả năng hỗ trợ các tác vụ bất đồng bộ cũng phù hợp với đặc điểm của Bot server, vì thành phần này cần xử lý các sự kiện phòng gọi, phiên nhận dạng và giao tiếp mạng với nhiều thành phần khác trong hệ thống.

2.2.7 PyTorch và TensorRT

PyTorch [84] là một thư viện học sâu phổ biến, hỗ trợ xây dựng, huấn luyện và suy luận mô hình bằng tensor trên CPU hoặc GPU. Trong khi đó, TensorRT [85] là bộ công cụ của NVIDIA dùng để tối ưu và triển khai mô hình học sâu cho suy luận hiệu năng cao trên GPU NVIDIA. Mô hình sau khi được chuyển đổi và tối ưu bởi TensorRT có thể được lưu dưới dạng TensorRT engine, là một biểu diễn đã được tối ưu cho phần cứng và cấu hình đầu vào cụ thể. So với việc chạy trực tiếp mô hình bằng PyTorch, TensorRT thường cho độ trễ thấp hơn và throughput cao hơn trong giai đoạn suy luận, đổi lại quá trình chuyển đổi và triển khai có thể phức tạp hơn.

2.2.8 RTX 3090 và RTX 4090

Các thực nghiệm trong khóa luận được thực hiện trên hai GPU: NVIDIA GeForce RTX 3090 [86] và NVIDIA GeForce RTX 4090 [87]. Hai GPU này đều có 24 GB VRAM, đủ để nạp đồng thời ba mô hình trong pipeline gồm Det, Pose và ISLR trong các cấu hình thực nghiệm của khóa luận. Do tổng kích thước trọng số của ba mô hình nhỏ hơn 1 GB, dung lượng VRAM còn lại có thể được sử dụng cho batch dữ liệu, dữ liệu đầu vào/đầu ra, các giá trị tạm thời và tài nguyên của framework suy luận. Bên cạnh dung lượng VRAM, khóa luận cũng ghi nhận hai thông số của GPU có liên quan đến hiệu năng suy luận là hiệu năng tính toán FP32 và băng thông bộ nhớ. Hai thông số này được quan tâm vì các mô hình trong pipeline được chạy ở định dạng FP32, đồng thời pipeline xử lý luồng video liên tục nên dữ liệu trung gian cần được đọc, ghi và chuyển tiếp liên tục trong bộ nhớ GPU. Vì vậy, hiệu năng FP32 và băng thông bộ nhớ có thể ảnh hưởng đến tốc độ xử lý thực tế của pipeline. RTX 3090 có hiệu năng FP32 lý thuyết khoảng 35,6 TFLOPS và băng thông bộ nhớ khoảng 936 GB/s [86], trong khi RTX 4090 có hiệu năng FP32 lý thuyết khoảng 82,6 TFLOPS và băng thông bộ nhớ khoảng 1008 GB/s [87]. Các thông số này cho thấy hai GPU có năng lực phần cứng cao và được kỳ vọng phù hợp cho các thử nghiệm suy luận của khóa luận. Tuy nhiên, các thông số phần cứng này chỉ cho thấy năng lực lý thuyết của GPU, còn hiệu năng thực tế của pipeline sẽ được đánh giá thông qua các thực nghiệm về độ trễ và tốc độ xử lý ở Chương 4.

Chương 3

Phân tích và thiết kế hệ thống

Chương này trình bày quá trình phân tích yêu cầu và thiết kế hệ thống hỗ trợ nhận dạng ngôn ngữ ký hiệu liên tục trong môi trường gọi video. Dựa trên mục tiêu và phạm vi của khóa luận, chương trước hết xác định các yêu cầu chức năng, yêu cầu phi chức năng và các ràng buộc của hệ thống. Sau đó, chương trình bày kiến trúc tổng thể, các luồng xử lý chính, cách tổ chức pipeline nhận dạng, thiết kế Server suy luận, cơ chế điều khiển phiên nhận dạng, kiến trúc frontend và các quyết định thiết kế liên quan đến kiểm soát độ trễ.

3.1 Phân tích yêu cầu hệ thống

Mục này trình bày các yêu cầu chính của hệ thống ở mức chức năng và phi chức năng, đồng thời làm rõ phạm vi và các ràng buộc của hệ thống trong khuôn khổ khóa luận.

3.1.1 Yêu cầu chức năng

Các yêu cầu chức năng của hệ thống được chia thành hai nhóm chính.

Nhóm yêu cầu thứ nhất liên quan đến việc tham gia và truyền phát video trong phòng gọi. Người dùng có thể tham gia và rời khỏi phòng gọi video thông qua ứng dụng frontend. Trong phòng gọi, người tham gia có thể chia sẻ các nguồn video của họ với phòng gọi. Ở đây, “nguồn video” được hiểu là nguồn dữ liệu hình ảnh do người dùng lựa chọn chia sẻ, bao gồm camera và chia sẻ màn hình. Việc hỗ trợ nhiều nguồn video giúp người dùng có nhiều lựa chọn hơn trong cách truyền tải nội dung hình ảnh. Ví dụ, người dùng có thể thực hiện ngôn ngữ ký hiệu trực tiếp trước camera, hoặc chia sẻ màn hình đang phát một video quay sẵn một câu ngôn ngữ ký hiệu. Sau khi một nguồn video được chia sẻ, hệ thống cần thu nhận và truyền các luồng video tương ứng đến những người tham gia còn lại trong phòng gọi. Nhờ đó, ứng dụng có thể hoạt động như một hệ thống gọi video cơ bản, làm nền tảng để tích hợp chức năng nhận dạng ký hiệu.

Nhóm yêu cầu thứ hai liên quan đến xử lý nhận dạng và hiển thị kết quả. Người tham gia phòng gọi có thể điều khiển phiên nhận dạng, bao gồm bắt đầu, tạm dừng, tiếp tục và kết thúc phiên nhận dạng. Khi bắt đầu một phiên nhận dạng, người dùng chọn một trong các nguồn video đã được chia sẻ làm đầu vào cho quá trình nhận dạng. Sau đó, hệ thống tiếp nhận luồng video từ nguồn được chọn để thực hiện nhận dạng và gửi kết quả về phòng gọi để frontend hiển thị cho người dùng. Mọi người tham gia trong phòng, bao gồm cả người đang sử dụng tính năng nhận dạng, đều có thể xem kết quả nhận dạng.

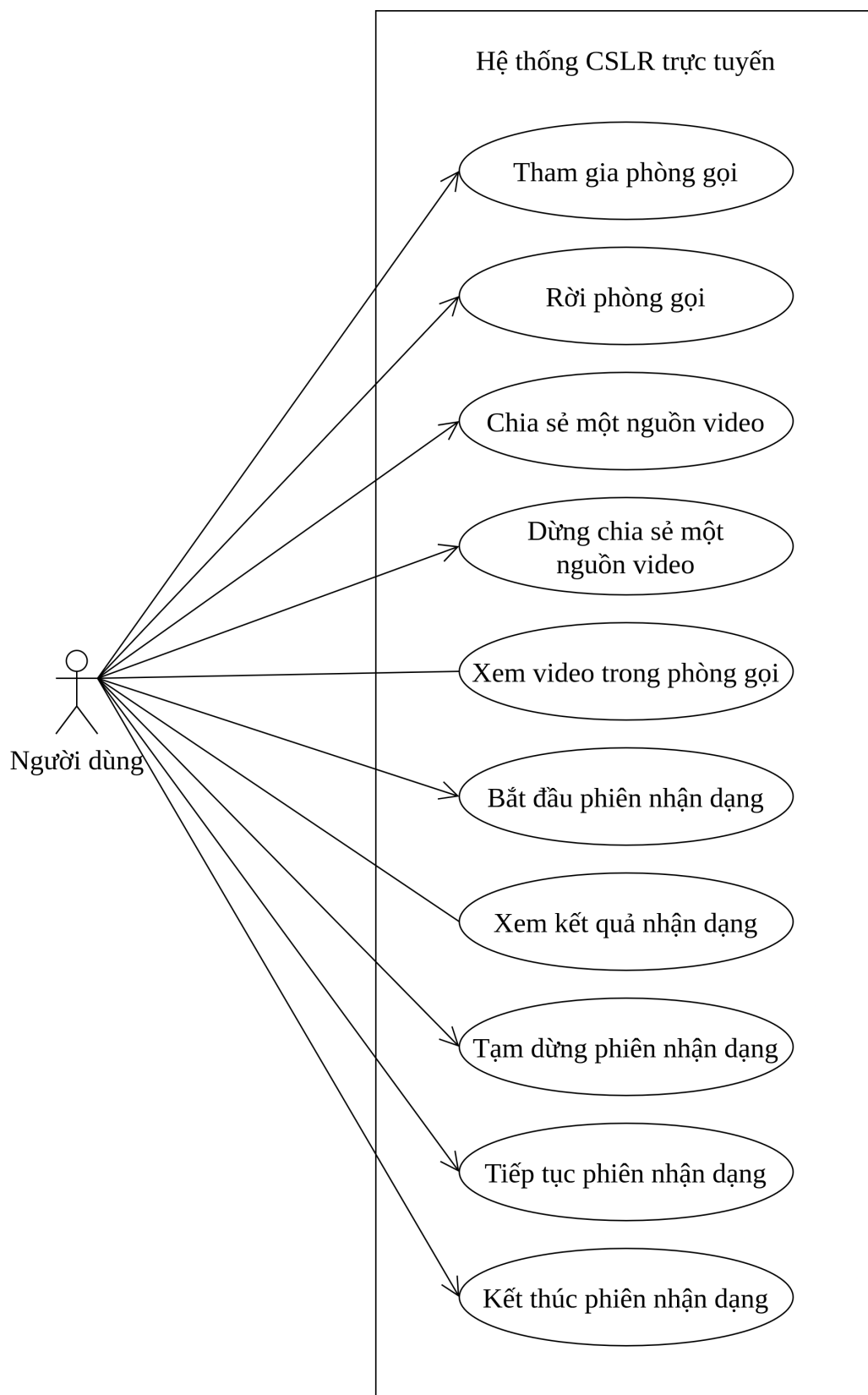
Tính năng tạm dừng và tiếp tục phiên nhận dạng được đưa vào nhằm hỗ trợ quản lý tài nguyên xử lý. Khi người sử dụng tính năng nhận dạng tạm thời không cần giao tiếp bằng ký hiệu, họ có thể tạm dừng phiên nhận dạng để hệ thống ngừng xử lý luồng video trong khoảng thời gian đó. Việc này giúp hạn chế sử dụng tài nguyên tính toán không cần thiết trên GPU. Đồng thời, cơ chế tạm dừng và tiếp tục cũng tránh việc phải kết thúc hoàn toàn phiên nhận dạng rồi khởi tạo lại các tài nguyên liên quan khi người dùng muốn tiếp tục sử dụng.

Dựa trên các yêu cầu chức năng trên, Hình 3.1 trình bày biểu đồ ca sử dụng của hệ thống. Biểu đồ sử dụng một tác nhân chính là *Người dùng*, đại diện cho tất cả người dùng cuối của hệ thống. Mỗi người dùng hệ thống đều có thể tham gia phòng gọi, chia sẻ nguồn video, điều khiển phiên nhận dạng và xem kết quả nhận dạng.

Trong phạm vi hệ thống hiện tại, mỗi phòng gọi chỉ cho phép một người tham gia sử dụng tính năng nhận dạng tại một thời điểm. Do đó, bất kỳ người tham gia nào trong phòng cũng có thể yêu cầu sử dụng tính năng nhận dạng nếu tại thời điểm đó chưa có phiên nhận dạng nào đang hoạt động. Khi một người đã bắt đầu phiên nhận dạng, người đó trở thành người đang được hệ thống tiếp nhận luồng video để xử lý nhận dạng, và chỉ người đó được quyền thực hiện các tính năng điều khiển phiên nhận dạng, bao gồm tạm dừng, tiếp tục và kết thúc phiên nhận dạng.

3.1.2 Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng, hệ thống cần đáp ứng một số yêu cầu phi chức năng liên quan đến chất lượng vận hành được trình bày ở Bảng 3.1. Các yêu cầu



Hình 3.1: Biểu đồ ca sử dụng của hệ thống

này được xác định dựa trên mục tiêu xây dựng một nguyên mẫu hệ thống nhận dạng CSLR trực tuyến có độ trễ thấp trong bối cảnh gọi video thử nghiệm.

Bảng 3.1: Các yêu cầu phi chức năng của hệ thống

Tiêu chí	Yêu cầu
Hiệu năng	Độ trễ nhận dạng phải đủ thấp để phù hợp với bối cảnh gọi video thử nghiệm. Trong phạm vi khóa luận, hệ thống đặt mục tiêu độ trễ nhận dạng dưới 5 giây.
Tính trực tuyến	Hệ thống cần xử lý video theo hướng trực tuyến, không chờ toàn bộ video kết thúc mới thực hiện nhận dạng.
Tính ổn định	Hệ thống cần duy trì độ trễ nhận dạng tương đối ổn định trong suốt phiên sử dụng, hạn chế các hiện tượng bất ổn như độ trễ tăng dần theo thời gian, dao động mạnh hoặc xuất hiện các đỉnh độ trễ lớn làm gián đoạn trải nghiệm gọi video.
Tính khả dụng	Giao diện frontend cần đơn giản, dễ sử dụng trong kịch bản thử nghiệm gọi video và nhận dạng ký hiệu. Frontend cũng cần được phát triển đa nền tảng để hỗ trợ nhiều loại thiết bị.
Khả năng bảo trì	Hệ thống cần tổ chức các thành phần chức năng sao cho dễ phát triển, kiểm thử và thay thế từng thành phần.

Trong các yêu cầu này, hiệu năng, tính trực tuyến và tính ổn định có vai trò đặc biệt quan trọng đối với hệ thống. Một hệ thống nhận dạng có thể tạo kết quả đúng nhưng chỉ trả kết quả sau khi toàn bộ video kết thúc sẽ không phù hợp với mục tiêu xử lý trực tuyến. Tương tự, một hệ thống có độ trễ ban đầu thấp nhưng tăng dần trong quá trình sử dụng cũng không phù hợp với kịch bản gọi video, vì người dùng sẽ dần nhận được kết quả nhận dạng quá muộn so với nội dung đang diễn ra.

3.1.3 Ràng buộc và phạm vi hệ thống

Trong phạm vi khóa luận, hệ thống hiện chỉ thực hiện nhận dạng ra chuỗi gloss, chưa thực hiện bước dịch chuỗi gloss sang câu ngôn ngữ tự nhiên. Vì vậy, kết quả

hiển thị trên frontend là kết quả nhận dạng ở mức gloss, chưa phải bản dịch hoàn chỉnh sang tiếng Việt hoặc ngôn ngữ nói.

Hệ thống sử dụng mô hình ISLR đã huấn luyện sẵn được kế thừa từ công trình tham khảo. Khóa luận không xây dựng lại từ điển ký hiệu, không tái huấn luyện mô hình ISLR và không đề xuất một kiến trúc mô hình nhận dạng mới. Trọng tâm của khóa luận là tích hợp pipeline nhận dạng vào một hệ thống gọi video có khả năng xử lý trực tuyến và có độ trễ thấp trong phạm vi thử nghiệm.

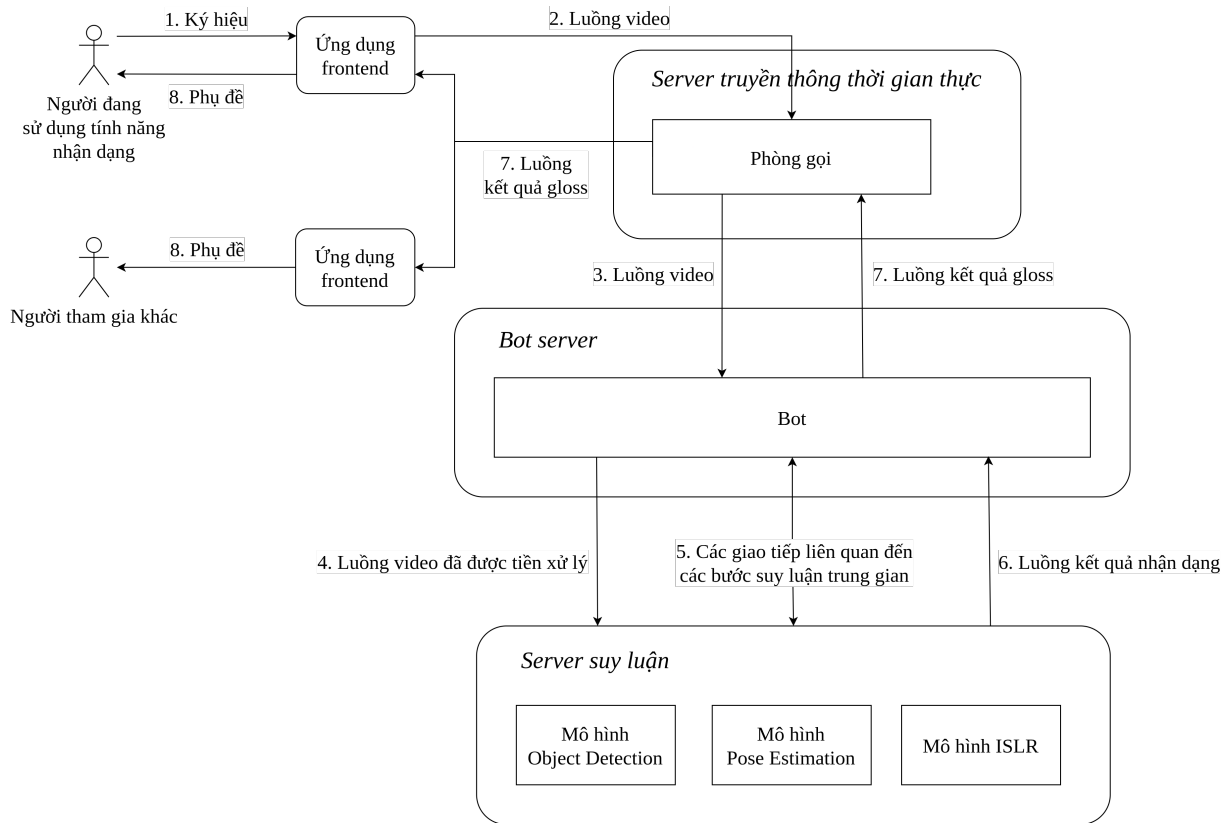
Do mô hình nhận dạng được huấn luyện trên miền dữ liệu RWTH-PHOENIX-Weather 2014T, khả năng nhận dạng của hệ thống phụ thuộc vào miền dữ liệu này. Vì vậy, hệ thống chưa thể được xem là có khả năng tổng quát hóa tốt sang mọi ngữ cảnh, mọi người ký hiệu hoặc mọi ngôn ngữ ký hiệu khác. Đây là một giới hạn quan trọng cần được xem xét khi đánh giá kết quả vận hành của hệ thống.

Hệ thống được xây dựng dưới dạng nguyên mẫu phục vụ thử nghiệm, chưa phải một sản phẩm hoàn chỉnh sẵn sàng triển khai trong môi trường thực tế. Các chức năng như quản lý người dùng, bảo mật ở mức sản phẩm, mở rộng cho số lượng lớn phòng gọi, hoặc hỗ trợ nhiều phiên nhận dạng đồng thời chưa nằm trong phạm vi chính của khóa luận.

Về giả định và ràng buộc, hệ thống giả định tại mỗi thời điểm chỉ có một người ký hiệu chính cần được nhận dạng trong luồng video đầu vào. Pipeline hiện tại chưa hỗ trợ nhận dạng đồng thời nhiều người ký hiệu trong cùng một frame. Ngoài ra, trong mỗi phòng gọi, hệ thống hiện chỉ cho phép một người tham gia sử dụng tính năng nhận dạng tại một thời điểm. Ràng buộc này giúp đơn giản hóa việc lựa chọn luồng video đầu vào, quản lý phiên nhận dạng và hiển thị kết quả, nhưng đồng thời cũng có nghĩa là hệ thống chưa hỗ trợ nhiều phiên nhận dạng song song trong cùng một phòng.

3.2 Thiết kế kiến trúc tổng thể

Dựa trên các yêu cầu đã phân tích ở Mục 3.1, hệ thống được thiết kế theo kiến trúc gồm nhiều thành phần tách biệt, trong đó mỗi thành phần đảm nhiệm một nhóm trách nhiệm riêng. Kiến trúc tổng thể của hệ thống được minh họa trong Hình 3.2.



Hình 3.2: Kiến trúc tổng thể của hệ thống

3.2.1 Các thành phần của hệ thống

Trong kiến trúc này, hệ thống bao gồm bốn thành phần chính: ứng dụng frontend, Server truyền thông thời gian thực, Bot server và Server suy luận.

Ứng dụng frontend là thành phần trực tiếp tương tác với người dùng. Thành phần này cung cấp giao diện để người dùng tham gia phòng gọi, chia sẻ video từ camera hoặc màn hình, xem video của các người tham gia khác và gửi các lệnh điều khiển phiên nhận dạng. Ngoài ra, frontend cũng nhận kết quả gloss từ hệ thống và hiển thị dưới dạng phụ đề trong giao diện gọi video.

Server truyền thông thời gian thực chịu trách nhiệm quản lý phòng gọi và truyền dữ liệu thời gian thực giữa các thành phần trong phòng. Các luồng video từ người tham gia được gửi lên thành phần này, sau đó được phân phối đến những người tham gia khác và Bot server. Ngoài luồng video, thành phần này còn đóng vai trò trung gian cho một số dữ liệu về lệnh điều khiển, trạng thái phiên nhận dạng và kết quả nhận dạng.

Bot server là thành phần điều phối chính của hệ thống nhận dạng. Bot server tạo cho mỗi phòng gọi một *bot* và cho bot tham gia phòng gọi. Trong mỗi phòng gọi, bot có nhiệm vụ nhận luồng video của người đang sử dụng tính năng nhận dạng, quản lý trạng thái phiên nhận dạng, điều phối quá trình nhận dạng và gửi kết quả gloss về các ứng dụng frontend.

Server suy luận là thành phần thực thi các mô hình học sâu và tùy theo phương án thiết kế có thể bao gồm một số bước xử lý khác. Theo pipeline ở Mục 2.1.8, các mô hình cần thiết bao gồm mô hình phát hiện người, mô hình ước lượng tư thế và mô hình nhận dạng ký hiệu đơn lẻ trên các cửa sổ video. Server suy luận giúp tách phần thực thi mô hình khỏi Bot server, đồng thời tạo điều kiện triển khai mô hình trên GPU và thử nghiệm các phương án tổ chức pipeline khác nhau.

Việc tách hệ thống thành các thành phần độc lập giúp giảm sự phụ thuộc trực tiếp giữa giao diện người dùng, logic điều phối và phần suy luận mô hình. Nhờ đó, từng thành phần có thể được thiết kế, kiểm thử, thay thế và tối ưu tương đối độc lập. Điều này phù hợp với yêu cầu về khả năng bảo trì của hệ thống.

Việc xây dựng Server truyền thông thời gian thực có chất lượng tốt là bài toán phức tạp do cần xử lý nhiều vấn đề như truyền luồng âm thanh–hình ảnh, quản lý phòng gọi, đồng bộ trạng thái người tham gia và duy trì kết nối thời gian thực. Vì đây không phải là trọng tâm của khóa luận, khóa luận không tự xây dựng thành phần truyền thông thời gian thực mà sử dụng lại LiveKit server [68].

Trong phạm vi khóa luận, Server suy luận được xem xét qua hai phiên bản: phiên bản tự xây dựng và phiên bản triển khai bằng NVIDIA Triton Inference Server [60]. Hai phiên bản này được trình bày lần lượt ở Mục 3.5 và Mục 3.6. Quyết định lựa chọn Triton để triển khai Server suy luận trong phiên bản sau được giải thích ở Mục 3.5.3.

Trong các phần tiếp theo, thuật ngữ *Server truyền thông thời gian thực* được dùng để chỉ thành phần truyền thông trong kiến trúc hệ thống. Khi cần nhấn mạnh công nghệ cụ thể, khóa luận sử dụng tên LiveKit hoặc LiveKit server. Tương tự, *Server suy luận* được dùng để chỉ thành phần phục vụ suy luận mô hình học sâu, còn Triton Inference Server là công nghệ được sử dụng để hiện thực thành phần này trong quá trình triển khai.

3.2.2 Luồng xử lý chính của hệ thống

Ở mức tổng thể, luồng xử lý chính của hệ thống bao gồm các bước như sau. Đầu tiên, người dùng tham gia phòng thông qua ứng dụng frontend và chia sẻ một nguồn video để thực hiện giao tiếp bằng ngôn ngữ ký hiệu. Ví dụ, người dùng có thể chia sẻ luồng video từ camera và thực hiện các ký hiệu trước camera. Khi đó, luồng video này được gửi đến Server truyền thông thời gian thực để phân phối cho các thành phần trong phòng.

Khi một người tham gia bắt đầu phiên nhận dạng, Bot server xác định người đó là người đang sử dụng tính năng nhận dạng và bắt đầu tiếp nhận luồng video của người đó từ Server truyền thông thời gian thực. Từ thời điểm này, Bot server chịu trách nhiệm đưa dữ liệu video đầu vào đến luồng xử lý nhận dạng và phối hợp với Server suy luận để thực hiện quá trình nhận dạng. Các chi tiết về các phương án tổ chức phối hợp giữa Bot server và Server suy luận sẽ được trình bày ở Mục 3.3.3.

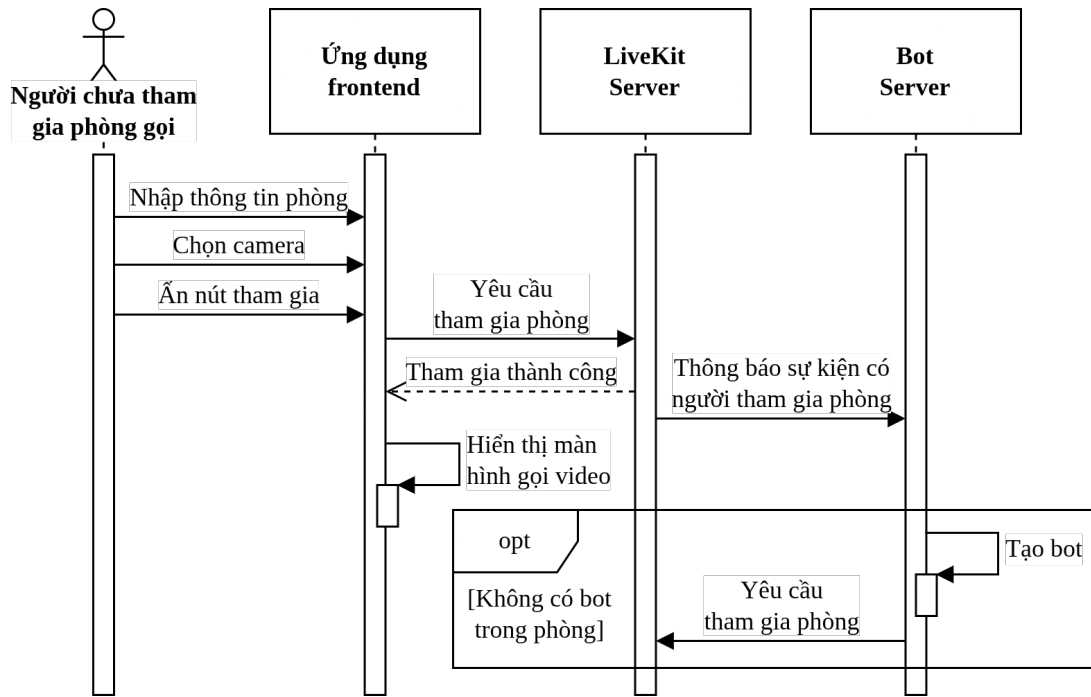
Kết quả nhận dạng sẽ được đưa ra dần theo thời gian. Bot server đảm nhiệm việc truyền các kết quả này dưới dạng luồng qua Server truyền thông thời gian thực đến các ứng dụng frontend của những người tham gia phòng. Mỗi ứng dụng frontend sẽ hiển thị luồng kết quả dưới dạng phụ đề trong giao diện gọi video. Nhờ đó, tất cả mọi người tham gia trong phòng đều có thể xem kết quả nhận dạng.

3.3 Thiết kế các luồng xử lý của hệ thống

3.3.1 Các luồng tham gia và rời phòng gọi

Hình 3.3 thể hiện biểu đồ tuần tự hiện thực ca sử dụng *Tham gia phòng gọi*, đồng thời cũng mô tả luồng khởi tạo bot. Để tham gia phòng gọi, người dùng thực hiện các thao tác nhập thông tin phòng, chọn camera để sử dụng trong phòng và ấn nút tham gia trên ứng dụng frontend. Yêu cầu tham gia sẽ được gửi từ ứng dụng frontend đến LiveKit server. Sau khi người dùng tham gia phòng thành công, ứng dụng frontend hiển thị màn hình phòng gọi. Trong khi đó, LiveKit server gửi thông báo sự kiện tham gia phòng đến Bot server. Nếu trong phòng chưa có bot, Bot server khởi tạo một bot mới và cho bot tham gia phòng.

Biểu đồ tuần tự trong Hình 3.4 hiện thực ca sử dụng *Rời phòng gọi*, đồng thời mô tả luồng kết thúc vòng đời của bot. Sau khi người tham gia ấn nút rời phòng, ứng



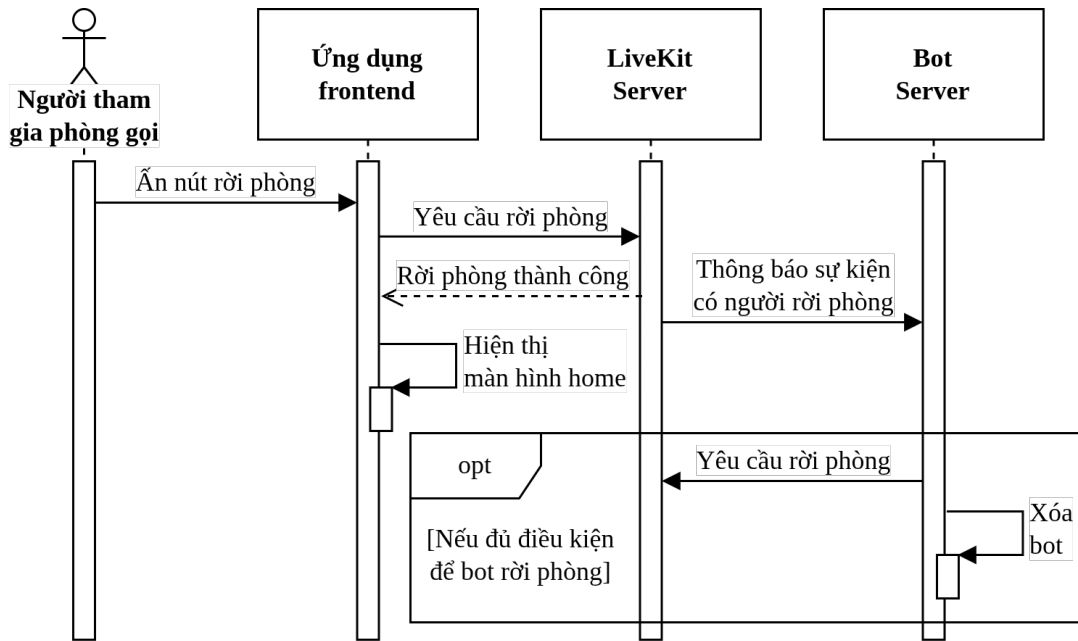
Hình 3.3: Biểu đồ tuần tự của ca sử dụng *Tham gia phòng gọi*

dụng frontend gửi yêu cầu rời phòng và đợi phản hồi từ LiveKit server, sau đó điều hướng đến màn hình home. Sau khi người dùng đã rời phòng, LiveKit thông báo sự kiện này đến Bot server. Nếu không còn người dùng nào trong phòng ngoài bot và không có người tham gia mới trong một khoảng thời gian timeout nhất định, Bot server gửi yêu cầu rời phòng của bot đến LiveKit server rồi thực hiện xóa bot. Việc này cũng tạo điều kiện cho LiveKit server có thể dọn dẹp các phòng không còn người tham gia, vì bot cũng được tính là một thành viên và chỉ khi bot rời phòng thì phòng mới được tính là rỗng.

Cách thiết kế này giúp mỗi phòng gọi có một bot xử lý riêng, đồng thời tránh việc bot tồn tại dư thừa ở phòng không có người tham gia.

3.3.2 Các luồng điều khiển phiên nhận dạng

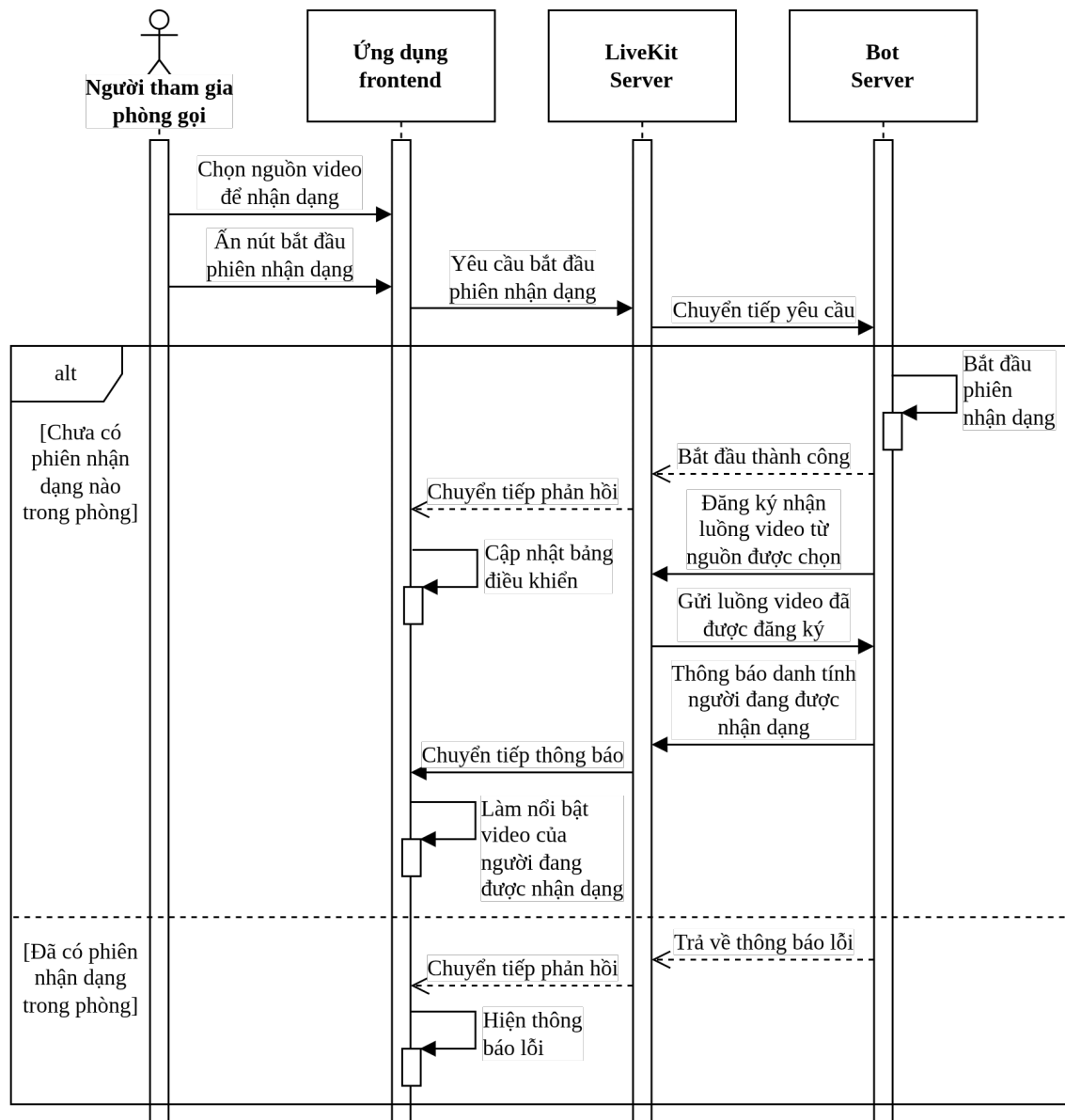
Hình 3.5 trình bày biểu đồ tuần tự của ca sử dụng *Bắt đầu phiên nhận dạng*. Trước khi thực hiện ca sử dụng này, người tham gia cần chia sẻ ít nhất một nguồn video. Sau đó, người tham gia chọn nguồn video cho quá trình nhận dạng và ấn nút bắt đầu phiên nhận dạng trên ứng dụng frontend. Trong Bot server, bot trong cùng phòng gọi với người dùng sẽ nhận yêu cầu bắt đầu phiên nhận dạng qua LiveKit



Hình 3.4: Biểu đồ tuần tự của ca sử dụng *Rời phòng gọi*

server. Nếu chưa có phiên nhận dạng nào đang diễn ra trong phòng gọi tương ứng, bot bắt đầu một phiên nhận dạng mới và trả về phản hồi cho ứng dụng frontend. Từ đó, ứng dụng frontend cập nhật bảng điều khiển trong màn hình phòng gọi của người dùng để hiển thị các nút tạm dừng phiên nhận dạng và kết thúc phiên nhận dạng thay cho nút bắt đầu phiên nhận dạng. Trong khi đó, bot đăng ký nhận luồng video từ nguồn mà người vừa bắt đầu phiên nhận dạng đã chọn, rồi thông báo người đang được nhận dạng đến các ứng dụng frontend của tất cả người tham gia trong phòng. Qua đó, các ứng dụng frontend có thể làm nổi bật video của người đang được nhận dạng. Trong trường hợp người dùng muốn bắt đầu phiên nhận dạng nhưng đã tồn tại một phiên trong phòng, bot trả về lỗi đến ứng dụng frontend qua LiveKit server, từ đó thông báo lỗi được hiển thị cho người dùng.

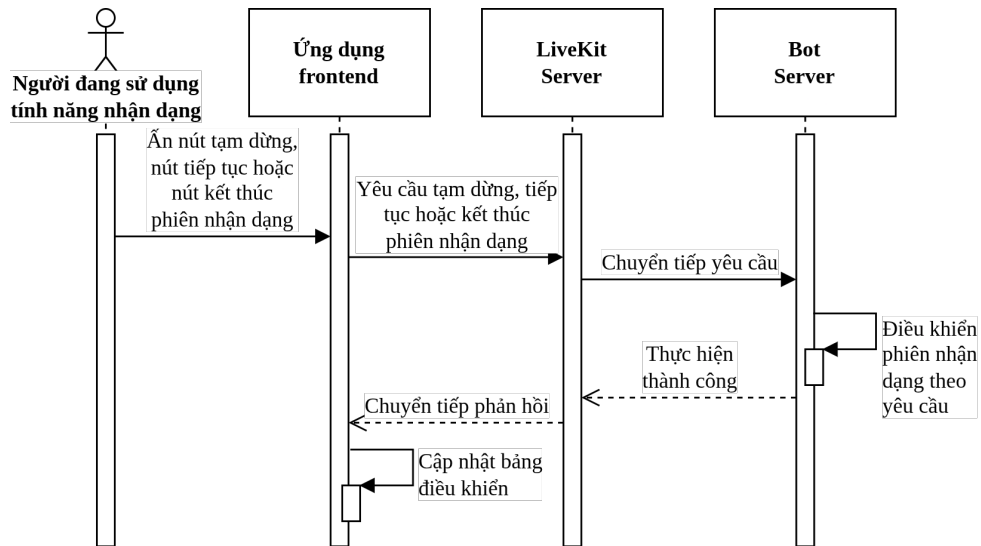
Biểu đồ tuần tự trong Hình 3.6 hiện thực các ca sử dụng liên quan đến điều khiển phiên nhận dạng, bao gồm *Tạm dừng phiên nhận dạng*, *Tiếp tục phiên nhận dạng* và *Kết thúc phiên nhận dạng*. Các thao tác này tương ứng với các nút tạm dừng phiên nhận dạng, tiếp tục phiên nhận dạng và kết thúc phiên nhận dạng trên bảng điều khiển của màn hình phòng gọi, và chỉ được ứng dụng frontend hiển thị cho người đang sử dụng tính năng nhận dạng. Khi người dùng ấn một trong các nút đó, bot sẽ nhận yêu cầu từ ứng dụng frontend qua LiveKit server và thực hiện



Hình 3.5: Biểu đồ tuần tự của ca sử dụng *Bắt đầu phiên nhận dạng*

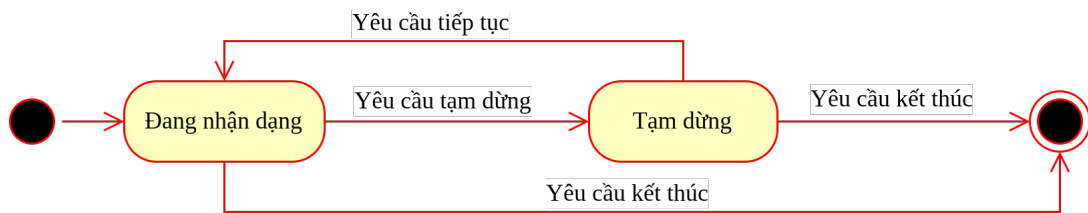
điều khiển phiên nhận dạng theo yêu cầu. Sau đó, bot gửi phản hồi về ứng dụng frontend để cập nhật bảng điều khiển theo trạng thái mới của phiên nhận dạng.

Mỗi phiên nhận dạng có các trạng thái được mô tả trong Hình 3.7. Khi một phiên nhận dạng bắt đầu, phiên này được đặt ở trạng thái *Đang nhận dạng*. Đồng thời, ứng dụng frontend hiển thị nút tạm dừng phiên và nút kết thúc phiên. Nếu người sử dụng tính năng nhận dạng thực hiện tạm dừng phiên, phiên được đưa về trạng thái *Tạm dừng*, và ứng dụng frontend thay nút tạm dừng bằng nút tiếp tục. Sau đó, nếu người này ấn nút tiếp tục, phiên sẽ được đưa trở lại trạng thái *Đang nhận dạng*, và ứng dụng frontend lại hiển thị nút tạm dừng thay cho nút tiếp tục. Ở cả



Hình 3.6: Biểu đồ tuần tự của các ca sử dụng điều khiển phiên nhận dạng

hai trạng thái *Đang nhận dạng* và *Tạm dừng*, ứng dụng frontend luôn hiển thị nút kết thúc để người dùng có thể kết thúc phiên.

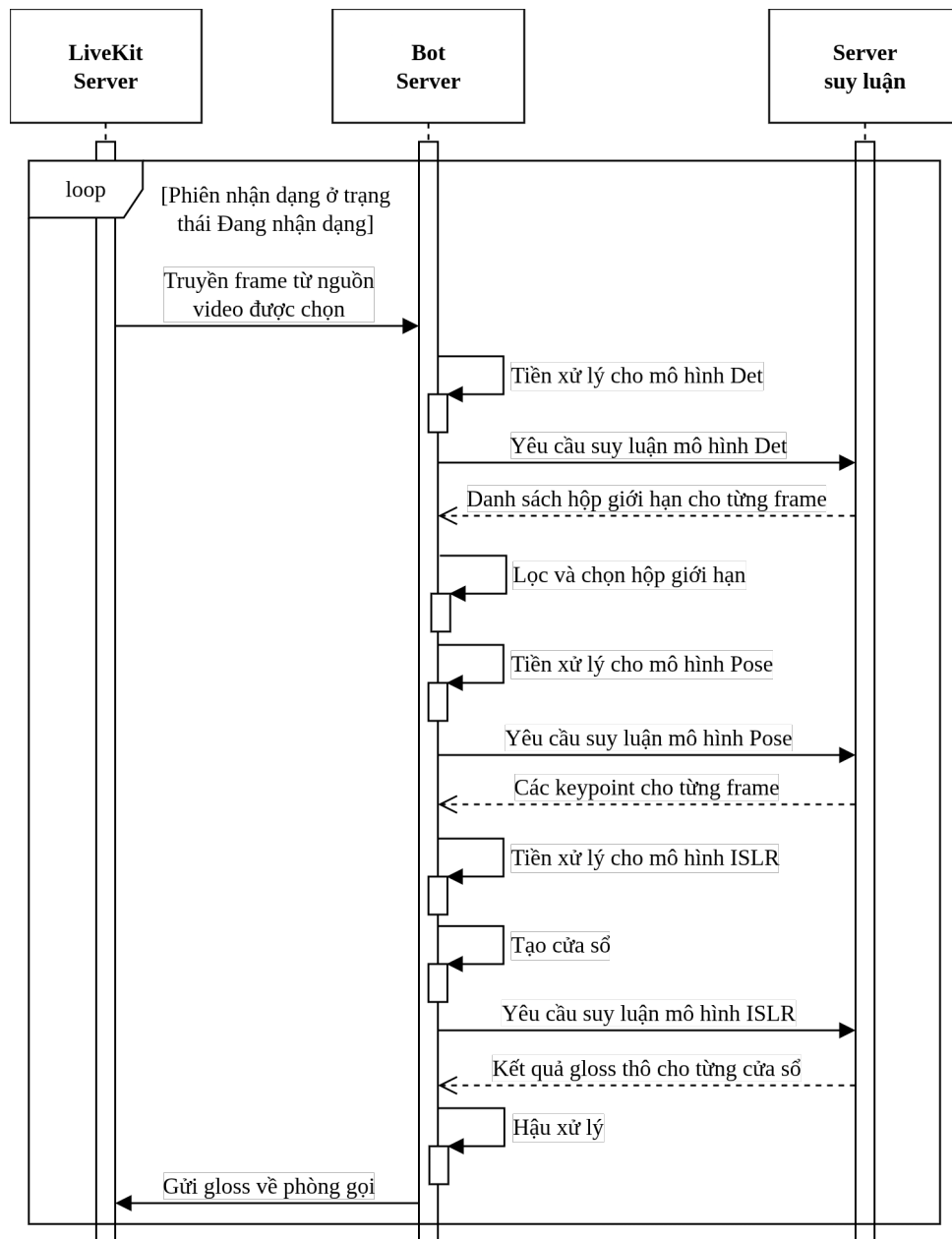


Hình 3.7: Biểu đồ trạng thái của một phiên nhận dạng

3.3.3 Luồng suy luận từ Bot server đến Server suy luận

Luồng suy luận từ Bot server đến Server suy luận bắt đầu sau khi người dùng đã bắt đầu một phiên nhận dạng. Khi phiên nhận dạng đang ở trạng thái *Đang nhận dạng*, bot ở phòng gọi tương ứng liên tục nhận các frame từ nguồn video mà người dùng đã chọn để làm đầu vào cho quá trình nhận dạng. Khi phiên ở trạng thái *Tạm dừng*, bot tạm dừng việc đưa các frame mới vào pipeline suy luận cho đến khi phiên trở lại trạng thái *Đang nhận dạng*.

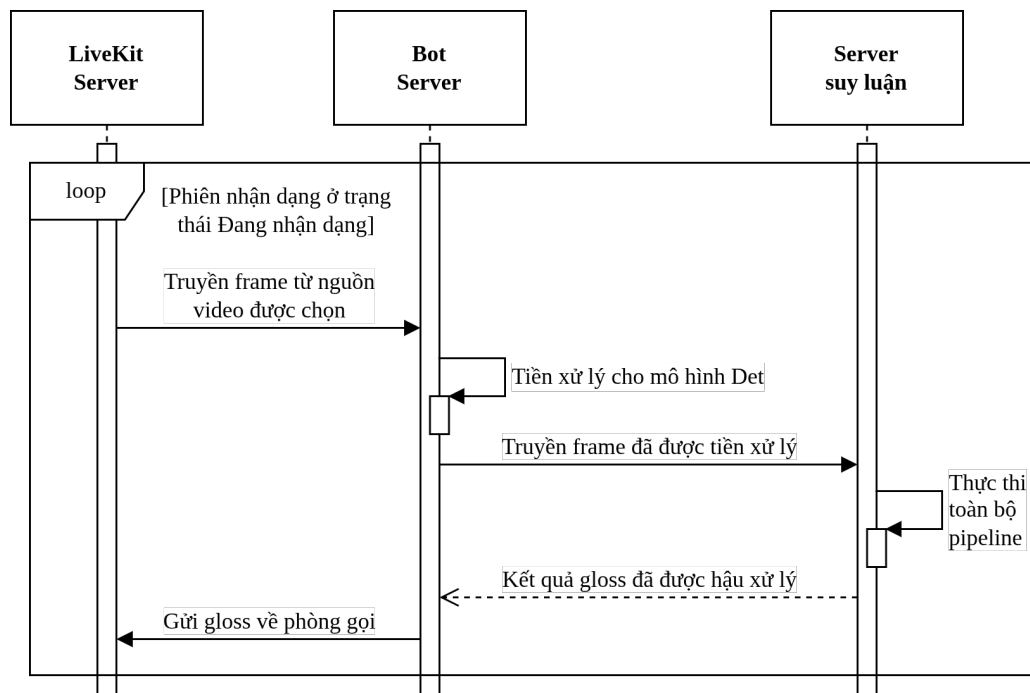
Tùy theo vị trí đặt các bước xử lý trung gian của pipeline, khóa luận xem xét hai phương án tổ chức luồng suy luận giữa Bot server và Server suy luận. Hai phương án này khác nhau chủ yếu ở chỗ các bước như lọc và chọn hộp giới hạn, tạo cửa sổ thời gian và hậu xử lý được thực hiện ở Bot server hay được đưa vào Server suy



Hình 3.8: Biểu đồ tuần tự luồng suy luận theo phương án Server suy luận phục vụ từng mô hình riêng lẻ

luận. Ngoài ra, trước khi truyền dữ liệu đến Server suy luận, Bot server cần thực hiện một số bước tiền xử lý theo yêu cầu của các mô hình Det, Pose và ISLR. Vì các bước tiền xử lý này thường bao gồm thu nhỏ kích thước frame đầu vào, việc đưa chúng vào Bot server có thể làm giảm lượng dữ liệu cần truyền qua mạng.

Hình 3.8 trình bày luồng suy luận theo phương án thứ nhất, trong đó Server suy luận chỉ phục vụ từng mô hình riêng lẻ. Khi nhận được frame đầu vào, bot tiền xử lý frame cho mô hình Det rồi gửi đến Server suy luận để thực hiện suy luận mô hình này và nhận lại danh sách hộp giới hạn. Tiếp theo, bot thực hiện bước lọc và chọn hộp giới hạn, tiền xử lý cho mô hình Pose và gửi yêu cầu suy luận mô hình Pose. Sau khi nhận được kết quả keypoint, bot tiền xử lý cho mô hình ISLR, thực hiện tạo các cửa sổ và gửi đến mô hình ISLR trên Server suy luận. Mô hình ISLR trả về dự đoán gloss thô, sau đó bot thực hiện hậu xử lý và gửi kết quả gloss cuối cùng về phòng gọi.



Hình 3.9: Biểu đồ tuần tự luồng suy luận theo phương án Server suy luận chứa toàn bộ pipeline

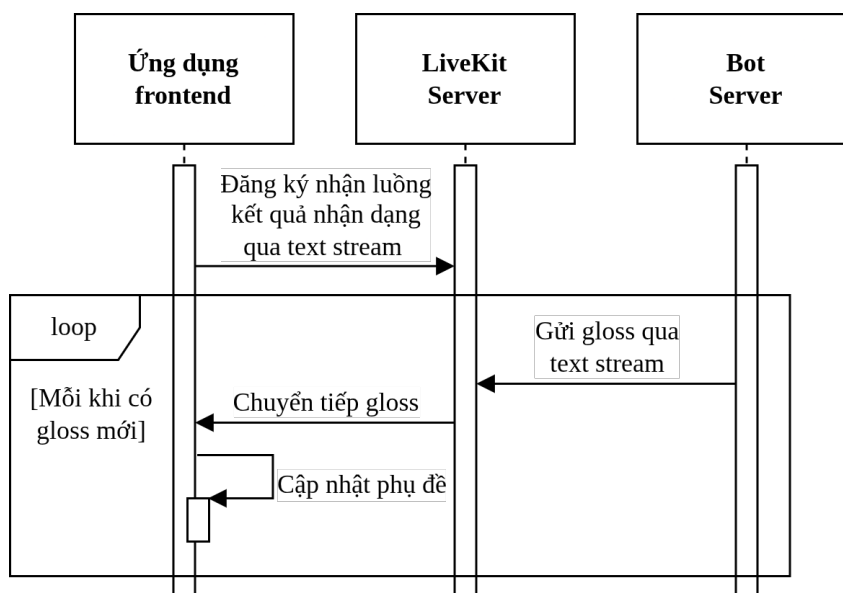
Trong phương án thứ hai được thể hiện bởi Hình 3.9, Server suy luận chứa toàn bộ pipeline nhận dạng. Bot server chỉ thực hiện tiền xử lý frame cho mô hình đầu tiên của pipeline là mô hình Det, rồi gửi luồng frame đã được tiền xử lý sang Server suy luận. Sau đó, Server suy luận thực hiện toàn bộ pipeline từ phát hiện đối tượng,

lọc và chọn hộp giới hạn, ước lượng tư thế, tạo cửa sổ thời gian, nhận dạng ISLR đến hậu xử lý kết quả. Đồng thời, Server suy luận cũng đảm nhiệm việc thực hiện tiền xử lý cho mô hình Pose và ISLR. Đầu ra trả về Bot server là luồng gloss đã qua hậu xử lý.

Phương án thứ nhất giúp Bot server dễ kiểm soát từng bước xử lý trung gian, trong khi phương án thứ hai giúp giảm số lần giao tiếp giữa Bot server và Server suy luận. Các ưu điểm, nhược điểm và phạm vi áp dụng của từng phương án sẽ được phân tích chi tiết hơn ở Mục 3.6.

3.3.4 Luồng trả kết quả gloss về frontend

Hình 3.10 trình bày biểu đồ tuần tự hiện thực ca sử dụng *Xem kết quả nhận dạng*, đồng thời mô tả luồng trả kết quả gloss về frontend. Trong mỗi phòng gọi, hệ thống sử dụng một text stream trên LiveKit để truyền luồng kết quả nhận dạng. Sau khi một người tham gia phòng gọi, ứng dụng frontend của người này đăng ký nhận luồng kết quả nhận dạng thông qua text stream của phòng đó. Mỗi khi có kết quả gloss mới, bot trong phòng tương ứng ghi kết quả vào text stream, sau đó các ứng dụng frontend của tất cả người tham gia trong phòng đọc kết quả từ text stream và cập nhật giao diện hiển thị phụ đề.



Hình 3.10: Biểu đồ tuần tự luồng trả kết quả gloss về frontend

3.4 Thiết kế tổ chức pipeline nhận dạng

Như đã trình bày ở Mục 2.1.8, pipeline nhận dạng của hệ thống bao gồm các bước chính: phát hiện người ký hiệu, lọc và chọn hộp giới hạn, ước lượng tư thế, tạo cửa sổ thời gian, nhận dạng bằng mô hình ISLR và hậu xử lý kết quả. Mục tiêu của thiết kế ở mục này là tổ chức các bước xử lý này sao cho phù hợp với dữ liệu đầu vào dạng liên tục, hỗ trợ xử lý theo batch ở các bước suy luận mô hình và tạo cơ sở cho việc kiểm soát độ trễ của toàn bộ hệ thống.

3.4.1 Tổ chức pipeline theo mô hình producer-consumer

Khóa luận tổ chức pipeline nhận dạng theo mô hình producer-consumer. Trong mô hình này, mỗi bước xử lý nhận dữ liệu từ bước trước, thực hiện chức năng của mình, sau đó tạo dữ liệu đầu ra cho bước tiếp theo. Như vậy, một bước trong pipeline có thể đóng vai trò là consumer đối với dữ liệu từ bước trước, đồng thời đóng vai trò là producer đối với bước sau.

Cách tổ chức này phù hợp với pipeline nhận dạng vì các bước xử lý có đặc điểm và tốc độ thực thi khác nhau. Chẳng hạn, bước phát hiện người và ước lượng tư thế là các bước suy luận mô hình trên từng frame hoặc từng batch frame; bước tạo cửa sổ cần tích lũy nhiều frame và keypoint liên tiếp theo thời gian; bước ISLR xử lý các cửa sổ video; còn bước hậu xử lý cần xem xét chuỗi các dự đoán thô. Nếu các bước này bị ràng buộc phải chạy hoàn toàn đồng bộ theo từng frame, tốc độ của toàn pipeline sẽ dễ bị chi phối bởi bước chậm nhất và khó tận dụng khả năng xử lý theo batch của các mô hình.

Do đó, pipeline được thiết kế với các buffer trung gian giữa các bước. Các buffer này cho phép tách nhịp xử lý của các bước liên kề: bước phía trước có thể tiếp tục sinh dữ liệu và đưa vào buffer, trong khi bước phía sau lấy dữ liệu từ buffer khi đủ điều kiện xử lý. Các buffer còn được thiết kế dựa trên cấu trúc hàng đợi, do đó cũng giúp duy trì thứ tự dữ liệu theo thời gian. Đây là một yêu cầu quan trọng đối với bài toán nhận dạng ngôn ngữ ký hiệu liên tục, vì mô hình ISLR và bước hậu xử lý đều phụ thuộc vào trình tự của các frame và cửa sổ video.

3.4.2 Vị trí các bước xử lý

Trong hệ thống, các mô hình học sâu của pipeline gồm Det, Pose và ISLR đều được đặt ở Server suy luận, sau đó Server suy luận được triển khai trong môi trường có GPU phù hợp. Quyết định này xuất phát từ đặc điểm của các mô hình học sâu: quá trình suy luận cần tài nguyên tính toán lớn và phù hợp hơn với môi trường có GPU.

Bảng 3.2: Vị trí các bước xử lý trong hai phương án tổ chức pipeline

Bước xử lý	Phương án 1: Server suy luận phục vụ từng mô hình riêng lẻ	Phương án 2: Server suy luận chứa toàn bộ pipeline
Det	Server suy luận	Server suy luận
Lọc và chọn hộp giới hạn	Bot server	Server suy luận
Pose	Server suy luận	Server suy luận
Tạo cửa sổ thời gian	Bot server	Server suy luận
ISLR	Server suy luận	Server suy luận
Hậu xử lý kết quả	Bot server	Server suy luận

Theo Mục 3.3.3, có hai phương án tổ chức luồng suy luận từ Bot server đến Server suy luận. Mặc dù các mô hình Det, Pose và ISLR vẫn luôn được đặt ở Server suy luận ở cả hai phương án này, có sự khác nhau về vị trí đặt các bước xử lý trung gian, tức là các bước lọc và chọn hộp giới hạn, tạo cửa sổ thời gian và hậu xử lý kết quả nhận dạng. Ở phương án Server suy luận phục vụ từng mô hình riêng lẻ, các bước này được đặt ở Bot server. Ở phương án còn lại, các bước này được đặt chung với các mô hình ở Server suy luận. Bảng 3.2 so sánh vị trí đặt các bước xử lý ở hai phương án.

3.4.3 Thiết kế buffer trung gian và xử lý theo batch

Trong phương án Server suy luận phục vụ từng mô hình riêng lẻ, cũng như trong phiên bản thử nghiệm tự xây dựng của Server suy luận, pipeline cần sử dụng các buffer trung gian để truyền dữ liệu giữa các bước xử lý. Các buffer này đóng vai trò lưu tạm dữ liệu đầu ra của bước trước và cung cấp đầu vào cho bước sau. Cách

tổ chức này giúp mỗi bước có thể xử lý dữ liệu theo tốc độ riêng, đồng thời vẫn giữ được thứ tự thời gian của luồng video. Các chi tiết về cách sử dụng các buffer sẽ được trình bày ở các Mục 3.5.1 và 3.8.

Các buffer trung gian chính trong pipeline bao gồm:

- buffer frame đầu vào cho bước Det;
- buffer chứa frame và danh sách hộp giới hạn từ Det cho bước lọc và chọn hộp giới hạn;
- buffer chứa frame và hộp giới hạn được chọn cho bước Pose;
- buffer chứa frame và keypoint cho bước tạo cửa sổ thời gian;
- buffer chứa cửa sổ frame và keypoint cho bước ISLR;
- buffer chứa dự đoán gloss thô cho bước hậu xử lý.

Buffer frame đầu vào nhận các frame từ luồng video và cung cấp dữ liệu cho mô hình Det. Sau khi Det tạo ra danh sách hộp giới hạn ứng viên, kết quả này được đưa vào buffer tiếp theo cho bước lọc và chọn hộp giới hạn xử lý. Kết quả của bước này được đưa vào buffer frame và hộp giới hạn để cung cấp đầu vào cho bước Pose.

Sau khi Pose tạo ra keypoint, các frame và keypoint tương ứng được đưa vào buffer phục vụ bước tạo cửa sổ thời gian. Bước tạo cửa sổ cần lấy nhiều frame và keypoint liên tiếp theo thứ tự thời gian để tạo thành các cửa sổ đầu vào cho mô hình ISLR. Các cửa sổ này được đưa vào buffer cửa sổ và sau đó được xử lý bởi ISLR. Dự đoán gloss thô từ ISLR tiếp tục được đưa vào buffer hậu xử lý trước khi gửi kết quả cuối cùng về frontend.

Ngoài chức năng lưu trữ trung gian, các buffer còn được thiết kế để hỗ trợ thao tác lấy một batch gồm nhiều phần tử liên tiếp theo thứ tự thời gian. Cơ chế này phù hợp với đặc điểm của các mô hình học sâu trong pipeline, vì Det, Pose và ISLR đều có thể xử lý nhiều mẫu trong một lần suy luận để tận dụng tốt hơn khả năng tính toán song song của GPU. Cơ chế lấy dữ liệu theo batch cũng có ý nghĩa đối với bước tạo cửa sổ thời gian. Mặc dù bước này không phải là một mô hình suy luận học sâu, nó cần gom nhiều frame và keypoint liên tiếp để tạo ra một hoặc nhiều cửa sổ theo kích thước cửa sổ và bước trượt đã định.

Với phương án Server suy luận chứa toàn bộ pipeline trong phiên bản triển khai bằng Triton, hệ thống sử dụng bộ lập lịch ensemble của Triton. Bộ lập lịch này đảm nhiệm việc thu các đầu ra của từng bước và cung cấp chúng làm đầu vào cho các bước sau theo cấu hình đã định [64]. Do đó, hệ thống không cần trực tiếp định nghĩa và cài đặt các buffer được nêu ở trên. Mục 3.6.2 sẽ trình bày các chi tiết về phương án triển khai này.

3.5 Thiết kế lập lịch và quản lý thực thi trong Server suy luận

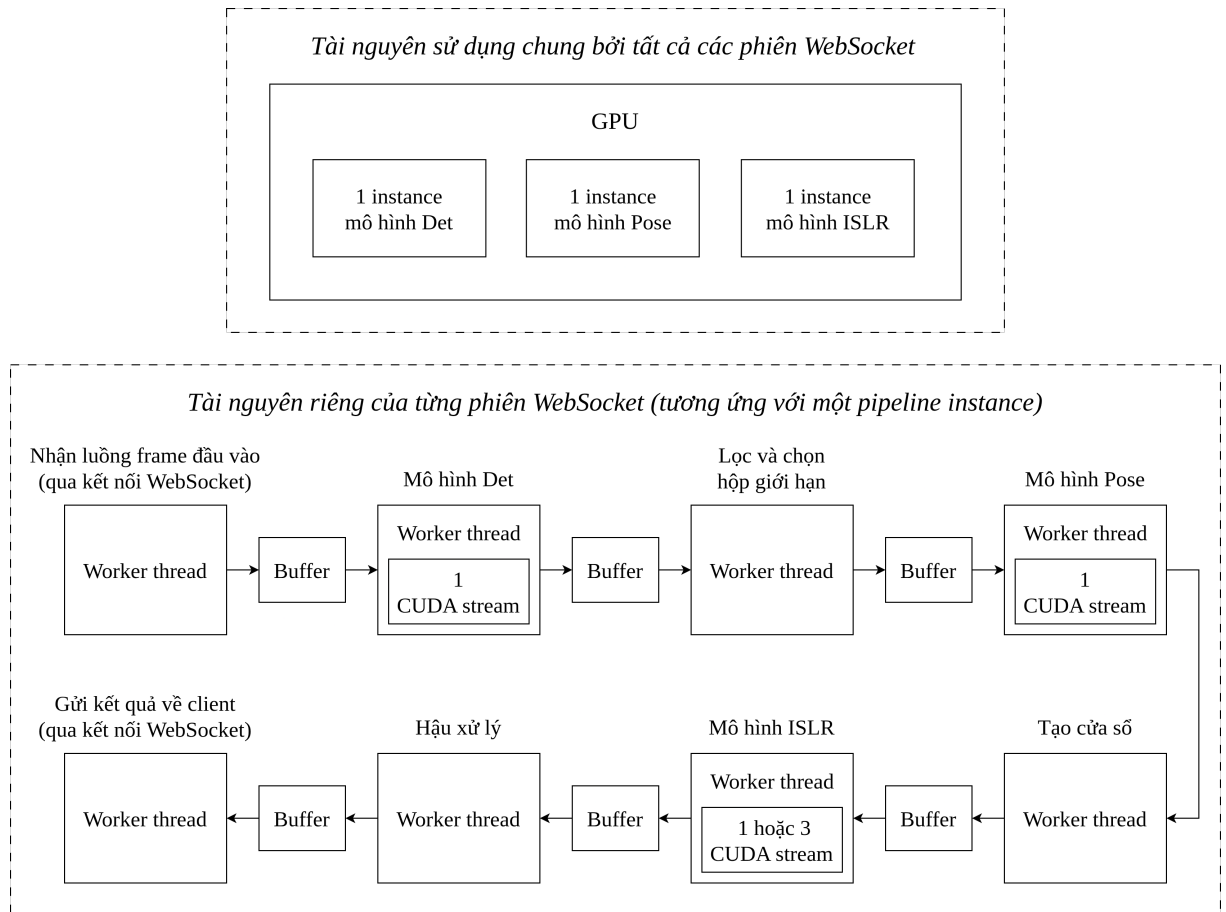
3.5.1 Thiết kế của phiên bản Server suy luận tự xây dựng

Ban đầu, khóa luận thử nghiệm tự xây dựng một Server suy luận đơn giản để kiểm chứng luồng xử lý của toàn bộ pipeline. Phiên bản này sử dụng WebSocket để giao tiếp với Bot server. Ở giai đoạn này, trọng tâm của Server suy luận là đảm bảo các mô hình có thể được nạp vào GPU, các bước xử lý có thể được kết nối với nhau, và kết quả nhận dạng có thể được trả về cho client.

Khi server khởi động, các mô hình trong pipeline được nạp sẵn vào GPU. Mỗi mô hình chỉ có một bản sao được nạp vào bộ nhớ GPU, tức là mỗi mô hình chỉ có một instance, hay gọi đầy đủ hơn là một *model instance*. Việc nạp mô hình một lần khi khởi động giúp tránh chi phí khởi tạo lại mô hình cho từng phiên WebSocket. Sau khi được nạp, các phiên WebSocket cùng sử dụng chung các instance này.

Khi có phiên WebSocket mới, server tạo một worker thread cho mỗi bước xử lý trong pipeline để thực hiện các xử lý của bước đó. Các worker này trao đổi dữ liệu với nhau thông qua các buffer theo thiết kế ở Mục 3.4. Cụ thể, mỗi buffer là một hàng đợi thread-safe có khả năng lấy ra N phần tử liên tiếp ở đầu hàng đợi. Ở đây, khóa luận gọi các tài nguyên worker thread và buffer cần thiết để thực thi pipeline là một *pipeline instance*. Như vậy, mỗi phiên WebSocket sẽ sở hữu một pipeline instance. Ngoài ra, server còn tạo một worker thread để nhận luồng frame đầu vào qua kết nối WebSocket đã tạo, và một worker thread để gửi kết quả gloss về client qua cùng kết nối đó. Hình 3.11 thể hiện các tài nguyên được sử dụng trong từng bước xử lý.

Trong quá trình phát triển, Server suy luận tự xây dựng có hai phiên bản chính.



Hình 3.11: Thiết kế của Server suy luận tự xây dựng

Ở phiên bản thứ nhất, mỗi worker thực thi mô hình học sâu (Det, Pose và ISLR) sử dụng một CUDA stream riêng để gửi tác vụ suy luận lên GPU. Các CUDA stream này tạo điều kiện cho việc chồng lán tính toán giữa các mô hình.

Ở phiên bản thứ hai, worker của mô hình Det và worker của mô hình Pose vẫn sử dụng một CUDA stream cho mỗi worker, trong khi worker của mô hình ISLR được cải tiến để sử dụng ba CUDA stream riêng. Ba stream này lần lượt phục vụ cho ba loại tác vụ: sao chép dữ liệu đầu vào từ host sang device, thực hiện suy luận trên GPU và sao chép kết quả từ device về host. Mục tiêu của phiên bản này là tăng thêm khả năng chồng lán giữa truyền dữ liệu và tính toán cho mô hình ISLR. Nói cách khác, ngoài khả năng chồng lán tính toán giữa các mô hình, phiên bản thứ hai còn có khả năng chồng lán giữa sao chép và tính toán trong worker của mô hình ISLR.

Việc cải tiến này được áp dụng trước cho mô hình ISLR vì với mã nguồn của tác giả, khóa luận có thể trực tiếp can thiệp vào cách tổ chức các thao tác H2D copy

của tensor đầu vào, tính toán mô hình và D2H copy của tensor đầu ra. Đối với mô hình Det và Pose, các thao tác này được xử lý nội bộ trong thư viện và ẩn sau API được cung cấp bởi hai thư viện MMDetection và MMPose. Việc thêm cải tiến này cho Det và Pose sẽ cần đến việc xem xét và chỉnh sửa nội dung bên trong các thư viện, do đó chưa được thực hiện trong phạm vi khóa luận.

Nhìn chung, cả hai phiên bản của server tự xây dựng đều phù hợp với vai trò phiên bản thử nghiệm. Chúng cho phép kiểm chứng pipeline nhận dạng, đánh giá khả năng chạy các mô hình trên GPU và thử nghiệm một số kỹ thuật chồng lán thực thi bằng CUDA stream. Tuy nhiên, cả hai phiên bản vẫn giữ cùng một cách tổ chức tổng thể: mỗi phiên WebSocket tạo một pipeline instance riêng, trong khi các phiên chia sẻ cùng GPU và cùng các model instance. Vì vậy, thiết kế ban đầu này chưa giải quyết triệt để các vấn đề lập lịch và quản lý tài nguyên khi hệ thống cần phục vụ nhiều phiên kết nối đồng thời.

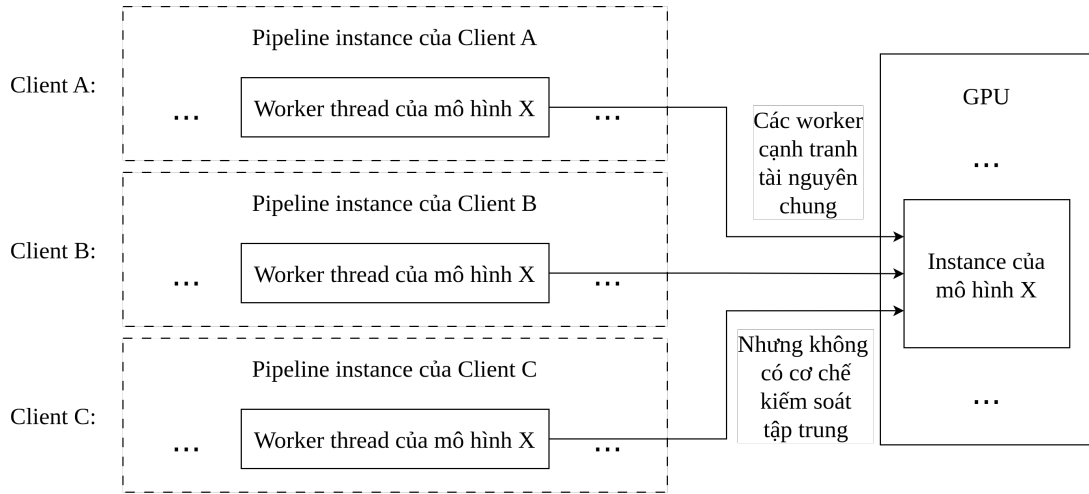
3.5.2 Hạn chế của thiết kế ban đầu

Cả hai phiên bản của server tự xây dựng có cùng bản chất kiến trúc. Cụ thể, chúng đều là các thiết kế *hướng client*. Trong thiết kế này, mỗi phiên WebSocket sở hữu một pipeline instance riêng, gồm các worker thread và buffer trung gian của riêng phiên đó. Tuy nhiên, ở mức tài nguyên vật lý, tất cả các phiên vẫn sử dụng chung GPU và các model instance đã được nạp khi server khởi động.

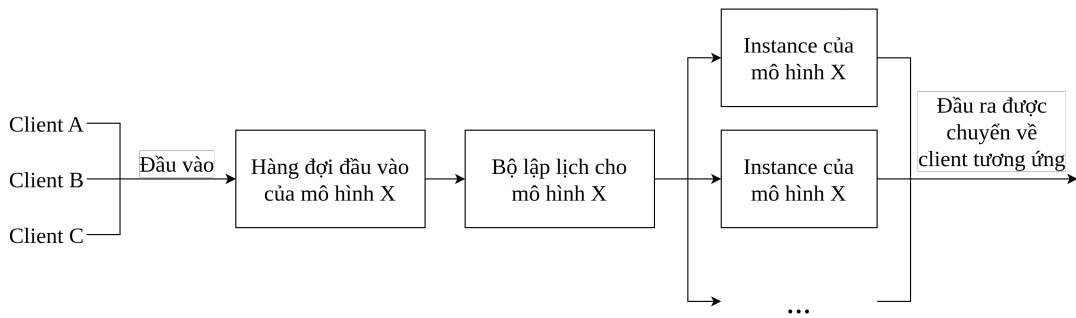
Cách tổ chức này tương đối trực quan khi hệ thống chỉ phục vụ một phiên kết nối. Mỗi client có một luồng video riêng, nên việc tạo một pipeline instance riêng cho client đó giúp việc cài đặt và kiểm thử ban đầu đơn giản hơn. Tuy nhiên, khi số lượng phiên tăng lên, cách tổ chức theo hướng client bộc lộ một số hạn chế. Nguyên nhân chính là phục vụ suy luận không chỉ là bài toán xử lý dữ liệu cho từng client riêng lẻ, mà còn là bài toán quản lý một tập tài nguyên tính toán dùng chung.

Trong một server suy luận, các tài nguyên khan hiếm thường là GPU, bộ nhớ GPU, model instance và khả năng thực thi của từng mô hình. Do đó, việc lập lịch nên được tổ chức xoay quanh các tài nguyên mô hình và tài nguyên GPU, thay vì xoay quanh từng phiên client. Đây có thể xem là cách tiếp cận *hướng tài nguyên*. Trong cách tiếp cận này, các đầu vào từ nhiều client được đưa vào các hàng đợi

chung tương ứng với từng mô hình, sau đó được lập lịch, gom batch và phân phối đến các model instance phù hợp.



(a) Thiết kế hướng client



(b) Thiết kế hướng tài nguyên

Hình 3.12: So sánh thiết kế hướng client và thiết kế hướng tài nguyên

Hạn chế đầu tiên của thiết kế ban đầu là số lượng thread, CUDA stream và buffer tăng tuyến tính theo số phiên kết nối. Vì hệ thống phục vụ nhiều phiên WebSocket đồng thời và mỗi phiên sẽ tạo thêm một bộ worker và buffer tương ứng, nên khi số phiên tăng, số worker thread, số CUDA stream và số buffer cũng tăng theo. Tuy nhiên, năng lực tính toán của GPU không tăng theo số phiên. Điều này khiến hệ thống tạo ra nhiều thực thể thực thi cạnh tranh cho cùng một tài nguyên GPU, nhưng lại thiếu một cơ chế trung tâm để kiểm soát mức độ cạnh tranh đó.

Hạn chế thứ hai là thiết kế ban đầu không có bộ lập lịch toàn cục giữa các phiên. Mỗi pipeline instance của từng phiên chỉ quản lý hàng đợi và tiến trình xử lý cục bộ của chính phiên đó. Trong khi đó, GPU và các model instance là tài nguyên dùng chung. Khi nhiều phiên cùng gửi tác vụ suy luận, server không có một thành

phần trung tâm để quyết định tác vụ nào nên được xử lý trước, tác vụ nào nên chờ, có nên bỏ qua frame cũ hay không, hoặc có nên giới hạn số lượng yêu cầu đang chờ hay không. Điều này có thể làm độ trễ tăng không kiểm soát khi tải tăng.

Một hệ quả trực tiếp của việc thiếu lập lịch toàn cục là hàng đợi có thể tích tụ khi một bước trong pipeline trở thành nút nghẽn. Ví dụ, nếu bước ước lượng tư thế xử lý chậm hơn bước phát hiện người ký hiệu, các kết quả phát hiện người ký hiệu có thể tích tụ trong buffer trước worker của mô hình Pose. Để tránh buffer này tích tụ quá nhiều kết quả trung gian, có thể áp dụng một chính sách như giới hạn số kết quả tối đa được lưu trữ, và nếu cần thêm chỗ cho kết quả mới, thì bỏ đi kết quả cũ nhất. Mặc dù chính sách này đã giới hạn được kích cỡ của các buffer ở các pipeline instance riêng biệt, nó vẫn chưa kiểm soát được tổng kích cỡ ở mức toàn cục. Do đó, nếu có quá nhiều phiên WebSocket khác nhau, mặc dù từng buffer đã được giới hạn kích cỡ, nhưng tổng kích cỡ của tất cả các buffer vẫn có thể gây tràn bộ nhớ.

Hạn chế thứ ba là thiết kế ban đầu khó tận dụng việc gom batch (batching) giữa nhiều phiên. Trong thiết kế hướng client, mỗi phiên WebSocket có pipeline và hàng đợi riêng, nên các input từ nhiều client được xử lý tách biệt. Ví dụ, nếu ba client cùng gửi frame đến mô hình phát hiện người ký hiệu, server sẽ xử lý các frame này qua các worker riêng của từng phiên. Trong khi đó, từ góc nhìn của GPU, các frame từ nhiều client có thể được gom thành một batch chung và đưa vào cùng một lần suy luận. Với các mô hình học sâu, xử lý theo batch thường giúp tận dụng GPU hiệu quả hơn so với nhiều lần suy luận đơn lẻ rời rạc.

Một hạn chế khác là server ban đầu chưa có chính sách mở rộng riêng cho từng mô hình. Trong pipeline, mỗi mô hình có chi phí tính toán khác nhau. Ví dụ, mô hình ISLR có thể là bước tốn nhiều thời gian hơn so với các bước khác. Với thiết kế ban đầu, số worker chủ yếu tăng theo số phiên WebSocket, không phải theo đặc điểm tải của từng mô hình. Do đó, hệ thống chưa có cách rõ ràng để cấu hình năng lực thực thi riêng cho từng bước, chẳng hạn như tăng số instance cho mô hình đang là nút nghẽn trong khi giữ nguyên số instance của các mô hình còn lại.

Hình 3.12 so sánh hai hướng thiết kế lập lịch và quản lý thực thi suy luận. Ở thiết kế hướng client, mặc dù mỗi client sở hữu một pipeline instance riêng, các worker của các client khác nhau vẫn cạnh tranh các tài nguyên chung như model instance và GPU. Do không có cơ chế lập lịch toàn cục ở mức tài nguyên, thiết kế này khó

kiểm soát việc phân phối tải giữa các client, thứ tự xử lý request, sự tích tụ ở các buffer và mức sử dụng GPU. Ngoài ra, thiết kế này cũng không tận dụng việc gom batch giữa các đầu vào đến từ nhiều client khác nhau. Ngược lại, trong thiết kế hướng tài nguyên, các đầu vào từ nhiều client được đưa về hàng đợi chung ở mức mô hình. Từ đó, bộ lập lịch có thể gom batch, phân phối đầu vào đến các model instance và áp dụng các chính sách kiểm soát tải dựa trên lượng tài nguyên hiện có. Cách tổ chức này phù hợp hơn với phục vụ suy luận, vì tài nguyên cần quản lý chính là mô hình, model instance và GPU, thay vì từng phiên client riêng lẻ.

Từ các phân tích trên, có thể thấy server tự xây dựng phù hợp với vai trò thử nghiệm và kiểm chứng pipeline, nhưng chưa phù hợp để đóng vai trò server suy luận lâu dài khi hệ thống cần xử lý nhiều phiên kết nối đồng thời. Vấn đề chính không nằm ở việc pipeline có chạy được hay không, mà ở việc thiết kế ban đầu thiếu các cơ chế lập lịch và quản lý tài nguyên ở mức server.

3.5.3 Cơ chế lập lịch và quản lý của Triton Inference Server

Để khắc phục các hạn chế trên, hệ thống chuyển sang sử dụng NVIDIA Triton Inference Server làm thành phần phục vụ mô hình. Cụ thể, Triton đảm nhiệm vai trò nạp mô hình, nhận yêu cầu suy luận, lập lịch thực thi và quản lý các model instance trên tài nguyên phần cứng.

Điểm khác biệt quan trọng là Triton tổ chức thực thi theo hướng tài nguyên. Các request từ nhiều client không tạo ra các worker suy luận riêng gắn trực tiếp với từng client hay từng phiên giao tiếp mạng. Thay vào đó, chúng được gửi đến các mô hình được triển khai trên Triton và được xử lý thông qua các cơ chế phục vụ mô hình chung [62]. Nhờ đó, việc thực thi suy luận được tổ chức quanh các tài nguyên mô hình và model instance, thay vì quanh từng kết nối với client.

Cơ chế đầu tiên là *per-model scheduler*. Mỗi mô hình được triển khai trên Triton có bộ lập lịch riêng để tiếp nhận và quản lý các request gửi đến mô hình đó [62]. Điều này giúp các yêu cầu từ nhiều client khác nhau có thể được đưa vào cùng một hàng đợi ở mức mô hình. Từ đó, server có cơ sở để quản lý thời gian chờ, phân phối yêu cầu đến model instance và kiểm soát cách mỗi mô hình sử dụng tài nguyên tính toán. So với thiết kế ban đầu, đây là sự thay đổi quan trọng: thay vì

mỗi phiên WebSocket sở hữu worker riêng, các request được gom về các điểm lập lịch chung tương ứng với từng mô hình.

Cơ chế thứ hai là *dynamic batching*. Với các mô hình hỗ trợ batch, Triton có thể gom nhiều request độc lập thành một batch trước khi đưa vào thực thi [65]. Cơ chế này đặc biệt hữu ích khi nhiều client gửi request đến cùng một mô hình trong khoảng thời gian gần nhau. Thay vì thực hiện nhiều lần suy luận đơn lẻ, Triton có thể tạo một batch chung và gọi mô hình một lần trên batch đó. Điều này giúp tăng khả năng tận dụng GPU và cải thiện throughput trong nhiều trường hợp.

Cơ chế thứ ba là Triton cho phép cấu hình số *model instance* riêng cho từng mô hình. Điều này giúp hệ thống mở rộng năng lực thực thi theo đặc điểm của từng bước trong pipeline [88]. Nếu một mô hình là nút nghẽn, có thể cấu hình nhiều instance hơn cho mô hình đó, trong khi các mô hình nhẹ hơn có thể dùng ít instance hơn. Cách làm này khác với thiết kế ban đầu, nơi số worker chủ yếu phụ thuộc vào số phiên WebSocket. Với Triton, năng lực thực thi được cấu hình theo từng tài nguyên mô hình, phù hợp hơn với bản chất của bài toán phục vụ mô hình.

Một điểm đáng lưu ý là Triton cũng hỗ trợ cơ chế *lập lịch ensemble* để thực thi pipeline nhiều mô hình bên trong Triton. Bộ lập lịch ensemble có nhiệm vụ định tuyến các kết quả trung gian giữa các mô hình thành phần theo cấu hình pipeline. Tuy nhiên, điểm quan trọng là các mô hình thành phần trong ensemble vẫn được thực thi thông qua cơ chế phục vụ của chính mô hình đó, bao gồm bộ lập lịch riêng, cơ chế gom batch và cấu hình và model instance riêng. Vì vậy, mặc dù ensemble cũng biểu diễn một pipeline, nó không giống với thiết kế hướng client của server tự xây dựng. Ensemble biểu diễn pipeline ở mức logic, còn việc thực thi vẫn được tổ chức quanh các tài nguyên mô hình. Do đó, Triton ensemble vẫn giữ được tính hướng tài nguyên ở mức thực thi [64].

Ngoài ra, Triton còn sử dụng các backend chuyên dụng để thực thi các mô hình. Các backend này đã được thiết kế để sử dụng các CUDA stream cho các tác vụ truyền dữ liệu và tính toán mô hình nhằm khai thác tối đa tiềm năng của GPU [62]. Vì vậy, việc sử dụng lại Triton cũng giúp giảm nhu cầu tự quản lý CUDA stream trong thiết kế server suy luận.

3.5.4 Lý do chuyển sang sử dụng Triton

Từ các phân tích trên, lý do chính của việc chuyển sang Triton là chuyển cách tổ chức thực thi suy luận từ hướng client sang hướng tài nguyên. Trong server tự xây dựng, mỗi phiên WebSocket tạo một pipeline instance riêng, trong khi tất cả các phiên vẫn chia sẻ cùng GPU và các model instance. Cách tổ chức này đơn giản khi kiểm chứng với một phiên kết nối, nhưng không cung cấp đầy đủ cơ chế lập lịch, gom batch và quản lý tài nguyên khi số phiên tăng lên.

Triton cung cấp các cơ chế phục vụ mô hình chuyên dụng hơn, bao gồm per-model scheduler, dynamic batching, model instance và các công cụ đo đặc hiệu năng. Các cơ chế này cho phép hệ thống tổ chức thực thi quanh từng mô hình và tài nguyên GPU, thay vì gắn trực tiếp việc thực thi với từng phiên client. Nhờ đó, request từ nhiều client có thể được quản lý ở mức mô hình, có khả năng được gom batch, và có thể được phân phối đến các model instance phù hợp.

Tóm lại, server suy luận tự xây dựng ban đầu có vai trò quan trọng trong việc kiểm chứng pipeline và thử nghiệm các ý tưởng tối ưu ở mức CUDA stream. Tuy nhiên, thiết kế này vẫn theo hướng client và thiếu các cơ chế cần thiết để quản lý thực thi suy luận khi có nhiều phiên kết nối. Do đó, việc chuyển sang NVIDIA Triton Inference Server là phù hợp hơn với mục tiêu của hệ thống. Triton giúp hệ thống tổ chức thực thi theo hướng tài nguyên thông qua việc tận dụng các cơ chế phục vụ mô hình chuyên dụng.

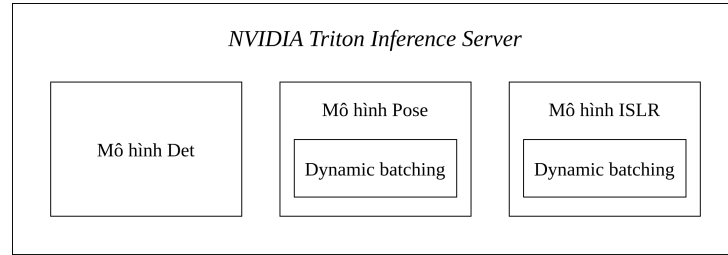
3.6 Thiết kế tổ chức suy luận trên Triton

Sau khi quyết định sử dụng Triton làm Server suy luận của hệ thống, khóa luận tiếp tục với việc trình bày thiết kế tổ chức suy luận trên Triton theo hai phương án đã được đưa ra ở Mục 3.3.3.

3.6.1 Phương án phục vụ từng mô hình riêng lẻ

Trong phương án thứ nhất, Triton phục vụ Det, Pose và ISLR như ba mô hình độc lập như được minh họa trên Hình 3.13. Trong đó, mô hình Pose và ISLR sử dụng cơ chế dynamic batching của Triton để cung cấp khả năng gom batch các đầu vào từ nhiều request khác nhau. Trong cấu hình triển khai của khóa luận, mô hình Det

chưa sử dụng dynamic batching do đầu ra của mô hình có cấu trúc phức tạp hơn và việc tích hợp batching cho bước này chưa nằm trong phạm vi hiện thực chính.



Hình 3.13: Phương án Triton chỉ phục vụ các mô hình học sâu

Ưu điểm đầu tiên của phương án này là dễ hiện thực logic điều phối. Các điều kiện như “nếu frame không có hộp giới hạn hợp lệ thì bỏ qua bước ước lượng cho frame đó” có thể được cài đặt trực tiếp ở Bot server. Bot server có thể quyết định khi nào cần gọi model nào, khi nào cần bỏ qua một bước và khi nào cần tạo cửa sổ.

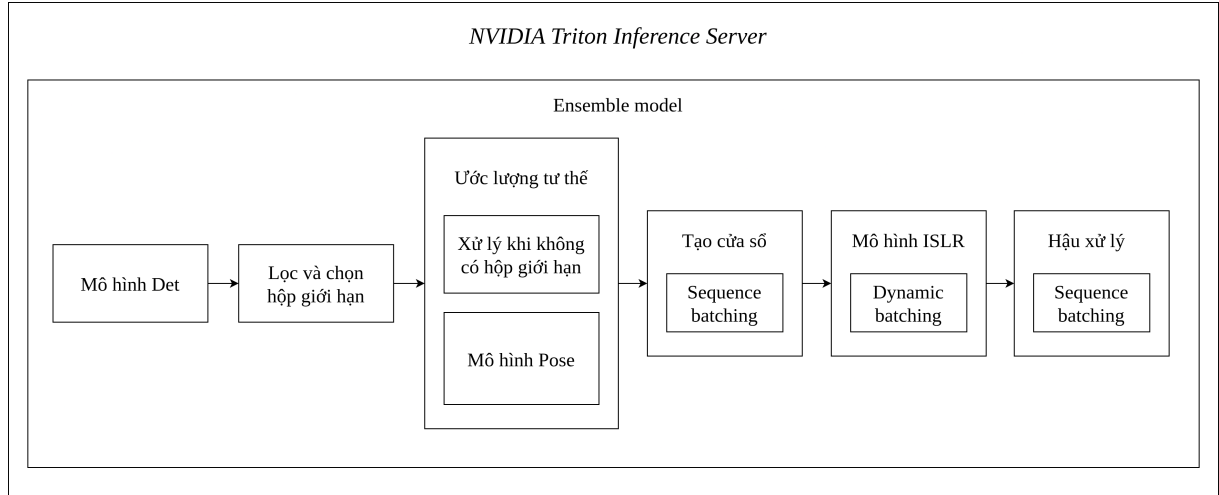
Ưu điểm thứ hai là dễ debug và quan sát đầu ra trung gian. Vì Bot server nhận kết quả sau từng bước, hệ thống có thể ghi log, kiểm tra tensor trung gian, lưu đầu ra của từng mô hình hoặc gọi riêng từng mô hình để kiểm thử. Điều này hữu ích khi pipeline gồm nhiều bước và lỗi có thể xuất hiện ở bất kỳ bước nào.

Ưu điểm thứ ba là tính linh hoạt triển khai. Các mô hình thành phần có thể được triển khai như các dịch vụ độc lập. Khi một mô hình trở thành nghẽn cổ chai, hệ thống có thể tăng số instance hoặc thay thế riêng mô hình đó mà không cần triển khai lại toàn bộ pipeline. Tương tự, việc cập nhật hoặc rollback một mô hình cũng thuận lợi hơn khi các mô hình được phục vụ độc lập.

Tuy nhiên, phương án này có hạn chế lớn về số lần giao tiếp giữa Bot server và Server suy luận. Vì Bot server phải gọi từng mô hình riêng lẻ, mỗi frame hoặc cửa sổ có thể tạo ra nhiều request. Điều này làm phát sinh thêm độ trễ mạng, chi phí serialization/deserialization và chi phí truyền dữ liệu trung gian. Nếu Bot server và Server suy luận không nằm trên cùng một máy hoặc cùng một mạng có độ trễ thấp, chi phí giao tiếp có thể trở thành điểm nghẽn quan trọng.

3.6.2 Phương án phục vụ toàn bộ pipeline

Hình 3.14 minh họa phương án thứ hai, trong đó Server suy luận chứa toàn bộ pipeline từ frame đầu vào đến kết quả gloss đầu ra. Phương án này đóng gói pipeline nhận dạng trong một ensemble model và sử dụng bộ lập lịch ensemble.



Hình 3.14: Phương án Triton phục vụ toàn bộ pipeline

Hai bước tạo cửa sổ và hậu xử lý đều có trạng thái theo thời gian. Bước tạo cửa sổ cần lưu lại một số frame và keypoint từ các lần thực thi trước để kết hợp với dữ liệu mới, từ đó tạo ra các cửa sổ đầu vào cho ISLR. Tương tự, bước hậu xử lý cần lưu lại một số dự đoán gloss thô trước đó để kết hợp với các dự đoán mới, từ đó tạo ra các voting bag. Do đó, hai bước này cần sử dụng các buffer để lưu lại các giá trị cũ cần thiết.

Vì Server suy luận có thể xử lý nhiều luồng video đến từ nhiều client hoặc nhiều phiên nhận dạng khác nhau, các bước có trạng thái là tạo cửa sổ và hậu xử lý cần một cơ chế phân biệt dữ liệu thuộc các phiên nhận dạng khác nhau. Nếu không có cơ chế này, dữ liệu từ các nguồn khác nhau có thể bị đưa vào cùng một buffer, làm sai lệch kết quả nhận dạng. Do đó, các bước tạo cửa sổ và hậu xử lý được thiết kế để sử dụng cơ chế sequence batching của Triton [66]. Mỗi phiên nhận dạng được gán một sequence ID riêng; các request thuộc cùng một phiên được gửi kèm sequence ID này. Nhờ cơ chế sequence batching, Triton có thể quản lý các request theo từng chuỗi và bảo đảm dữ liệu của các phiên khác nhau không bị trộn lẫn khi được xử lý bởi các bước có trạng thái.

Ưu điểm chính của phương án này là giảm số lần giao tiếp qua mạng. Thay vì Bot server phải gọi từng mô hình riêng lẻ, Bot server chỉ gửi dữ liệu đầu vào và nhận kết quả cuối cùng. Các tensor trung gian như hộp giới hạn, keypoint hoặc dự đoán thô không cần được truyền ra ngoài Triton. Điều này có tiềm năng giảm độ trễ tổng thể, đặc biệt trong trường hợp Bot server và Server suy luận được triển khai trên các máy khác nhau.

Phương án này cũng đóng gói toàn bộ pipeline thành một mô hình logic duy nhất từ góc nhìn của Bot server. Bot server không cần biết chi tiết từng bước bên trong pipeline, mà chỉ cần gửi dữ liệu đầu vào và nhận gloss đầu ra. Điều này giúp giảm trách nhiệm điều phối của Bot server.

Tuy nhiên, phương án này phức tạp hơn trong triển khai. Các logic điều kiện như bỏ qua bước ước lượng tư thế khi không có hộp giới hạn hợp lệ hoặc duy trì trạng thái để tạo cửa sổ thời gian cần được đưa vào bên trong Triton. Việc debug các tensor trung gian cũng khó hơn vì dữ liệu được truyền nội bộ giữa các bước. Ngoài ra, khi toàn bộ pipeline được đóng gói chặt hơn, mức độ coupling giữa các bước tăng lên và việc thay thế từng thành phần riêng lẻ có thể kém linh hoạt hơn so với phương án thứ nhất.

Trong phạm vi khóa luận, phương án này được xem là hướng tối ưu tiềm năng để giảm số lần giao tiếp mạng và cải thiện độ trễ tổng thể, nhưng cũng đòi hỏi nhiều công sức thực hiện hơn.

3.6.3 So sánh hai phương án thiết kế

Bảng 3.3 so sánh hai phương án tổ chức suy luận trên Triton. Từ bảng so sánh, có thể thấy phương án Triton phục vụ từng mô hình riêng lẻ phù hợp hơn cho giai đoạn thử nghiệm ban đầu vì dễ cài đặt, dễ gỡ lỗi và linh hoạt trong việc thay đổi từng mô hình thành phần. Ngược lại, phương án Triton chứa toàn bộ pipeline phù hợp hơn với mục tiêu giảm độ trễ khi triển khai phân tán, do giảm số lần giao tiếp giữa Bot server và Triton. Tuy nhiên, phương án này phức tạp hơn và cần thêm công sức cài đặt. Do đó, trong phạm vi hiện thực của khóa luận, phương án Triton phục vụ từng mô hình riêng lẻ được lựa chọn làm phương án triển khai chính, còn phương án Triton chứa toàn bộ pipeline được xem là hướng tối ưu hóa tiếp theo.

Bảng 3.3: So sánh hai phương án tổ chức suy luận trên Triton

Tiêu chí	Triton phục vụ từng mô hình riêng lẻ	Triton chứa toàn bộ pipeline
Độ phức tạp cài đặt	Thấp hơn	Cao hơn
Mức độ linh hoạt triển khai	Cao hơn; mô hình có thể được triển khai độc lập	Thấp hơn; các bước pipeline phụ thuộc chặt hơn
Mở rộng từng mô hình	Linh hoạt hơn	Bị ràng buộc nhiều hơn bởi tổ chức pipeline
Logic điều phối phức tạp	Phù hợp hơn	Khó hơn nếu có nhiều logic điều kiện
Mức độ phụ thuộc giữa các bước	Thấp hơn	Cao hơn
Số request mạng	Nhiều hơn	Ít hơn
Truyền dữ liệu trung gian	Có chi phí truyền dữ liệu trung gian	Tránh đưa tensor trung gian ra ngoài Server suy luận
Tiềm năng giảm độ trễ khi triển khai từ xa	Thấp hơn	Cao hơn
Trạng thái trong khóa luận	Đã thử nghiệm	Hướng hoàn thiện tiếp theo

3.7 Thiết kế điều khiển phiên nhận dạng

Để thực hiện các tác vụ liên quan đến phiên nhận dạng, ứng dụng frontend và bot giao tiếp với nhau thông qua cơ chế RPC của LiveKit. Cơ chế này phù hợp với hệ thống vì bot cùng tham gia phòng gọi với các ứng dụng frontend của người dùng. Do RPC của LiveKit được thực hiện giữa các thành viên trong cùng phòng, Bot server không cần tự xây dựng một endpoint riêng để nhận các lệnh điều khiển từ frontend hoặc tự quản lý việc định tuyến request theo phòng gọi [72].

Sau khi tham gia phòng, mỗi bot sẽ cung cấp các hàm RPC phục vụ việc điều khiển phiên nhận dạng cho phòng gọi, bao gồm `start_sign_recognition`, `pause_sign_recognition`, `resume_sign_recognition` và `stop_sign_recognition`. Các hàm

này tương ứng với các nút bắt đầu phiên nhận dạng, tạm dừng phiên nhận dạng, tiếp tục phiên nhận dạng và kết thúc phiên nhận dạng trong ứng dụng frontend.

Bên cạnh đó, mỗi ứng dụng frontend khi tham gia phòng gọi còn cung cấp hàm RPC `update_signer`. Mỗi khi có phiên nhận dạng mới, bot sẽ gọi hàm này ở tất cả các ứng dụng frontend tham gia cùng phòng gọi để thông báo mã định danh của người đang sử dụng phiên nhận dạng. Từ đó, ứng dụng frontend có thể làm nổi bật video của người này trên giao diện phòng gọi.

Ngoài ra, bot cũng cung cấp một hàm `get_sign_recognition_state` để cung cấp trạng thái của phiên nhận dạng đang diễn ra (nếu có) cùng với mã định danh của người được nhận dạng. Khi tham gia phòng, ứng dụng frontend sẽ gọi hàm này, và nếu có phiên nhận dạng đang diễn ra thì sẽ làm nổi bật video của người được nhận dạng tương ứng.

3.8 Thiết kế điều phối suy luận của Bot server

Mỗi bot điều phối quá trình suy luận với Server suy luận thông qua các cơ chế giao tiếp dạng luồng. Cụ thể, bot và Server suy luận giao tiếp với nhau qua giao thức WebSocket ở phiên bản Server suy luận tự xây dựng, và giao thức streaming gRPC khi Server suy luận là Triton. Việc sử dụng các giao thức dạng luồng phù hợp với yêu cầu về tính trực tuyến của hệ thống, trong đó hệ thống cần liên tục nhận frame đầu vào và trả về gloss đầu ra.

Ở phương án thứ nhất trong Mục 3.3.3, bot tổ chức pipeline bằng các worker tương ứng với từng bước xử lý, bao gồm các bước tiền xử lý, gọi các mô hình Det, Pose và ISLR, lọc và chọn hộp giới hạn, tạo cửa sổ và hậu xử lý. Các worker này trao đổi dữ liệu thông qua các buffer trung gian đã được thiết kế ở Mục 3.4.3.

3.9 Thiết kế kiến trúc phân lớp của ứng dụng frontend

Ứng dụng frontend của hệ thống được phát triển bằng Flutter nhằm xây dựng một ứng dụng gọi video có thể chạy trên nhiều nền tảng. Ứng dụng được thiết kế theo kiến trúc phân lớp được khuyến nghị trong phát triển ứng dụng Flutter [81]. Do phạm vi chức năng của ứng dụng không quá phức tạp, frontend được chia thành hai lớp chính: lớp UI và lớp Data. Trên cơ sở kiến trúc phân lớp này, frontend áp

dụng mô hình MVVM để tổ chức mã nguồn. Trong cách tổ chức đó, lớp UI bao gồm View và ViewModel, còn lớp Data cung cấp các thành phần đảm nhiệm vai trò Model trong MVVM.

View là các màn hình và thành phần UI hiển thị giao diện cho người dùng. Trong hệ thống, View bao gồm màn hình nhập thông tin phòng, màn hình chọn camera, màn hình gọi video, bảng điều khiển và vùng hiển thị phụ đề. View chịu trách nhiệm hiển thị trạng thái và gửi thao tác của người dùng đến ViewModel.

ViewModel quản lý trạng thái giao diện và logic liên quan đến giao diện. Các trạng thái này bao gồm trạng thái kết nối phòng, trạng thái camera, trạng thái phiên nhận dạng, video của những người tham gia và kết quả gloss. ViewModel nhận dữ liệu từ Repository và cập nhật trạng thái để View hiển thị.

Repository thuộc lớp Data và chịu trách nhiệm giao tiếp với Server truyền thông thời gian thực. Repository quản lý logic kết nối phòng, chia sẻ nguồn video, gửi lệnh điều khiển phiên nhận dạng và nhận dữ liệu gloss. Ngoài ra, repository cũng là nơi cung cấp các nguồn dữ liệu chính cho ứng dụng, bao gồm dữ liệu về trạng thái phiên nhận dạng. Do đó, các nguồn dữ liệu này có thể được đồng bộ và chia sẻ giữa nhiều ViewModel và View.

Cách tổ chức này giúp frontend có cấu trúc rõ ràng hơn và dễ bảo trì hơn. View chỉ tập trung vào giao diện, ViewModel quản lý trạng thái, còn Repository xử lý giao tiếp với hệ thống bên ngoài. Nhờ đó, các thay đổi ở tầng giao tiếp với Server truyền thông thời gian thực ít ảnh hưởng trực tiếp đến giao diện.

3.10 Thiết kế kiểm soát độ trễ

Việc bảo đảm độ trễ thấp và ổn định là yêu cầu quan trọng trong hệ thống để phục vụ cho bối cảnh giao tiếp trực tuyến. Có nhiều yếu tố ảnh hưởng đến độ trễ cần được xem xét trong quá trình thiết kế.

3.10.1 Nguồn phát sinh độ trễ trong hệ thống

Độ trễ của hệ thống có thể phát sinh ở nhiều vị trí trong toàn bộ luồng xử lý, từ khi hình ảnh được tạo ra ở phía người dùng đến khi kết quả gloss được hiển thị trên ứng dụng frontend. Nguồn độ trễ đầu tiên đến từ quá trình thu nhận và truyền

frame từ nguồn video của người dùng. Trong quá trình nhận dạng, thời gian tiền xử lý dữ liệu, giao tiếp giữa Bot server và Server suy luận, và thời gian thực thi các bước trong pipeline nhận dạng là các nguồn độ trễ được tập trung xử lý bởi khóa luận. Cuối cùng là quá trình truyền kết quả gloss về frontend để hiển thị. Như vậy, độ trễ tổng thể của hệ thống không chỉ phụ thuộc vào tốc độ của các mô hình nhận dạng, mà còn phụ thuộc vào cách tổ chức luồng dữ liệu, buffer trung gian và giao tiếp giữa các thành phần.

3.10.2 Độ trễ tích lũy trong pipeline nhận dạng

Trong hệ thống xử lý video trực tuyến, một vấn đề quan trọng là độ trễ có thể bị tích lũy theo thời gian. Hiện tượng này xảy ra khi tốc độ xử lý trung bình của pipeline thấp hơn tốc độ nhận dữ liệu đầu vào. Khi đó, các frame hoặc cửa sổ video mới tiếp tục được đưa vào hệ thống trong khi các dữ liệu trước đó chưa được xử lý xong, làm cho dữ liệu tồn đọng trong các buffer trung gian ngày càng nhiều.

Nếu mỗi bước trong pipeline xử lý kịp tốc độ dữ liệu đầu vào, độ trễ có thể dao động quanh một mức tương đối ổn định. Ngược lại, nếu một bước trở thành nút thắt cổ chai, dữ liệu sẽ bị dồn lại trước bước đó. Khi đó, dù thời gian xử lý cho từng phần tử dữ liệu có thể không thay đổi nhiều, độ trễ quan sát được ở đầu ra vẫn tăng dần vì mỗi frame hoặc cửa sổ phải chờ lâu hơn trước khi được xử lý. Đây là lý do việc kiểm soát độ trễ trong hệ thống không chỉ là tối ưu từng mô hình riêng lẻ, mà còn cần kiểm soát tốc độ xử lý của toàn bộ pipeline.

3.10.3 Các quyết định thiết kế nhằm giảm độ trễ

Để hỗ trợ mục tiêu xử lý trực tuyến có độ trễ thấp, hệ thống áp dụng một số quyết định thiết kế nhằm giảm thời gian xử lý và hạn chế độ trễ tích lũy trong pipeline.

Thứ nhất, hệ thống kế thừa phương pháp xử lý video theo các cửa sổ ngắn thay vì chờ toàn bộ video kết thúc. Cách tiếp cận này phù hợp với bài toán CSLR trực tuyến, vì hệ thống có thể tạo dự đoán dần theo thời gian khi luồng video đang diễn ra.

Thứ hai, các bước suy luận mô hình được tách khỏi ứng dụng frontend và được thực hiện ở phía Server suy luận. Cách thiết kế này giúp frontend không phải trực tiếp tải và thực thi các mô hình học sâu, vốn yêu cầu tài nguyên tính toán lớn.

Frontend chỉ cần đảm nhiệm các chức năng giao diện, truyền video và hiển thị kết quả, trong khi các bước suy luận nặng được thực hiện ở phía server có GPU.

Thứ ba, khóa luận cân nhắc sử dụng các mô hình Det và Pose nhẹ hơn để giảm thời gian xử lý ở các bước đầu của pipeline. Đây là các bước được thực hiện lặp lại thường xuyên trên luồng video đầu vào, nên thời gian xử lý của chúng có ảnh hưởng trực tiếp đến tốc độ của toàn bộ pipeline.

Thứ tư, giao tiếp giữa Bot server và Server suy luận được thiết kế theo hướng ưu tiên cơ chế dạng luồng, phù hợp với đặc điểm đầu vào video liên tục.

Thứ năm, pipeline và các buffer trung gian được tổ chức theo hướng hỗ trợ xử lý theo batch ở các bước suy luận. Điều này giúp các mô hình học sâu có thể tăng khả năng tận dụng GPU.

Thứ sáu, khóa luận xem xét phương án đưa toàn bộ pipeline nhận dạng vào Server suy luận do có tiềm năng giảm độ trễ giao tiếp giữa Bot server và Server suy luận.

3.10.4 Cân bằng giữa độ trễ, độ ổn định và độ chính xác

Trong hệ thống nhận dạng ngôn ngữ ký hiệu liên tục, việc giảm độ trễ cần được cân bằng với độ ổn định và độ chính xác của kết quả nhận dạng. Một thay đổi giúp giảm thời gian xử lý không nhất thiết luôn cải thiện chất lượng hệ thống tổng thể, vì nó có thể làm mất thông tin thời gian hoặc làm giảm chất lượng đầu vào của mô hình nhận dạng.

Một ví dụ là tham số bước trượt trong quá trình tạo cửa sổ. Khi tăng bước trượt, số lượng cửa sổ cần được tạo và đưa vào mô hình ISLR sẽ giảm, từ đó giảm khối lượng suy luận và có thể giảm độ trễ. Tuy nhiên, nếu bước trượt quá lớn, hệ thống có thể bỏ qua một số đoạn chuyển động quan trọng, làm tăng nguy cơ mất gloss hoặc phát hiện gloss muộn hơn.

Tương tự, việc giảm frame rate hoặc giảm kích thước cửa sổ có thể làm giảm lượng dữ liệu cần xử lý, nhưng cũng có thể ảnh hưởng đến mô hình đã huấn luyện. Mô hình ISLR được huấn luyện với một cách biểu diễn đầu vào nhất định, nên nếu số frame, độ dài cửa sổ hoặc nhịp lấy mẫu thay đổi quá nhiều có thể dẫn đến kết quả nhận dạng sai lệch. Ví dụ, quá trình huấn luyện mô hình ISLR giữ cố định frame rate của video đầu vào là 25 FPS [14]. Việc giảm frame rate có thể trực tiếp giảm

số frame cần được xử lý, nhưng lại thay đổi phân phối frame đầu vào so với phân phối mà mô hình đã học.

Việc sử dụng mô hình Det và Pose nhẹ hơn cũng tạo ra sự đánh đổi. Các mô hình nhẹ có thể giúp tăng tốc độ xử lý và giảm tải GPU, nhưng nếu chất lượng phát hiện người hoặc chất lượng keypoint giảm, mô hình ISLR phía sau có thể nhận đầu vào kém chính xác hơn. Khi đó, độ trễ giảm nhưng kết quả gloss có thể kém ổn định hoặc kém chính xác.

Batch size cũng là một tham số cần cân nhắc. Batch size lớn có thể giúp tận dụng GPU tốt hơn và tăng throughput của các bước suy luận. Tuy nhiên, nếu hệ thống phải chờ đủ batch trước khi xử lý, độ trễ của từng frame hoặc từng cửa sổ có thể tăng lên. Vì vậy, batch size cần được thiết kế phù hợp với tốc độ đầu vào, năng lực xử lý của GPU và yêu cầu độ trễ của hệ thống.

Do đó, mục tiêu của thiết kế không phải là giảm độ trễ bằng mọi giá, mà là đạt được sự cân bằng hợp lý giữa độ trễ, độ ổn định và chất lượng nhận dạng. Trong phạm vi khóa luận, các quyết định thiết kế giảm độ trễ được đưa ra theo hướng ưu tiên những tối ưu mà vẫn giữ được tính chất của đầu vào mà mô hình nhận dạng đã được huấn luyện.

Chương 4

Cài đặt, thực nghiệm và đánh giá hệ thống

Chương này trình bày quá trình cài đặt, thực nghiệm và đánh giá hệ thống dựa trên thiết kế đã được mô tả ở Chương 3. Nội dung chương bao gồm cách cài đặt các thành phần chính của hệ thống, quá trình triển khai pipeline nhận dạng và các thực nghiệm đánh giá liên quan đến độ chính xác và độ trễ.

Trong quá trình phát triển, khóa luận thực hiện các thực nghiệm nhằm kiểm tra tính khả thi của từng giai đoạn triển khai, phát hiện các vấn đề kỹ thuật phát sinh và đưa ra hướng phát triển tiếp theo. Bên cạnh đó, khóa luận cũng thực hiện hai thực nghiệm đánh giá chính. Thực nghiệm thứ nhất đánh giá ảnh hưởng của việc thay thế mô hình Det và Pose bằng các mô hình có tốc độ xử lý cao hơn thông qua chỉ số WER. Thực nghiệm thứ hai đánh giá độ trễ của pipeline khi triển khai client thử nghiệm và Triton Inference Server trên cùng một máy, qua đó hạn chế ảnh hưởng của chi phí giao tiếp client–server và tập trung hơn vào độ trễ xử lý của pipeline suy luận. Các kết quả này được sử dụng để phân tích mức đánh đổi giữa tốc độ xử lý, độ ổn định độ trễ và chất lượng nhận dạng của hệ thống.

4.1 Phiên bản hiện thực ban đầu của pipeline nhận dạng

Trước khi đi sâu vào thiết kế tổ chức pipeline trên Server suy luận, khóa luận cần xác định liệu pipeline có khả năng xử lý liên tục luồng video đầu vào để thỏa mãn yêu cầu về tính trực tuyến của hệ thống hay không. Đây là bước quan trọng vì mã nguồn gốc của Zuo và cộng sự chủ yếu phục vụ đánh giá trên dữ liệu đã tiền xử lý, trong khi hệ thống của khóa luận cần xử lý trực tiếp luồng frame đầu vào theo hướng gần thời gian thực.

Cụ thể, mã nguồn gốc không trực tiếp xử lý luồng video theo từng frame trong quá trình suy luận trực tuyến. Thay vào đó, mã nguồn này thực hiện tiền xử lý toàn bộ tập dữ liệu bằng cách đưa các video qua các mô hình phát hiện đối tượng và ước lượng tư thế để trích xuất keypoint, sau đó lưu lại kết quả. Trong quá trình

huấn luyện hoặc đánh giá, chương trình nạp lại các frame và keypoint đã được tạo sẵn, sau đó thực hiện tạo cửa sổ, chạy mô hình ISLR và hậu xử lý [14, 58, 89].

Cách tổ chức này phù hợp với mục tiêu đánh giá trên benchmark, nhưng chưa phản ánh đầy đủ yêu cầu của một hệ thống trực tuyến hoặc gần thời gian thực, trong đó frame video cần được tiếp nhận, xử lý và đưa vào pipeline một cách liên tục. Vì vậy, khóa luận hiện thực lại pipeline theo hướng xử lý trực tiếp trên luồng frame đầu vào. Ở giai đoạn này, khóa luận chỉ tổ chức lại pipeline, còn định dạng và cách thực thi các mô hình vẫn được giữ theo mã nguồn của tác giả. Khóa luận chưa đưa các mô hình về định dạng tối ưu hơn và chưa triển khai thông qua Server suy luận chuyên dụng. Mục tiêu chính của giai đoạn này là tổ chức lại pipeline nhận dạng theo hướng gần với điều kiện triển khai thực tế, đồng thời kiểm tra xem pipeline có thể xử lý luồng video với độ trễ phù hợp hay không.

Phiên bản thử nghiệm ban đầu của pipeline được cài đặt theo thiết kế lập lịch và tổ chức thực thi tương tự thiết kế của Server suy luận phiên bản tự xây dựng đã được trình bày ở Mục 3.5.1. Điểm khác biệt chính là thay vì nhận frame đầu vào qua kết nối WebSocket, phiên bản này đọc luồng frame từ một tệp video có sẵn trên máy chạy.

4.2 Thử nghiệm ban đầu

Các thực nghiệm đầu tiên của khóa luận được thực hiện trên phiên bản hiện thực ở Mục 4.1 nhằm đánh giá tính khả thi của phương pháp của Zuo và cộng sự [14] trước khi quyết định kế thừa phương pháp này làm thành phần nhận dạng cốt lõi của hệ thống. Cụ thể, khóa luận thực hiện đo độ trễ của toàn bộ pipeline nhận dạng, sau đó thực hiện một số tối ưu để giảm độ trễ.

Trong thử nghiệm ban đầu, một video trong tập kiểm thử của PHOENIX-2014T được nạp và truyền vào pipeline với tốc độ 25 FPS để mô phỏng luồng video đầu vào. Trong thử nghiệm này và tất cả các thử nghiệm còn lại, giá trị kích cỡ cửa sổ luôn giữ nguyên là 16 theo cấu hình của pipeline gốc. Bước trượt được đặt là 1. Theo mã nguồn gốc [58], pipeline sử dụng mô hình phát hiện đối tượng là Faster R-CNN và ước lượng tư thế là HRNet. Thử nghiệm ban đầu được thực hiện trên GPU RTX 3090, với kích cỡ batch của Det, Pose và ISLR lần lượt là 8, 1 và 8. Cả

ba mô hình đều trải qua 20 bước khởi động trước khi bắt đầu đo. Để đơn giản hóa thử nghiệm, kích cỡ voting bag của bước hậu xử lý được đặt bằng 1.

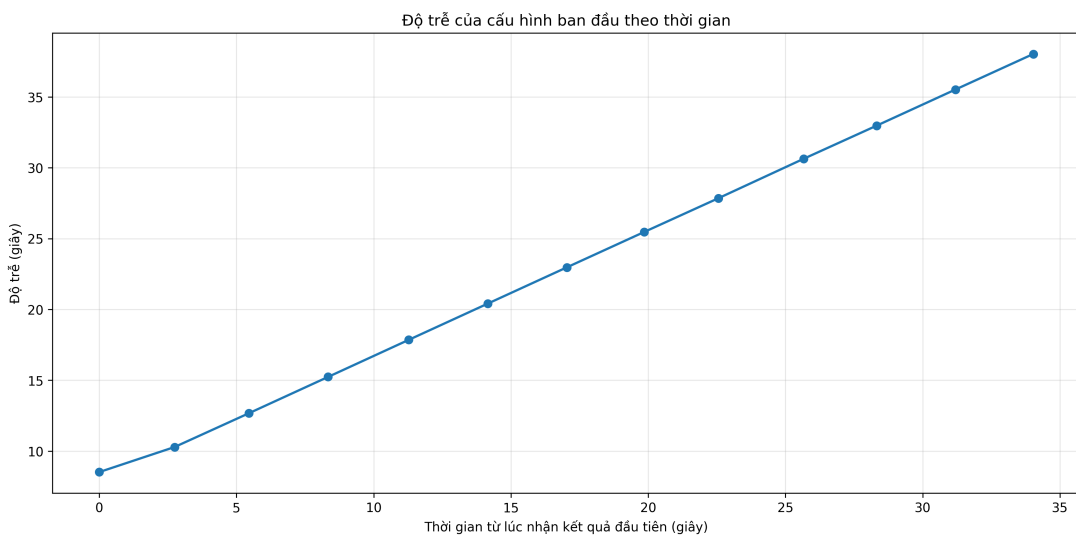
4.2.1 Phương pháp đo độ trễ

Để đánh giá khả năng vận hành gần thời gian thực của pipeline, khóa luận đo độ trễ ở mức cửa sổ thời gian. Cụ thể, độ trễ của một cửa sổ được tính bằng hiệu giữa thời điểm cửa sổ đó được xử lý xong bởi pipeline và thời điểm frame đầu tiên của cửa sổ bắt đầu đi vào pipeline.

Cách đo này phản ánh khoảng thời gian từ khi dữ liệu đầu tiên cần thiết cho một cửa sổ xuất hiện trong hệ thống đến khi hệ thống tạo được kết quả nhận dạng cho cửa sổ đó. Do mô hình ISLR nhận đầu vào là một đoạn ngắn theo thời gian chứ không phải một frame đơn lẻ, việc đo độ trễ ở mức cửa sổ phù hợp hơn so với việc đo riêng thời gian xử lý từng frame.

4.2.2 Kết quả thử nghiệm

Kết quả thử nghiệm ở Hình 4.1 cho thấy độ trễ của pipeline ở cấu hình ban đầu rất cao. Ngay kết quả đầu tiên, độ trễ đã đạt hơn 8 giây; với kết quả thứ ba, độ trễ tăng lên khoảng 12 giây. Mặc dù độ trễ cũng có xu hướng tăng theo thời gian, vấn đề nổi bật nhất ở giai đoạn này là độ trễ tuyệt đối đã quá cao ngay từ các batch đầu, khiến cấu hình này không phù hợp với mục tiêu gần thời gian thực.



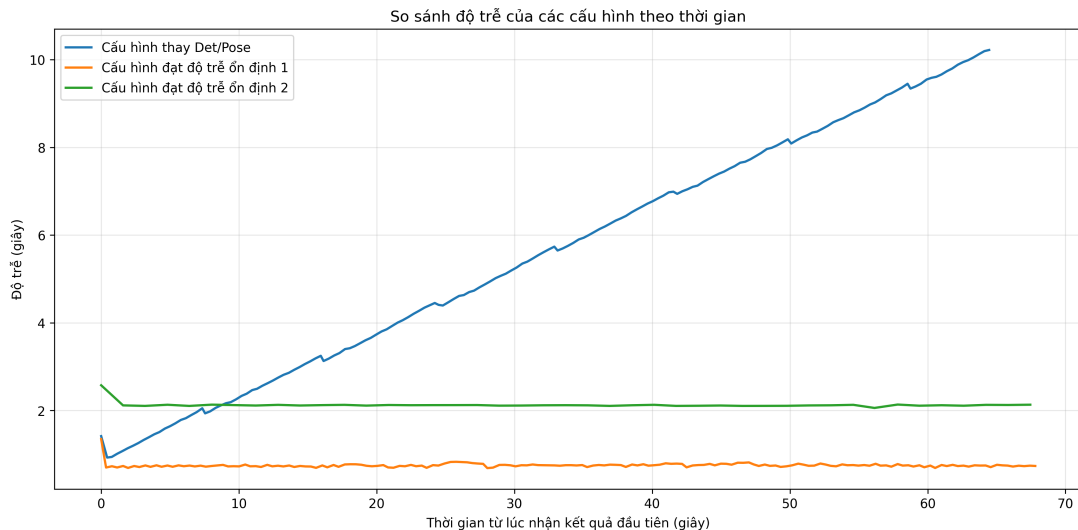
Hình 4.1: Độ trễ theo thời gian của cấu hình thử nghiệm đầu

Một nguyên nhân được khóa luận đưa ra là chi phí xử lý lớn của các mô hình Faster R-CNN và HRNet. Đây là các mô hình có độ chính xác tốt nhưng chi phí tính toán cao, không nhất thiết phù hợp với yêu cầu độ trễ thấp của hệ thống gần thời gian thực. Do đó, trước khi tiếp tục phân tích các hướng tối ưu khác, khóa luận thay thế hai mô hình này bằng các mô hình nhẹ hơn nhằm giảm độ trễ của phần đầu pipeline.

4.3 Phân tích và tối ưu độ trễ

4.3.1 Thiết lập baseline mới với YOLOv3 và ResNet50

Trước khi tiếp tục phân tích các hướng tối ưu khác, khóa luận thử nghiệm thay thế hai mô hình Faster R-CNN và HRNet bằng các mô hình có tốc độ xử lý cao hơn. Cụ thể, YOLOv3 từ MMDetection được sử dụng cho bước phát hiện đối tượng, trong khi ResNet50 từ MMPose được sử dụng cho bước ước lượng tư thế. Kết quả thử nghiệm với mô hình Det và Pose mới được thể hiện trên Hình 4.2. Để thuận tiện cho việc so sánh xu hướng độ trễ trong quá trình tối ưu, hình này cũng trình bày thêm hai cấu hình đạt độ trễ ổn định hơn, sẽ được đề cập ở Mục 4.3.6.



Hình 4.2: So sánh độ trễ theo thời gian giữa cấu hình thay Det/Pose và hai cấu hình đạt độ trễ ổn định

Sau khi thay hai mô hình này, độ trễ ban đầu của pipeline giảm đáng kể so với cấu hình ban đầu. Ví dụ, kết quả đầu tiên có độ trễ khoảng 1,4 giây, kết quả thứ hai có độ trễ khoảng 0,8 giây. Tuy nhiên, khi tiếp tục truyền video vào pipeline,

độ trễ không giữ ổn định mà tăng dần theo thời gian. Cũng có thể thấy kết quả đầu tiên có độ trễ cao hơn kết quả thứ hai do ở thời điểm khởi đầu, các buffer của pipeline còn rỗng và bộ tạo cửa sổ phải chờ tích lũy đủ số frame cần thiết trước khi tạo được cửa sổ đầu tiên.

Bên cạnh mục tiêu giảm độ trễ, việc thay thế các mô hình Det và Pose cũng cần được đánh giá về ảnh hưởng đến chất lượng nhận dạng. Lý do là mô hình ISLR trong pipeline nhận dữ liệu đầu vào gồm cả frame RGB và keypoint. Do đó, nếu mô hình phát hiện người hoặc mô hình ước lượng tư thế thay đổi, chất lượng hộp giới hạn và keypoint đầu vào cũng có thể thay đổi, từ đó ảnh hưởng đến kết quả dự đoán gloss.

Nhằm so sánh kết quả trong điều kiện gần với thiết lập của Zuo và cộng sự, khóa luận sử dụng WER (*Word Error Rate*) làm độ đo chính để đánh giá chất lượng nhận dạng. WER phản ánh mức độ sai khác giữa chuỗi gloss dự đoán và chuỗi gloss đúng; giá trị WER càng thấp thì kết quả nhận dạng càng chính xác. Để bảo đảm tính tương đồng với thiết lập suy luận trong công trình gốc, khóa luận sử dụng cùng các tham số chính của cơ chế cửa sổ trượt và hậu xử lý, bao gồm kích cỡ cửa sổ bằng 16, bước trượt bằng 1 và kích cỡ voting bag bằng 7 [14].

Trong quá trình đánh giá, Zuo và cộng sự cũng áp dụng cơ chế cửa sổ trượt trên luồng video đầu vào để đưa một số phương pháp vốn được thiết kế cho xử lý ngoại tuyến, gồm FCN [90] và TwoStream-SLR [57], sang bối cảnh nhận dạng trực tuyến [14]. Khóa luận trích lại các kết quả này trong Bảng 4.1 như các mốc tham khảo, nhằm đặt kết quả của cấu hình thử nghiệm vào cùng bối cảnh đánh giá.

Bảng 4.1: So sánh WER giữa cấu hình gốc, cấu hình thay mô hình Det/Pose và một số phương pháp offline được áp dụng cơ chế cửa sổ trượt

Cấu hình	WER Dev ↓	WER Test ↓
Cấu hình gốc của Zuo và cộng sự [14]	22,2	22,1
Thay mô hình Det và Pose nhanh hơn	24,3	25,1
FCN [90]	38,8	39,1
TwoStream-SLR [57]	38,4	39,3

Bảng 4.1 trình bày kết quả WER trên tập PHOENIX-2014T khi thay thế các mô hình Det và Pose ban đầu bằng YOLOv3 và ResNet50. Kết quả cho thấy khi thay

thế Det và Pose bằng các mô hình nhanh hơn, WER tăng từ 22,2 lên 24,3 trên tập dev và từ 22,1 lên 25,1 trên tập test của PHOENIX-2014T. Điều này cho thấy việc thay thế Det/Pose có làm suy giảm chất lượng nhận dạng, nhiều khả năng do đánh đổi một phần chất lượng keypoint để đạt được tốc độ xử lý nhanh hơn.

Tuy nhiên, kết quả sau khi thay Det/Pose vẫn tốt hơn đáng kể so với các baseline được đánh giá trong bối cảnh trực tuyến bằng cơ chế cửa sổ trượt với cùng kích thước cửa sổ. Cụ thể, cấu hình thay Det/Pose đạt WER 24,3/25,1 trên tập dev/test, trong khi FCN đạt 38,8/39,1 và TwoStream-SLR đạt 38,4/39,3. Điều này cho thấy mặc dù chất lượng nhận dạng có giảm so với cấu hình gốc, phương pháp vẫn duy trì được hiệu quả nhận dạng tương đối tốt so với các baseline trong bảng.

Từ các kết quả trên, có thể thấy mặc dù làm WER tăng khoảng 2–3 điểm so với phương pháp gốc, cấu hình mới vẫn duy trì chất lượng nhận dạng tương đối tốt trong bối cảnh tối ưu độ trễ của hệ thống. Do đó, kể từ thời điểm này, cấu hình sử dụng YOLOv3 và ResNet50 được xem là baseline mới cho các thử nghiệm tối ưu độ trễ còn lại trong chương.

4.3.2 Xử lý hiện tượng độ trễ tích lũy

Kết quả của mục 4.3.1 cho thấy sau khi thay thế Det/Pose, vấn đề chính của pipeline không còn là độ trễ tuyệt đối quá cao ngay từ đầu, mà chuyển sang hiện tượng độ trễ tích lũy đã được đề cập ở Mục 3.10.2. Cụ thể, độ trễ ban đầu đã giảm xuống mức thấp hơn, nhưng throughput trung bình của pipeline vẫn chưa đủ để bắt kịp tốc độ sinh frame của video đầu vào.

Do đây là vấn đề về throughput, việc tối ưu pipeline được tiếp cận theo hai hướng chính. Hướng thứ nhất là tăng throughput xử lý của pipeline, tức là làm cho các bước trong pipeline xử lý nhanh hơn hoặc tận dụng GPU tốt hơn. Các biện pháp thuộc hướng này gồm điều chỉnh kích cỡ batch, sử dụng CUDA stream và thử nghiệm trên GPU mạnh hơn. Hướng thứ hai là giảm lượng dữ liệu cần xử lý, tức là giảm số lần pipeline phải thực hiện suy luận trong cùng một khoảng thời gian. Trong khóa luận, hướng này được thử nghiệm thông qua việc tăng stride của cơ chế cửa sổ trượt, từ đó giảm số cửa sổ cần đưa qua mô hình ISLR.

4.3.3 Tăng throughput bằng kích cỡ batch và CUDA stream

Một cách trực tiếp để tăng throughput của pipeline là điều chỉnh kích cỡ batch cho các mô hình Det, Pose và ISLR để tận dụng GPU tốt hơn. Khóa luận thử nghiệm một số tổ hợp kích cỡ batch, đồng thời theo dõi độ trễ, mức sử dụng tài nguyên tính toán và bộ nhớ GPU để tìm ra cấu hình cho hiệu năng tốt hơn.

Bên cạnh đó, khóa luận cũng thử nghiệm thiết kế sử dụng ba CUDA stream và các cơ chế truyền dữ liệu bất đồng bộ giữa host và device để cung cấp thêm khả năng chồng lán giữa sao chép H2D, tính toán mô hình và sao chép D2H. Mục tiêu của hướng này là giảm thời gian chờ giữa các thao tác và cải thiện throughput tổng thể của pipeline.

4.3.4 Tăng năng lực xử lý bằng GPU mạnh hơn

Một hướng tiếp cận khác là tăng năng lực xử lý phần cứng. Khóa luận thử nghiệm pipeline trên RTX 4090 nhằm kiểm tra xem hiện tượng độ trễ tích lũy có còn xảy ra khi GPU có năng lực xử lý cao hơn hay không.

Hướng thử nghiệm này không thay đổi bản chất thuật toán của pipeline, nhưng giúp đánh giá liệu nút nghẽn chính có đến từ giới hạn throughput của phần cứng hay không. Nếu cùng baseline mới và cùng bước trượt bằng một nhưng GPU mạnh hơn có thể làm độ trễ ổn định, điều đó cho thấy pipeline có khả năng vận hành gần thời gian thực khi năng lực xử lý trung bình đủ lớn so với tốc độ sinh frame đầu vào.

4.3.5 Giảm số cửa sổ cần xử lý bằng cách tăng stride

Ngoài việc tăng throughput, pipeline cũng có thể được tối ưu bằng cách giảm số lượng cửa sổ cần xử lý. Với bước trượt bằng 1, cơ chế cửa sổ trượt tạo ra các cửa sổ rất dày, giúp giảm nguy cơ bỏ sót ký hiệu nhưng làm tăng số lần gọi mô hình ISLR. Điều này tạo áp lực lớn lên bước nhận dạng và hậu xử lý.

Vì vậy, khóa luận thử nghiệm tăng bước trượt lên 5. Khi bước trượt tăng, số cửa sổ được tạo ra trong cùng một khoảng thời gian giảm xuống, từ đó giảm tải cho ISLR và hậu xử lý. Tuy nhiên, hướng tối ưu này có rủi ro làm giảm mật độ quan sát theo thời gian, có thể dẫn đến bỏ sót một số ký hiệu ngắn hoặc diễn ra nhanh.

Do đó, kết quả với bước trượt bằng 5 cần được diễn giải thận trọng và cần đánh giá thêm trên nhiều video hơn.

4.3.6 Hai cấu hình tối ưu đạt độ trễ ổn định

Trong quá trình thử nghiệm các hướng tiếp cận trên, khóa luận tìm ra được nhiều cấu hình kết hợp có khả năng làm độ trễ ổn định trong phạm vi thử nghiệm, trong đó có hai cấu hình được ghi nhận trong Bảng 4.2. Hai cấu hình này được đối chiếu với cấu hình baseline mới trên Hình 4.2.

Bảng 4.2: Hai cấu hình có độ trễ ổn định

Tiêu chí	Baseline mới	Cấu hình 1	Cấu hình 2
GPU	RTX 3090	RTX 4090	RTX 3090
Det/Pose	YOLOv3/ResNet50	YOLOv3/ResNet50	YOLOv3/ResNet50
Stride	1	1	5
Các tối ưu chính	Thay Det/Pose gốc bằng mô hình nhẹ hơn	Điều chỉnh kích cỡ batch của Det và ISLR bằng 4 và 10, dùng thêm CUDA stream, GPU mạnh hơn	Giữ nguyên kích cỡ batch như cấu hình đầu, giảm số cửa sổ bằng việc tăng stride lên 5
Kết quả độ trễ	Độ trễ ban đầu thấp hơn nhưng tăng dần theo thời gian	Ổn định, khoảng 0,7 giây	Ổn định, khoảng 2,1 giây
Phạm vi thử nghiệm	Một video	Phát lặp một video trong 10 phút	Phát lặp một video trong 10 phút
Nhận xét	Là baseline cho các thử nghiệm tối ưu còn lại	Cho thấy tăng throughput có thể khắc phục độ trễ tích lũy	Cho thấy giảm mật độ cửa sổ có tiềm năng giảm tải pipeline

Nhìn chung, các kết quả trên cho thấy có ít nhất hai hướng khả thi để giải quyết độ trễ tích lũy: tăng throughput xử lý hoặc giảm số lượng cửa sổ cần xử lý. Tuy nhiên, do phạm vi thử nghiệm còn hạn chế, các kết quả này chỉ chứng minh tính

khả thi ban đầu của pipeline gần thời gian thực, chưa thay thế cho đánh giá định lượng đầy đủ trên toàn bộ tập dữ liệu.

4.4 Cài đặt phiên bản Server suy luận tự xây dựng

Sau khi pipeline PyTorch đã được tối ưu bước đầu, khóa luận thực hiện cài đặt phiên bản Server suy luận tự xây dựng theo thiết kế ở Mục 3.5.1. Server này sử dụng FastAPI với WebSocket để cung cấp endpoint nhận luồng video và trả về kết quả gloss. Đồng thời, PyTorch, MMDetection, MMPose và mã nguồn mô hình nhận dạng của tác giả được sử dụng để nạp và thực thi các mô hình Det, Pose và ISLR tương ứng. Khi server khởi động, các mô hình này được nạp vào GPU. Sau đó, mỗi khi có một phiên WebSocket mới, server tạo mới một pipeline instance để xử lý luồng video đầu vào.

Phiên bản Server suy luận này giúp xác nhận rằng kiến trúc tách Bot server và Server suy luận là khả thi. Tuy nhiên, do phiên bản này còn nhiều hạn chế như đã trình bày ở Mục 3.5.2, khóa luận thực hiện bước tiếp theo là triển khai Server suy luận bằng Triton theo phương án phục vụ từng mô hình riêng lẻ.

4.5 Triển khai Triton theo phương án phục vụ từng mô hình riêng lẻ

4.5.1 Đóng gói các mô hình

Trước khi triển khai các mô hình trên Triton, khóa luận thực hiện đóng gói các mô hình. Với các mô hình Det (YOLOv3) và Pose (ResNet50), việc này được thực hiện thông qua việc sử dụng MMDeploy. Công cụ này hỗ trợ đưa các mô hình Det và Pose về định dạng TensorRT để có hiệu suất tối ưu hơn trên môi trường NVIDIA GPU. Ngoài ra, MMDeploy SDK còn được đóng gói chung với các mô hình nhằm sử dụng lại một số thao tác xử lý mô hình ở mức thấp của thư viện này.

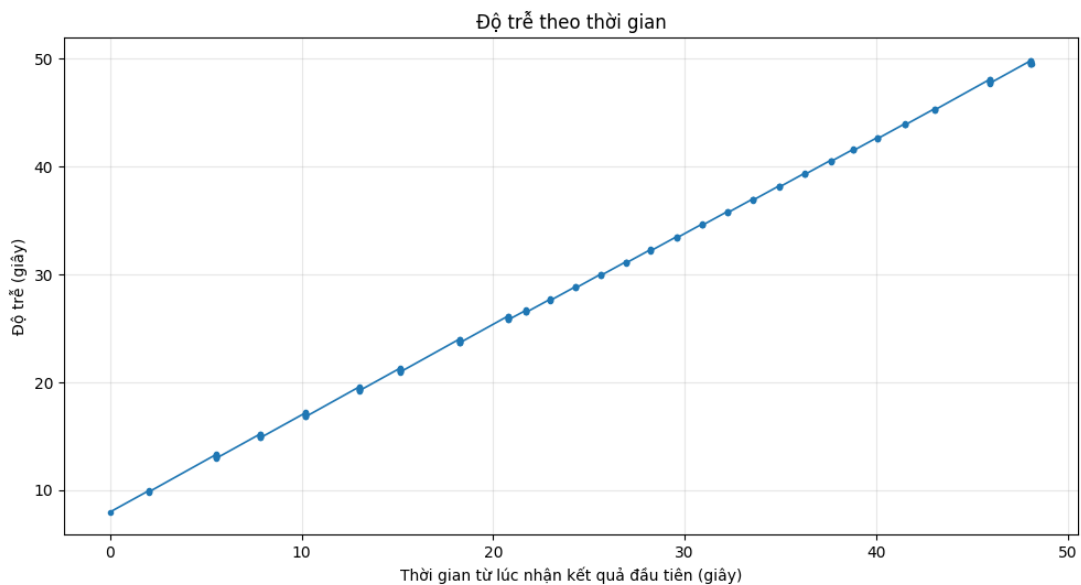
Khác với Det và Pose, mô hình ISLR không được chuyển sang dạng TensorRT trong phạm vi khóa luận. Nguyên nhân là mô hình này phụ thuộc vào mã nguồn PyTorch của nhóm tác giả và có kiến trúc phức tạp hơn, trong khi trọng tâm của khóa luận không nằm ở việc tối ưu sâu mô hình ISLR. Vì vậy, mô hình ISLR được đóng gói bằng Triton Python backend, sử dụng trực tiếp mã nguồn PyTorch đã

được kế thừa từ công trình gốc. Cách tiếp cận này giúp đưa ISLR vào cùng môi trường Triton với Det và Pose, đồng thời giữ nguyên logic suy luận của mô hình đã huấn luyện sẵn.

4.5.2 Vấn đề độ trễ giao tiếp khi triển khai từ xa

Như được phân tích ở Mục 3.6.1, hạn chế chính của phương án triển khai này là số lượng request giữa Bot server và Triton. Một frame hoặc cửa sổ có thể dẫn đến nhiều request đến các mô hình khác nhau. Khi Bot server và Triton được triển khai xa nhau về mặt mạng, chi phí giao tiếp có thể lấn át thời gian xử lý mô hình. Hiện tượng này được quan sát rõ trong quá trình khóa luận thử nghiệm triển khai Triton trên một số máy GPU thuê từ VastAI.

Khóa luận thực hiện một thử nghiệm đo độ trễ của toàn bộ pipeline từ khi nhận frame đến khi ra kết quả. Ở đây, độ trễ vẫn được đo theo phương pháp ở Mục 4.2.1. Cụ thể, độ trễ của một cửa sổ được tính bằng hiệu giữa thời điểm Bot server nhận được kết quả cho cửa sổ đó và thời điểm frame đầu tiên của cửa sổ được nhận bởi Bot server. Trong thử nghiệm này, kích cỡ batch của các mô hình Det, Pose và ISLR lần lượt là 4, 4 và 10. Cả ba mô hình trải qua 20 bước khởi động trước khi đo. Kích cỡ cửa sổ, bước trượt và kích cỡ voting bag lần lượt là 16, 1 và 7. Luồng video đầu vào có frame rate khoảng 25 FPS.



Hình 4.3: Độ trễ theo thời gian khi triển khai Bot server và Triton xa nhau

Các phép đo được thực hiện từ máy tính cá nhân của tác giả đến một máy có một GPU RTX 4090 thuê đặt tại một trung tâm dữ liệu ở Nhật Bản. Theo thông tin do VastAI cung cấp, máy thuê có “Internet upload speed (shared)” khoảng 3200 Mbps và “Internet download speed (shared)” khoảng 3800 Mbps. Tuy nhiên, tốc độ mạng quảng cáo cao không đồng nghĩa với độ trễ request-response thấp và ổn định, đặc biệt trong trường hợp nhiều request nhỏ được gửi liên tục.

Hình 4.3 thể hiện độ trễ đo được trong một phiên nhận dạng. Ngay kết quả đầu tiên, độ trễ đã đạt gần 8 giây, sau đó tiếp tục tăng theo thời gian. Điều này cho thấy phương án triển khai từng mô hình riêng lẻ không phù hợp khi Bot server và Triton Server được triển khai xa nhau.

Để phân tích nguyên nhân gây độ trễ, khóa luận sử dụng Triton Performance Analyzer [67]. Công cụ này đóng vai trò một client tạo tải suy luận đến mô hình đang được Triton phục vụ, sau đó ghi nhận các chỉ số như throughput, độ trễ phía client và độ trễ xử lý phía server.

Các phép đo được thực hiện ở chế độ time windows. Ở chế độ này, Performance Analyzer chia quá trình đo thành các cửa sổ thời gian cố định. Với mỗi cửa sổ đo, công cụ thống kê các request đã hoàn thành trong khoảng thời gian đó và tính các chỉ số throughput, độ trễ trung bình và các phân vị độ trễ [67]. Trong thí nghiệm của khóa luận, độ dài mỗi cửa sổ đo được đặt là 120 giây. Việc sử dụng cửa sổ đo tương đối dài giúp giảm ảnh hưởng của các dao động ngắn hạn, đặc biệt trong bối cảnh Triton được triển khai trên máy GPU thuê từ xa và độ trễ mạng có thể thay đổi giữa các request.

Cách đo này phù hợp với mục tiêu thử nghiệm của khóa luận vì hệ thống nhận dạng được thiết kế để xử lý luồng video liên tục. Do đó, hiệu năng cần được đánh giá trên một khoảng thời gian vận hành thay vì chỉ dựa trên một vài request riêng lẻ. Ngoài ra, Performance Analyzer còn thực hiện nhiều lần đo và chỉ kết thúc khi các chỉ số throughput và độ trễ đạt mức ổn định, hoặc khi đạt đến số lần thử tối đa. Kết quả được xem là ổn định nếu trong 3 lần đo gần nhất, tỉ lệ hai giá trị lớn nhất và nhỏ nhất của cả throughput và độ trễ nằm trong một ngưỡng phần trăm [91]. Khóa luận sử dụng ngưỡng ổn định là 10% và số lần đo tối đa là 10. Cách đo này giúp hạn chế việc kết quả cuối cùng bị chi phối bởi các dao động ngẫu nhiên trong một cửa sổ đo đơn lẻ.

Để phân tích phần độ trễ phát sinh ngoài xử lý mô hình trong Triton, khóa luận sử dụng hai chỉ số chính từ Triton Performance Analyzer: Server Avg Request Latency và gRPC Response Wait. Server Avg Request Latency biểu diễn thời gian trung bình mà một request được xử lý bên trong Triton server, tính từ khi server nhận request đến khi server gửi response. Trong khi đó, gRPC Response Wait biểu diễn thời gian phía client dành cho việc ghi request gRPC ra mạng, chờ phản hồi từ server và đọc response từ mạng.

Do đó, phần độ trễ giao tiếp client-server ở tầng ứng dụng được ước lượng bằng công thức:

$$T_{server} = \text{Server Avg Request Latency}$$

$$T_{comm} \approx \text{gRPC Response Wait} - \text{Server Avg Request Latency}$$

Trong đó, T_{server} là thời gian xử lý trung bình của server cho một request, còn T_{comm} là độ trễ giao tiếp client-server ước lượng. Giá trị T_{comm} không phải RTT thuần ở tầng mạng, nhưng phản ánh chi phí request-response quan sát được khi gọi Triton qua gRPC.

Kết quả đo được trình bày trong Bảng 4.3 cho thấy trong phương án Bot server gọi từng mô hình riêng lẻ, chi phí giao tiếp có thể lấn át thời gian xử lý mô hình khi Bot server và Triton Server được triển khai xa nhau. Do đó, tối ưu mô hình đơn lẻ là chưa đủ; kiến trúc triển khai, số lượng request giữa các thành phần và vị trí đặt server có ảnh hưởng trực tiếp đến khả năng vận hành gần thời gian thực của hệ thống.

Bảng 4.3: Kết quả đo thời gian xử lý server và độ trễ giao tiếp ước lượng khi gọi Triton từ xa

Mô hình	Thời gian xử lý trung bình của server T_{server}	Độ trễ giao tiếp client-server ước lượng T_{comm}
Det	129,6 ms	0,579 s
Pose	5,4 ms	1,435 s
ISLR	177,3 ms	1,835 s

Từ kết quả thực nghiệm này, khóa luận đưa ra phương án Triton phục vụ toàn bộ pipeline ở Mục 3.6.2 làm hướng tối ưu tiềm năng. Tuy nhiên, do độ phức tạp triển

khai cao, phương án này chưa được hoàn thiện đầy đủ và được xem là hướng phát triển tiếp theo của hệ thống.

4.5.3 Đánh giá độ trễ pipeline khi giảm thiểu chi phí client–server

Khóa luận thực hiện thêm một thử nghiệm trong đó một client thử nghiệm được đóng gói và triển khai trên cùng máy với Triton. Client thử nghiệm có nhiệm vụ nạp toàn bộ các video trong tập test của PHOENIX-2014T, nối lại thành một video tổng hợp, rồi sử dụng video tổng hợp để mô phỏng luồng đầu vào cho pipeline. Client sử dụng lại chức năng điều phối quá trình suy luận đã được cài đặt trong Bot Server để thực hiện nhận dạng trên luồng video mô phỏng này. Mục tiêu của thử nghiệm là đánh giá độ trễ của pipeline suy luận khi giảm thiểu chi phí giao tiếp client–server, từ đó tách biệt rõ hơn ảnh hưởng của bản thân pipeline nhận dạng so với ảnh hưởng của độ trễ mạng trong phương án triển khai từ xa.

Thử nghiệm này vẫn sử dụng phương pháp đo độ trễ, các thông số cấu hình và máy có GPU RTX 4090 như trong thử nghiệm ở Mục 4.5.2. Điểm khác biệt chính là trong thử nghiệm này, client thử nghiệm được triển khai trên cùng máy với Triton nhằm giảm thiểu chi phí giao tiếp client–server. Tổng số frame trong tập test của PHOENIX-2014T là 64627 [16] và được truyền ở frame rate 25 FPS.

Bên cạnh độ trễ của từng kết quả nhận dạng, khóa luận cũng sử dụng thêm hệ số thời gian thực (*Real-Time Factor* – RTF) để đánh giá khả năng xử lý của hệ thống so với tốc độ phát video thực tế. RTF được tính bằng tỉ lệ giữa tổng thời gian hệ thống cần để xử lý dữ liệu đầu vào và tổng thời lượng thực tế của các video được xử lý:

$$RTF = \frac{T_{\text{runtime}}}{T_{\text{video}}}$$

Trong đó, T_{runtime} là tổng thời gian xử lý toàn bộ video tổng hợp của hệ thống, được tính từ thời điểm gửi frame đầu tiên đến thời điểm nhận kết quả cuối cùng, còn T_{video} là tổng thời lượng của video tổng hợp. Với dữ liệu video có frame rate FPS và tổng số frame là N_{frame} , tổng thời lượng video được tính như sau:

$$T_{\text{video}} = \frac{N_{\text{frame}}}{FPS}$$

Ý nghĩa của RTF là so sánh tốc độ xử lý của hệ thống với thời gian thực. Nếu $RTF < 1$, hệ thống xử lý nhanh hơn tốc độ phát video thực tế. Nếu $RTF = 1$, hệ

thống xử lý xấp xỉ thời gian thực. Ngược lại, nếu $RTF > 1$, hệ thống xử lý chậm hơn tốc độ phát video, dẫn đến khả năng tích lũy độ trễ khi chạy trên luồng video liên tục. Do đó, RTF là một độ đo phù hợp để đánh giá liệu pipeline nhận dạng có thể theo kịp tốc độ sinh frame của video đầu vào hay không.

Bảng 4.4: Kết quả đo tốc độ xử lý và độ trễ của pipeline

(a) Các chỉ số liên quan đến tốc độ xử lý		(b) Độ trễ suy luận	
Chỉ số	Giá trị	Chỉ số độ trễ	Giá trị (giây)
Số cửa sổ được xử lý	64 605	Trung bình	1,36
Tổng thời lượng video	2585,08 giây	Độ lệch chuẩn	0,16
Tổng thời gian xử lý	2585,38 giây	Phân vị P50	1,34
RTF	1,0001	Phân vị P90	1,59
Throughput theo frame	25,00 frame/s	Phân vị P95	1,64
Throughput theo cửa sổ	24,99 cửa sổ/s	Phân vị P99	1,72
		Tối đa	2,44

Kết quả thử nghiệm trong Bảng 4.4 cho thấy hệ thống có khả năng xử lý gần thời gian thực trên tập đánh giá. Với tổng thời lượng video là 2585,08 giây, thời gian xử lý thực tế của hệ thống là 2585,38 giây, tương ứng với $RTF \approx 1$. Throughput đạt 24,99 cửa sổ/s và 25 frame/s, tương đương frame rate đầu vào 25 FPS. Điều này cho thấy pipeline bắt kịp tốc độ sinh frame của video đầu vào.

Về độ trễ đầu cuối, hệ thống đạt độ trễ trung bình 1,36 giây và trung vị 1,34 giây. Các phân vị cao P95 và P99 lần lượt là 1,64 giây và 1,72 giây, cho thấy phần lớn kết quả được trả về trong khoảng dưới 1,7 giây. Mặc dù có một số trường hợp độ trễ cao hơn, với giá trị lớn nhất là 2,44 giây, các trường hợp này chỉ xuất hiện ở phần đuôi của phân phối độ trễ.

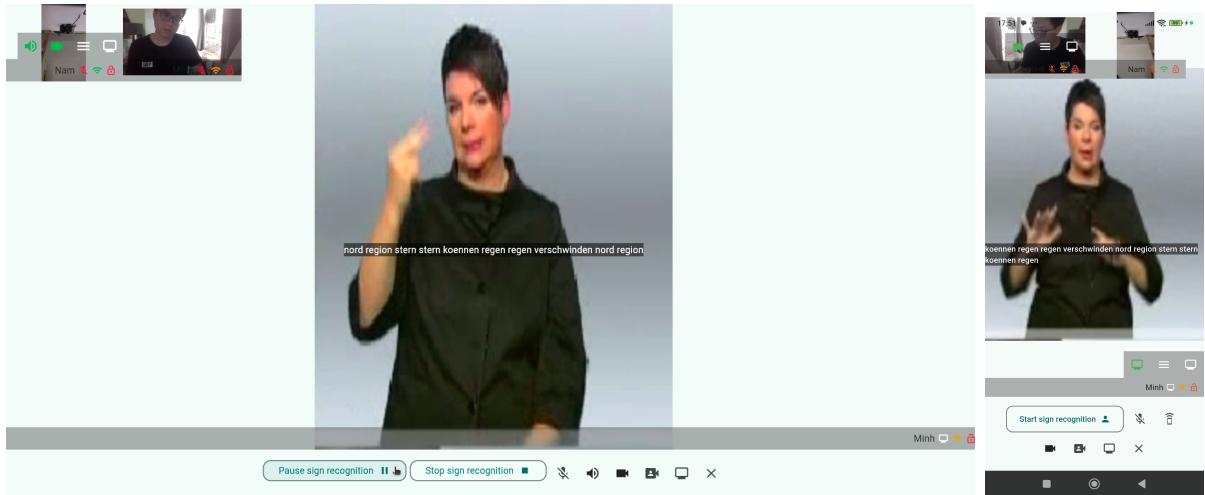
Nhìn chung, kết quả thử nghiệm cho thấy khi giảm thiểu chi phí giao tiếp client-server, pipeline có thể xử lý luồng video đầu vào với tốc độ xấp xỉ thời gian thực trên RTX 4090. Tuy nhiên, RTF chỉ vừa đạt ngưỡng 1, cho thấy hệ thống chưa có nhiều dư địa xử lý nếu có thêm chi phí mạng, nhiều luồng đồng thời hoặc các bước xử lý bổ sung. Do đó, kết quả này khẳng định tính khả thi của pipeline trong điều kiện triển khai cục bộ, đồng thời cho thấy việc giảm độ trễ của các bước suy luận và chi phí truyền thông vẫn là hướng tối ưu quan trọng khi triển khai hệ thống

trong môi trường gọi video thực tế. Tuy nhiên, do thử nghiệm sử dụng client thử nghiệm chạy cùng máy với Triton, kết quả này chưa bao gồm đầy đủ độ trễ của toàn bộ hệ thống gọi video thực tế như thu nhận video từ frontend, truyền qua LiveKit và hiển thị phụ đề trên thiết bị người dùng.

4.6 Cài đặt ứng dụng frontend và Bot server

Hệ thống sử dụng lại LiveKit server làm Server truyền thông thời gian thực, do đó ứng dụng frontend sử dụng LiveKit Client SDK còn Bot server sử dụng LiveKit Server SDK để giao tiếp với LiveKit server. Hai SDK cung cấp các tính năng gọi video thông qua LiveKit Server bao gồm tham gia phòng, chia sẻ luồng video với phòng gọi và truyền nhận luồng video của người tham gia. Ngoài ra, cả LiveKit Server SDK và Client SDK đều hỗ trợ cơ chế cung cấp và gọi hàm RPC được dùng bởi ứng dụng frontend và bot trong việc điều khiển phiên nhận dạng. Server SDK cũng cung cấp cơ chế viết luồng dữ liệu vào text stream, còn Client SDK cung cấp cơ chế đọc luồng dữ liệu trong text stream. Qua đó, bot có thể liên tục viết kết quả gloss vào text stream, và ứng dụng frontend có thể liên tục nhận kết quả này để hiển thị phụ đề.

Việc cài đặt ứng dụng frontend bao gồm việc cài đặt các màn hình và thành phần UI, các ViewModel và các Repository theo kiến trúc ở Mục 3.9. Màn hình phòng gọi được cài đặt sao cho dễ sử dụng và hiển thị rõ được video của người dùng, dòng chữ phụ đề và các nút trên bảng điều khiển, như trên Hình 4.4. Một số thành phần UI về gọi video trong LiveKit Client SDK được sử dụng lại, bao gồm khung hiển thị video của người tham gia và các nút điều khiển camera. Ngoài ra, khóa luận cài đặt các ViewModel để thực hiện các logic như hiển thị các nút điều khiển phiên nhận dạng theo trạng thái hiện tại của phiên, làm nổi bật video của người đang được nhận dạng, và quản lý trạng thái hiển thị phụ đề. Còn lại các xử lý gọi video cơ bản như hiển thị video của người dùng và quản lý trạng thái phòng gọi đã được xử lý bởi LiveKit Client SDK và được sử dụng lại trong một số ViewModel. Cuối cùng, khóa luận cài đặt việc điều khiển và lấy dữ liệu về phiên nhận dạng trong một Repository, và tích hợp LiveKit Client SDK vào một Repository để quản lý chung các dữ liệu phòng gọi.



Hình 4.4: Màn hình phòng gọi video trong một phiên nhận dạng ở nền tảng Web và Android

Bot server được cài đặt để thực hiện tất cả các trách nhiệm về quản lý vòng đời của bot và điều phối quá trình nhận dạng như đã trình bày trong Chương 3. Khóa luận sử dụng FastAPI để cài đặt endpoint webhook nhận sự kiện từ LiveKit. Sau đó, khóa luận cài đặt các xử lý sự kiện và quản lý vòng đời bot bao gồm kiểm tra sự tồn tại của bot trong phòng, khởi tạo bot, kiểm tra điều kiện rời phòng của bot và xóa bot. Bên cạnh đó, khóa luận cũng cài đặt các hàm RPC được đề cập trong Mục 3.7 cùng với các xử lý về quản lý trạng thái phiên nhận dạng. Ngoài ra, khóa luận còn cài đặt một bộ điều phối nhận dạng theo các phương án thiết kế ở Mục 3.3.3. Giao diện của bộ điều phối bao gồm một hàm nhận tham số là luồng video đầu vào và trả về luồng kết quả gloss. Khóa luận cài đặt hai phiên bản của bộ điều phối, ứng với hai phiên bản của Server suy luận là phiên bản tự xây dựng và phiên bản Triton phục vụ các mô hình riêng lẻ. Ở phiên bản đầu, bộ điều phối chỉ cần tạo phiên WebSocket với Server suy luận, thực hiện tiền xử lý nhẹ luồng video đầu vào và gửi tới Server suy luận qua phiên WebSocket đó để nhận về kết quả nhận dạng. Ở phiên bản sau, bộ điều phối cần tạo ra các worker để thực hiện một số tiền xử lý và sử dụng Triton Client SDK để gọi tới các mô hình Det, Pose và ISLR trên Server suy luận. Đồng thời, bộ điều phối cũng cần tạo các buffer để truyền kết quả trung gian giữa các worker.

Kết luận

Khóa luận đã trình bày quá trình xây dựng một nguyên mẫu hệ thống nhận dạng ngôn ngữ ký hiệu liên tục theo hướng trực tuyến trong môi trường gọi video. Hệ thống kế thừa phương pháp CSLR trực tuyến của Zuo và cộng sự, trong đó mô hình ISLR được áp dụng trên các cửa sổ thời gian của luồng video đầu vào, sau đó kết quả được hậu xử lý để tạo chuỗi gloss đầu ra. Trên cơ sở đó, khóa luận tổ chức lại pipeline nhận dạng để có thể xử lý trực tiếp luồng video, đồng thời tích hợp pipeline này với một hệ thống gọi video gồm ứng dụng frontend, Server truyền thông thời gian thực, Bot server và Server suy luận.

Về mặt cài đặt, khóa luận đã hiện thực được các thành phần chính của hệ thống, bao gồm ứng dụng frontend gọi video, Bot server điều phối phiên nhận dạng và Server suy luận phục vụ các mô hình Det, Pose và ISLR. Trong quá trình phát triển, khóa luận đã thử nghiệm phiên bản Server suy luận tự xây dựng, phân tích các hạn chế của thiết kế này, sau đó chuyển sang triển khai các mô hình trên Triton Inference Server nhằm tận dụng các cơ chế phục vụ mô hình chuyên dụng hơn. Bên cạnh đó, khóa luận cũng thực hiện các thử nghiệm tối ưu độ trễ, bao gồm thay thế mô hình Det/Pose bằng các mô hình nhanh hơn, điều chỉnh batch size, sử dụng CUDA stream và đánh giá ảnh hưởng của môi trường triển khai đến độ trễ.

Kết quả thực nghiệm cho thấy việc thay thế Faster R-CNN và HRNet bằng YOLOv3 và ResNet50 giúp giảm đáng kể độ trễ của các bước đầu pipeline, đổi lại WER tăng ở mức vừa phải so với cấu hình gốc. Khi giảm thiểu chi phí giao tiếp client-server bằng cách triển khai client thử nghiệm cùng máy với Triton, pipeline đạt tốc độ xử lý xấp xỉ thời gian thực trên RTX 4090, với RTF xấp xỉ 1 và độ trễ trung bình khoảng 1,36 giây. Các kết quả này cho thấy pipeline có tính khả thi trong điều kiện thử nghiệm cục bộ, đồng thời khẳng định rằng kiến trúc triển khai và chi phí giao tiếp giữa các thành phần có ảnh hưởng lớn đến khả năng vận hành gần thời gian thực.

Tuy nhiên, hệ thống hiện vẫn còn một số giới hạn. Thứ nhất, kết quả nhận dạng phụ thuộc vào mô hình đã huấn luyện trên PHOENIX-2014T, do đó khả năng tổng quát hóa sang video ngoài miền dữ liệu, người ký hiệu khác hoặc ngôn ngữ ký hiệu

khác còn hạn chế. Thứ hai, hệ thống mới trả về chuỗi gloss, chưa thực hiện dịch sang câu ngôn ngữ tự nhiên. Thứ ba, phương án Triton phục vụ từng mô hình riêng lẻ vẫn phát sinh nhiều request giữa Bot server và Server suy luận, nên chưa tối ưu trong môi trường triển khai từ xa. Ngoài ra, các thử nghiệm hiện tại chủ yếu tập trung vào một luồng đầu vào, chưa đánh giá đầy đủ khả năng phục vụ nhiều phiên nhận dạng đồng thời.

Trong tương lai, hệ thống có thể được phát triển theo một số hướng. Hướng thứ nhất là đưa toàn bộ pipeline vào Triton để giảm số lần giao tiếp giữa Bot server và Server suy luận, qua đó cải thiện độ trễ khi triển khai phân tán. Hướng thứ hai là tiếp tục tối ưu các mô hình và các bước tiền/hậu xử lý, chẳng hạn chuyển đổi thêm các thành phần sang TensorRT hoặc cải thiện cơ chế batching. Hướng thứ ba là mở rộng đánh giá trên nhiều luồng video đồng thời và trong các điều kiện gọi video thực tế hơn. Về mặt mô hình, cần nghiên cứu thêm các phương pháp thích nghi miền, huấn luyện hoặc tinh chỉnh trên dữ liệu đa dạng hơn, cũng như mở rộng từ nhận dạng gloss sang dịch câu ngôn ngữ tự nhiên. Đây là các hướng cần thiết để hệ thống tiến gần hơn đến khả năng hỗ trợ giao tiếp trong môi trường thực tế.

Tài liệu tham khảo

- [1] David F Armstrong, William C Stokoe, and Sherman E Wilcox. *Gesture and the nature of language*. Cambridge University Press, 1995.
- [2] Warren R Goldmann and James R Mallory. Overcoming communication barriers: communicating with deaf people. 1992.
- [3] Ruth Butler, Tracey Skelton, and Gill Valentine. Language barriers: Exploring the worlds of the deaf. *Disability Studies Quarterly*, 21(4), 2001.
- [4] Neena Aloysius and M. Geetha. Understanding vision-based continuous sign language recognition. *Multimedia Tools and Applications*, 79:22177 – 22209, 2020.
- [5] Fangyun Wei and Yutong Chen. Improving continuous sign language recognition with cross-lingual signs. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 23555–23564, 2023.
- [6] Muhammad Al-Qurishi, Thariq Khalid, and R. Souissi. Deep learning for sign language recognition: Current techniques, benchmarks, and open issues. *IEEE Access*, 9:126917–126951, 2021.
- [7] Mathieu De Coster, D. Shterionov, M. Herreweghe, and J. Dambre. Machine translation from signed to spoken languages: state of the art and challenges. *Universal Access in the Information Society*, pages 1–27, 2022.
- [8] Asma Khan, Seyong Jin, Geon hee Lee, Gul E. Arzu, L. Minh Dang, Tan N. Nguyen, Woong Choi, and Hyeonjoon Moon. Deep learning approaches for continuous sign language recognition: A comprehensive review. *IEEE Access*, 13:55524–55544, 2025.
- [9] M. Sisto, Vincent Vandeghinste, Santiago Egea Gómez, Mathieu De Coster, D. Shterionov, and Horacio Saggion. Challenges with sign language datasets for sign language recognition and translation. pages 2478–2487, 2022.
- [10] M. Amin, Syed Tahir Hussain Rizvi, and Md. Murad Hossain. A comparative review on applications of different sensors for sign language recognition. *Journal of Imaging*, 8, 2022.

- [11] Tangfei Tao, Yizhe Zhao, Tianyu Liu, and Jieli Zhu. Sign language recognition: A comprehensive review of traditional and deep learning approaches, datasets, and challenges. *IEEE Access*, 12:75034–75060, 2024.
- [12] Adrián Núñez-Marcos, Olatz Perez de Viñaspre, and Gorka Labaka. A survey on sign language machine translation. *Expert Syst. Appl.*, 213:118993, 2022.
- [13] Maria Papatsimouli, Panos Sarigiannidis, and G. Fragulis. A survey of advancements in real-time sign language translators: Integration with iot technology. *Technologies*, 2023.
- [14] Ronglai Zuo, Fangyun Wei, and Brian Mak. Towards online continuous sign language recognition and translation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 11050–11067, 2024.
- [15] Sarah N. Alyami, Hamzah Luqman, and M. Hammoudeh. Reviewing 25 years of continuous sign language recognition research: Advances, challenges, and prospects. *Inf. Process. Manag.*, 61:103774, 2024.
- [16] Necati Cihan Camgöz, Simon Hadfield, Oscar Koller, H. Ney, and R. Bowden. Neural sign language translation. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7784–7793, 2018.
- [17] William C Stokoe Jr. Sign language structure: An outline of the visual communication systems of the american deaf. *Journal of deaf studies and deaf education*, 10(1):3–37, 2005.
- [18] Danielle Bragg, Oscar Koller, Mary Bellard, Larwan Berke, Patrick Boudreault, Annelies Braffort, Naomi K. Caselli, Matt Huenerfauth, Hernisa Kacorri, T. Verhoef, Christian Vogler, and M. Morris. Sign language recognition, generation, and translation: An interdisciplinary perspective. In *Proceedings of the 21st International ACM SIGACCESS Conference on Computers and Accessibility*, 2019.
- [19] R. Meier. Diminishing diversity of signed languages. *Science*, 288:1965 – 1965, 2000.
- [20] Justin M. Power, G. Grimm, and Johann-Mattis List. Evolutionary dynamics in the dispersal of sign languages. *Royal Society Open Science*, 7, 2020.

- [21] Nguyen Thi Hoa and James Woodward. Education and services for deaf people in viet nam. In Harry Knoors, Maria Brons, and Marc Marschark, editors, *Deaf Education Beyond the Western World: Context, Challenges, and Prospects*. Oxford University Press, 02 2019.
- [22] James Woodward. Sign languages and sign language families in thailand and viet nam. pages 30–52, 2013.
- [23] Sneha Sharma, Ritu Gupta, and Anil Kumar. Continuous sign language recognition using isolated signs data and deep transfer learning. *Journal of Ambient Intelligence and Humanized Computing*, 14:1531–1542, 2021.
- [24] Xiangzu Han, Fei Lu, and G. Tian. Efficient 3d cnns with knowledge transfer for sign language recognition. *Multimedia Tools and Applications*, 81:10071 – 10090, 2022.
- [25] R. Rastgoo, Kouros Kiani, and Sergio Escalera. Word separation in continuous sign language using isolated signs and post-processing. *ArXiv*, abs/2204.00923, 2022.
- [26] Sarah N. Alyami, Hamzah Luqman, Sadam Al-Azani, Maad Alowaiifeer, Y. Alharbi, and Yaser Alonaizan. Isharah: A large-scale multi-scene dataset for continuous sign language recognition. *ArXiv*, abs/2506.03615, 2025.
- [27] Mathias Müller, Zifan Jiang, Amit Moryossef, Annette Rios, and Sarah Ebling. Considerations for meaningful sign language machine translation based on glosses. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 682–693, Toronto, Canada, July 2023. Association for Computational Linguistics.
- [28] Hamid Reza Vaezi Joze and Oscar Koller. Ms-asl: A large-scale data set and benchmark for understanding american sign language. *ArXiv*, abs/1812.01053, 2018.
- [29] Ozge Mercanoglu Sincan and H. Keles. Autsl: A large scale multi-modal turkish sign language dataset and baseline methods. *IEEE Access*, 8:181340–181355, 2020.

- [30] Peng Jin, Hongkai Li, Jun Yang, Yazhou Ren, Yuhao Li, Lilan Zhou, Jin Liu, Mei Zhang, X. Pu, and Siyuan Jing. A large dataset covering the chinese national sign language for dual-view isolated sign language recognition, 2025.
- [31] A. Duarte, Shruti Palaskar, Deepti Ghadiyaram, Kenneth J. DeHaan, Florian Metze, Jordi Torres, and Xavier Giró i Nieto. How2sign: A large-scale multi-modal dataset for continuous american sign language. *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2734–2743, 2020.
- [32] Hao Zhou, Wengang Zhou, Weizhen Qi, Junfu Pu, and Houqiang Li. Improving sign language translation with monolingual data by sign back-translation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 1316–1325, 2021.
- [33] N. C. Camgoz, Ben Saunders, Guillaume Rochette, Marco Giovanelli, Giacomo Inches, Robin Nachtrab-Ribback, and R. Bowden. Content4all open research sign language translation datasets. *2021 16th IEEE International Conference on Automatic Face and Gesture Recognition (FG 2021)*, pages 1–5, 2021.
- [34] Yutong Chen, Fangyun Wei, Xiao Sun, Zhirong Wu, and Stephen Lin. A simple multi-modality transfer learning baseline for sign language translation. *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5110–5120, 2022.
- [35] K. S. Sindhu, Mehnaaz, B. Nikitha, Penumathsa Likhita Varma, and Chandrasekhar Uddagiri. Sign language recognition and translation systems for enhanced communication for the hearing impaired. *2024 1st International Conference on Cognitive, Green and Ubiquitous Computing (IC-CGU)*, pages 1–6, 2024.
- [36] Nguyen Son Dinh, Tuan Dung Nguyen, Duc Tri Tran, Nguyen Dang Huy Pham, Thuan Hieu Tran, Ngoc Anh Tong, Quang Huy Hoang, and Phi Le Nguyen. Sign language recognition: A large-scale multi-view dataset and comprehensive evaluation. In *2025 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 7887–7897. IEEE, 2025.

- [37] K. Dang and I. Bessmertny. One-shot: generating vietnamese sign language data set. *Scientific and Technical Journal of Information Technologies, Mechanics and Optics*, 2024.
- [38] A. Vo, Van-Huy Pham, and Bao T. Nguyen. Deep learning for vietnamese sign language recognition in video sequence. *International Journal of Machine Learning and Computing*, 2019.
- [39] Jens Forster, C. Schmidt, Thomas Hoyoux, Oscar Koller, Uwe Zelle, J. Piater, and H. Ney. Rwth-phoenix-weather: A large vocabulary sign language recognition and translation corpus. pages 3785–3789, 2012.
- [40] Oscar Koller. Quantitative survey of the state of the art in sign language recognition. *ArXiv*, abs/2008.09918, 2020.
- [41] Jens Forster, C. Schmidt, Oscar Koller, M. Bellgardt, and H. Ney. Extensions of the sign language recognition and translation corpus rwth-phoenix-weather. pages 1911–1916, 2014.
- [42] Necati Cihan Camgöz, Simon Hadfield, Oscar Koller, H. Ney, and R. Bowden. RWTH-PHOENIX-Weather 2014 T: Parallel Corpus of Sign Language Video, Gloss and Translation.
- [43] M. Faisal, Angad Singh, and Shailendra Narayan Singh. A review of real-time sign language recognition for virtual interaction on meeting platforms. *2024 14th International Conference on Cloud Computing, Data Science Engineering (Confluence)*, pages 364–369, 2024.
- [44] Sadaf Manzoor, Zakawat Abbas, Gunjan Chhabra, Keshav Kaushik, Mehak Zehra, Zoya Haider, and Inam Ullah Khan. Voice of hearing and speech impaired people. *2024 International Conference on Communication, Computer Sciences and Engineering (IC3SE)*, pages 1891–1897, 2024.
- [45] I. Adeyanju, O. O. Bello, and M. A. Adegboye. Machine learning methods for sign language recognition: A critical review and analysis. *Intell. Syst. Appl.*, 12:200056, 2021.
- [46] Fatma M. Najib. Sign language interpretation using machine learning and artificial intelligence. *Neural Computing and Applications*, 37:841 – 857, 2024.

- [47] Dinushika Chithrani, Adithya Rajapakshe, Dhanuka Jayasinghe, Umayala Bala-
galla, Bhathiya Pilanawithana, Uditha Wijewardhana, and Udaya Wijenayake. Advancements in vision-based sign language recognition: A comprehensive re-
view. *Proceedings of Conference on Transdisciplinary Research in Engineering*, 2024.
- [48] Amina Ben Haj Amor, Oussama El Ghoul, and Mohamed Jemni. Sign lan-
guage recognition using the electromyographic signal: A systematic literature
review. *Sensors (Basel, Switzerland)*, 23, 2023.
- [49] Mosab A. Hassan, Alaa. H. Ali, and A. Sabri. Enhancing communication: Deep
learning for arabic sign language translation. *Open Engineering*, 14, 2024.
- [50] Neil Buckley, Lewis Sherrett, and E. Secco. A cnn sign language recognition
system with single double-handed gestures. *2021 IEEE 45th Annual Comput-
ers, Software, and Applications Conference (COMPSAC)*, pages 1250–1253,
2021.
- [51] Hanan A. Taher and Subhi R. M. Zeebaree. A critical study of recent deep
learning-based continuous sign language recognition. *The Review of Socionet-
work Strategies*, 19:131 – 161, 2025.
- [52] Federal Communications Commission. Consumer Guide: Video relay ser-
vices. <https://www.fcc.gov/consumers/guides/video-relay-services>,
July 2022. Accessed: 2026-05-22.
- [53] N. Sarhan and Simone Frintrop. Unraveling a decade: A comprehensive survey
on isolated sign language recognition. In *2023 IEEE/CVF International Con-
ference on Computer Vision Workshops (ICCVW)*, pages 3202–3211, 2023.
- [54] Sign. Sign Translate: Effortless real-time sign language translation. <https://github.com/sign/translate>, 2026. Accessed: 2026-05-22.
- [55] SignCaption. SignCaption — add captions sign language to any video. <https://signcaption.ai/>, 2026. Accessed: 2026-05-22.
- [56] Google DeepMind. SignGemma is our most advanced model for sign lan-
guage understanding. [https://www.linkedin.com/posts/googledeepmind_
signgemma-is-our-most-advanced-model-for-activity-7342957078249955329-Jw](https://www.linkedin.com/posts/googledeepmind_signgemma-is-our-most-advanced-model-for-activity-7342957078249955329-Jw).
June 2025. LinkedIn post. Accessed: 2026-05-22.

- [57] Yutong Chen, Ronglai Zuo, Fangyun Wei, Yu Wu, Shujie Liu, and Brian Mak. Two-stream network for sign language recognition and translation. *Advances in neural information processing systems*, 35:17043–17056, 2022.
- [58] Fangyun Wei and contributors. gen_pose.py in SLRT. https://github.com/FangyunWei/SLRT/blob/main/Online/CSLR/gen_pose.py, 2024. Accessed: 2026-05-22.
- [59] Fangyun Wei and contributors. SLRT: Sign Language Processing. <https://github.com/FangyunWei/SLRT>, 2024. Accessed: 2026-05-22.
- [60] NVIDIA. NVIDIA Triton Inference Server Documentation. <https://docs.nvidia.com/deeplearning/triton-inference-server/>. Accessed: 2026-05-22.
- [61] NVIDIA. CUDA Programming Guide. <https://docs.nvidia.com/cuda/cuda-programming-guide/index.html>, 2026. Accessed: 2026-05-22.
- [62] NVIDIA. NVIDIA Triton Inference Server Architecture. https://docs.nvidia.com/deeplearning/triton-inference-server/archives/triton_inference_server_230/user-guide/docs/architecture.html. Accessed: 2026-05-22.
- [63] NVIDIA. Inference Protocols and APIs — NVIDIA Triton Inference Server. https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/customization_guide/inference_protocols.html. Accessed: 2026-05-22.
- [64] NVIDIA. Ensemble Models — NVIDIA Triton Inference Server Documentation. https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/ensemble_models.html. Accessed: 2026-05-22.
- [65] NVIDIA. Batchers — NVIDIA Triton Inference Server Documentation. https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/batcher.html. Accessed: 2026-05-22.
- [66] NVIDIA. Implicit State Management — NVIDIA Triton Inference Server. <https://docs.nvidia.com/deeplearning/triton-inference-server/>

- `user-guide/docs/user_guide/implicit_state_management.html`. Accessed: 2026-05-22.
- [67] NVIDIA. Triton Performance Analyzer — NVIDIA Triton Inference Server. https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/perf_analyzer/README.html. Accessed: 2026-05-22.
- [68] LiveKit. Livekit documentation. <https://docs.livekit.io/intro/overview/>. Accessed: 2026-05-22.
- [69] LiveKit. `livekit_client`: Livekit client sdk for flutter. <https://docs.livekit.io/reference/client-sdk-flutter/index.html>, 2026. Dart API documentation. Accessed: 2026-05-22.
- [70] LiveKit. Python SDK for LiveKit. <https://github.com/livekit/python-sdks>, 2026. GitHub repository. Accessed: 2026-05-22.
- [71] LiveKit. Sending text. <https://docs.livekit.io/transport/data/text-streams/>, 2026. LiveKit documentation. Accessed: 2026-05-22.
- [72] LiveKit. RPC — LiveKit Documentation. <https://docs.livekit.io/transport/data/rpc/>. Accessed: 2026-05-22.
- [73] OpenMMLab. MMDetection Documentation. <https://mmdetection.readthedocs.io/>. Accessed: 2026-05-22.
- [74] OpenMMLab. MMPose Documentation. <https://mmpose.readthedocs.io/>. Accessed: 2026-05-22.
- [75] OpenMMLab. Mmdeploy documentation. <https://mmdeploy.readthedocs.io/>. Accessed: 2026-05-22.
- [76] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. In C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 28. Curran Associates, Inc., 2015.
- [77] Ke Sun, Bin Xiao, Dong Liu, and Jingdong Wang. Deep high-resolution representation learning for human pose estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.

- [78] Joseph Redmon and Ali Farhadi. Yolov3: An incremental improvement. *ArXiv*, abs/1804.02767, 2018.
- [79] Bin Xiao, Haiping Wu, and Yichen Wei. Simple baselines for human pose estimation and tracking. In *Proceedings of the European Conference on Computer Vision (ECCV)*, September 2018.
- [80] Flutter. Flutter: Build apps for any screen. <https://flutter.dev/>, 2026. Accessed: 2026-05-22.
- [81] Flutter. Architecting flutter apps. <https://docs.flutter.dev/app-architecture>. Accessed: 2026-05-22.
- [82] MDN Web Docs. WebSocket API (WebSockets). https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API, 2026. Accessed: 2026-06-19.
- [83] FastAPI. Fastapi documentation. <https://fastapi.tiangolo.com/>. Accessed: 2026-05-22.
- [84] PyTorch. PyTorch Documentation. <https://docs.pytorch.org/docs/stable/index.html>, 2026. PyTorch 2.12 documentation. Accessed: 2026-05-22.
- [85] NVIDIA. NVIDIA TensorRT Documentation. <https://docs.nvidia.com/deeplearning/tensorrt/latest/>, 2026. Accessed: 2026-05-22.
- [86] NVIDIA. NVIDIA Ampere GA102 GPU Architecture. <https://www.nvidia.com/content/PDF/nvidia-ampere-ga-102-gpu-architecture-whitepaper-v2.1.pdf>, 2021. White paper. Accessed: 2026-06-19.
- [87] NVIDIA. NVIDIA Ada GPU Architecture. <https://images.nvidia.com/aem-dam/Solutions/geforce/ada/nvidia-ada-gpu-architecture.pdf>, 2022. White paper. Accessed: 2026-06-19.
- [88] NVIDIA. Model Configuration — NVIDIA Triton Inference Server Documentation. https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/model_configuration.html. Accessed: 2026-05-22.

- [89] Fangyun Wei and contributors. `prediction_slide.py` in SLRT. https://github.com/FangyunWei/SLRT/blob/main/Online/CSLR/prediction_slide.py. Accessed: 2026-05-22.
- [90] Ka Leong Cheng, Zhaoyang Yang, Qifeng Chen, and Yu-Wing Tai. Fully convolutional networks for continuous sign language recognition. In *Computer Vision – ECCV 2020*, pages 697–714, 2020.
- [91] NVIDIA. Performance Analyzer CLI — NVIDIA Triton Inference Server. https://docs.nvidia.com/deeplearning/triton-inference-server/a-guide/docs/perf_analyzer/docs/cli.html. Accessed: 2026-05-22.